

PRINT-FLOW-360 · RESEARCH LIBRARY

Feature Rd

7 research documents · each with plain-language key takeaways

Compiled 2026-06-26

Table of Contents

1. Feature Counsel - How to Make Print-Flow-360 More Useful for Non-Technical Shop Owners
2. Make What We Have Better - Feature Improvement R&D
3. Make What We Have Better - Feature Improvement R&D, Round 2 (Wave-5)
4. Make What We Have Better - Feature Improvement R&D (Part 2)
5. Frontstore Design R&D - 2026-06-02
6. Storefront (Customer) R&D - Make the Shopping Journey Easy to Trust & Complete
7. Game-Changer R&D - Strategy for Print-Flow-360

Feature Counsel - How to Make Print-Flow-360 More Useful for Non-Technical Shop Owners

Source: [readme/research/feature-rd/FEATURE_COUNSEL_2026-06-01.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a deep audit of 18 features already built into Print-Flow-360. It was written to help the team understand why non-technical shop owners get confused or stuck — and what to fix first. The central finding is simple: the platform is powerful but speaks “developer language” to people who just want to run a print shop.

The main parts explained simply:

- **Silent lies** — Several buttons and pages look like they work but actually do nothing. The pricing calculator always shows the wrong price (it ignores your rules). Three dashboard “Report” buttons spin and go nowhere. Customer notification switches in the storefront save nothing. Fake phone numbers and fake “Recent Quotes” appear on the customer-facing quote page. These destroy trust and can cost real money — a shop owner believes their price is right, or their customer got an email, when neither is true.
- **Developer words shown to shopkeepers** — Internal codes like “ecommerce”, “hybrid”, “Manual”, “Unposted”, “Slug”, “Option Key”, “Critical”, “Info”, and “void” show up all over the admin. A print-shop owner cannot tell a printed banner (“printing”) from a physical mug (“ecommerce”) just by reading those words. Every label needs to be in plain shop language.
- **Wrong defaults that force extra setup** — Built-in staff roles (Designer, Sales Rep, etc.) are created with zero permissions, so new team members log in to a blank screen and think the app is broken. Guest checkout is turned off by default, so first-time buyers hit a login wall and leave. The daily alert summary email is off by default, so owners miss overdue invoices. The fix is: ship the settings most shops want already turned on.
- **Blank screens that look like “all clear”** — When the internet is slow or a request fails, many pages just show nothing — no message, no retry button. The dashboard “Action Center” widget shows a green “nothing needs attention” even when it failed to load. An overdue invoice behind a failed request looks like no problem at all.
- **No guidance when something isn’t set up** — When a payment gateway isn’t connected, customers can’t pay but the owner only finds out from a complaint. The Stripe “Pay now” button on invoices leads to a dead error page. The webhook URL (needed for paid orders to be marked as paid) is never shown anywhere. Every dead end should explain what’s missing and link to the exact place to fix it.
- **Destructive actions with no warning** — Deleting a product, a staff role, or an automation shows a generic “Are you sure?” with no details about what gets wiped. A single click can erase hours of pricing setup or lock out active staff with no explanation of consequences.
- **Wrong currency on screen** — Stores using euros, rupees, or pounds see dollar signs in several places — on product pages before the price loads, in the admin product list, and in the pricing calculator. Every price must use the store’s actual currency from the first moment.
- **Onboarding that doesn’t finish** — After signing up, no store is created automatically, so the “Get your store ready” checklist points at a store that doesn’t exist. The demo products count as “your first product” so the step appears done when the owner hasn’t added anything real. Placeholder fake stats (“10,000+ orders delivered”, “4.9 rating”) get seeded and can accidentally go live.

- **Pricing engine confusion** — The pricing rules form uses terms like “Adjustment vs Replacement” and strategy names like “Area Based” and “Conditional” that don’t match any goal a shop owner would have. Choosing the wrong default silently doubles the price. The guided fix: offer plain-language choices (“Add to the price” vs “Set the whole price”) and a “What do you want to do?” shortcut for common goals.
- **B2B pricing not showing up** — A logged-in business buyer sees full store prices on every product card even if the owner promised them 15% off. The discounted price only appears deep inside the product configurator, which makes the shop look untrustworthy to corporate clients.
- **Artwork quality hidden from the owner** — Customer files (up to 200MB) land on the order as a bare download link with no check on whether they’re print-ready. A 72-DPI screen-resolution file looks the same as a 300-DPI print file until the job fails on press.

What to do with this:

Fix the “silent lies” first — the fake pricing calculator, dead payment buttons, and non-saving notification toggles destroy trust and can cause real financial errors. Then apply plain-language labels and sensible defaults platform-wide; these are small code changes with outsized impact for every shop owner who uses the product every day.

Generated 2026-06-01 by a multi-agent audit (18 features × Profile → Advise, then synthesis; 37 agents). Lens: every recommendation is judged through the eyes of a **non-technical print-shop owner** (CLAUDE.md §0), and proposes changes to **existing** features - not new modules. **110 recommendations** across 18 features (62 high-priority).

Executive summary

Your platform is feature-rich but leaks its engineering internals to the non-technical shop owners it serves. The audit found the same five problems repeating across nearly every feature: (1) raw developer values shown as labels (enum codes like “ecommerce”/“hybrid”, “Slug”, “Option Key”, “items_sum_r_o_u_n_damount_tax2”, severity words like “Critical/Info”, strategy-class names); (2) controls and pages that look functional but silently do nothing (a pricing calculator that always returns the base price, three dashboard Report buttons that go nowhere, storefront notification toggles with no API, a Stripe invoice path that 404s, fake “\$555-123-4567” contact data and fake “Recent Quotes”); (3) defaults that punish the owner (built-in roles seeded with ZERO permissions so new staff see a blank app, guest checkout off so first buyers hit a login wall, the daily action-summary email off, engagement columns hidden); (4) missing loading/empty/error states so a failed fetch reads as “all clear” or “broken”; and (5) no guided setup - no store auto-created at signup, no webhook URL shown, no “test connection”, destructive deletes with no stated consequences. The throughline: owners cannot tell what is broken, what is fake, what is safe to click, or where to finish a task. The highest-leverage work is not new features - it is making the powerful features you already shipped legible, honest, and safe for someone who has never used software like this. Fix the silent lies first (they destroy trust and lose real money), then ship plain-language

labels and sensible defaults platform-wide, then invest in guided onboarding and the calculator/payment correctness that make the product genuinely worth paying for.

Cross-cutting themes

Patterns that recur across many features - fixing these once pays off platform-wide.

1. Plain-language labels everywhere - stop showing developer values to shopkeepers

Owners cannot make decisions when the screen speaks in enum codes, strategy-class names, and bookkeeping jargon. The same internal strings ('ecommerce', 'hybrid', 'Slug', 'Option Key', 'Critical'/'Info', 'Manual', 'void', 'Unposted', 'authorizenet', 'minor-third', 'single-page/accordion', 'items_sum_r_o_u_n_damount_tax2', 'Material Library', 'Client Page Link') surface unchanged across the catalog, pricing, orders, quotes, invoices, customers, action center, themes, designer and AI builder. A consistent plain-language label map (and reusing the helpers you already have, e.g. `statusLabel()`, `customerTypeBadge()`) makes the product read as finished and trustworthy.

Spans: Products & Catalog, Pricing Engine & Calculator, Orders & Print Jobs, Quotes, Invoices, Payments & Accounting, Customers & Activity, Action Center, Storefront Themes & CMS, Storefront Cart & Checkout, Designer Studio, AI Product Builder, Roles & Permissions (Team Access Control)

2. Kill the silent lies - no control or page may look functional while doing nothing or showing fake data

Nothing erodes trust faster than a button that pretends to work. The admin pricing calculator always returns the un-discounted base price (false confidence on real prices), three dashboard Report buttons spin then go nowhere, the storefront notification toggles persist nothing, the Stripe invoice link POSTs to a non-existent route, an email bug silently drops subscribed recipients, automations fire into a void on mismatched status IDs, and the storefront quote pages show a fake US phone number and fake 'Recent Quotes'. Each of these makes a careful owner ship a wrong price, miss a payment, or apologise for a page they didn't know existed. Fix or feature-flag-hide every one.

Spans: Pricing Engine & Calculator, Dashboard & Reporting, Notifications & Automation, Invoices, Payments & Accounting, Quotes, PDF / Artwork Pipeline

3. Sensible defaults so nobody has to configure to get a working store

Your [CLAUDE.md](#) §0 demands defaults set to what 90% want, but the platform repeatedly ships the opposite. Built-in staff roles are seeded with zero permissions (new hire sees a blank app), no store is created at signup (the whole setup checklist points at a store that doesn't exist), guest checkout is off (first-time buyers hit a login wall and abandon), the daily Action Center email is off (owners who log in twice a week miss overdue invoices), customer engagement columns are hidden, and the setup checklist counts seeded demo products as the owner's 'first product'. Right defaults turn a confusing first session into a working store.

Spans: Roles & Permissions (Team Access Control), Tenant Onboarding & Setup, Storefront Cart & Checkout, Action Center, Customers & Activity

4. Mandatory loading, empty, and error states - a blank or green screen must never mean 'failed'

Multiple data views swallow errors in console.error and render as empty or 'all caught up', which for money and production is dangerous: the dashboard Action Center widget shows a reassuring green check when the fetch failed, invoice/quote/metric widgets show empty tables indistinguishable from 'no data', the job detail page renders fully blank on a timed-out session, and roles save/load failures show nothing. Owners on flaky shop wifi need to know whether data is real or unverified, with a one-click retry.

Spans: Dashboard & Reporting, Action Center, Orders & Print Jobs, Roles & Permissions (Team Access Control), Customers & Activity, Designer Studio, PDF / Artwork Pipeline

5. Guided setup and unconfigured-integration guidance - never leave a dead end, always point to the fix

Owners get stranded at the exact moments that cost sales: no payment gateway connected (customers can't pay and the owner only learns from a complaint), the webhook URL is never shown (paid orders stick on 'Pending' forever), there's no 'Test connection' so a fat-fingered Stripe key surfaces at a real customer's checkout, a new storefront quote arrives with no alert, and the Print File Checker 503s with a raw server error. Every dead end should follow your \$0 pattern: explain in plain language what's missing and link to where to fix it.

Spans: Integrations & Payment Gateways, Storefront Cart & Checkout, Quotes, PDF / Artwork Pipeline, Tenant Onboarding & Setup, B2B / Business Accounts

6. Destructive actions and money actions must state consequences and capture the real record

Your \$0 rule on confirmations is violated repeatedly, and the cost is lost work and broken books: product/role/automation deletes and the one-click 'Remove samples' wipe configured work with a generic 'Are you sure?'; refunds don't state the amount or where the money goes; and 'Mark as Paid' flips a status without recording the amount, method, or date - so a disputed cash payment has no proof. Naming exactly what's lost (or captured) lets a non-accountant owner act confidently.

Spans: Products & Catalog, Roles & Permissions (Team Access Control), Notifications & Automation, Invoices, Payments & Accounting

7. Currency correctness - one store, one currency, from first paint

Hardcoded USD ('\$ ', Intl 'en-US', currency=>'USD') leaks into the product-page price flash, the admin product list (two columns showing different symbols), the Top Products chart, the pricing calculator, and the AI builder result - so euro/rupee/pound shops see dollar signs on their own money and distrust every number. Routing all prices through the existing tenant-aware formatter is a small, repeated fix with outsized trust impact.

Spans: Pricing Engine & Calculator, Products & Catalog, Dashboard & Reporting, AI Product Builder

Quick wins (small effort, high client value - do these first)

#	Feature	Change	Client benefit
1	Pricing Engine & Calculator	Fix or hide the admin Pricing Calculator that always returns the base price - The 'Test your pricing rules' widget is a stub: calculate() waits 500ms then always returns {final_price: base_price, rules:[]} and even shows a green 'no rules applied' panel as if that were the real answer. Wire it to the real ProductPricingCalculator the storefront already uses, or feature-flag it hidden until wired. Also replace its hardcoded '\$' with the tenant currency formatter.	An owner who set up '100+ flyers = \$0.08 each' currently gets false confidence - a calculator that silently confirms the wrong, un-discounted price - and ships a mispriced product. Honest output (or no widget) prevents real pricing mistakes going live.
2	Dashboard & Reporting	Make the three dashboard Report buttons go somewhere - or remove them - Sales/Customer/Inventory Report buttons spin a fake 1s timer then do nothing (no @click, no route). Point them at existing views (paid invoices, customers, products/stock) or gate the dead ones out entirely.	On the very first screen after login, three official-looking buttons that do nothing read as a broken product. One-click shortcuts to real data turn dead rows into useful navigation.
3	Quotes	Remove the fake contact data and fake 'Recent Quotes' from storefront quote pages - The /quote calculator hardcodes '+1 (555) 123-4567', 'quotes@printflow360.com', and a dummy Business Cards/Flyers 'Recent Quotes' list; /profile/quotes shows Paid/Void tabs the workflow never produces. Swap in the tenant's real store settings and the customer's real quotes (with empty states), and drop the dead tabs.	A customer landing on a page showing a phone number that isn't the shop's and prices that aren't theirs makes the shop look fraudulent. Removing fake data means the owner never apologises for a page they didn't know existed.
4	Action Center	Turn the daily Action Center email ON by default and add a sidebar entry with a live count - Default the daily summary ON (07:30) with a one-line dismissible confirmation banner, and add a named 'Action Center' sidebar item with the live count badge the header already polls (instead of only an unlabelled bolt icon).	An owner who logs in twice a week currently never discovers the digest (buried two clicks deep, off by default) and misses overdue invoices on the days they're away. On-by-default plus a labelled sidebar link means what's on fire reaches them whether or not they log in.
5	Orders & Print Jobs	Make 'Jobs Late' and order/quote status colours reflect reality - 'Jobs Late' counts every past-due job with no status filter, so finished jobs inflate it - exclude terminal statuses. Separately, the order/quote lists use hardcoded colour switches that paint custom and 'on_hold' statuses red/grey; drive badges from the tenant's own per-status hex colours (which the detail page already reads) and run labels through statusLabel().	An owner triaging at 9am sees 'Jobs Late: 12' but can't tell how many still need rescuing; and their carefully colour-coded statuses show wrong colours. Both fixes let them trust the board at a glance.

#	Feature	Change	Client benefit
6	Products & Catalog	Replace raw severity/type/status labels with shopkeeper words using maps you already have - Add one shared label map and reuse existing helpers: product types (Printing → 'Print Product', ecommerce → 'Physical Product', hybrid → 'Print + Physical'), Action Center severities (Critical → 'Urgent', Info → 'Good to know' - matching the email), customer 'Manual' → 'Added by store' (customerTypeBadge already does this), order 'void' → 'Cancelled' (statusLabel already does this). Apply across list badges, dropdowns and detail headers so all surfaces match word-for-word.	Owners scanning their catalog, customer list, or alert feed currently read lowercase database codes and can't tell a printed banner from a physical mug, or whether 'Info' matters. Plain words make every list instantly readable.
7	Pricing Engine & Calculator	Fix the dollar-sign flash and the two-formatter currency disagreement - formatProductPrice() hardcodes '\$' and is the pre-calculation fallback, so euro/rupee stores flash '\$0.00'; the admin product list mixes a tenant-aware formatMoney() with a local USD-hardcoded formatPrice() so two columns show different symbols. Pull the store currency from useThemeSettings() (or render a neutral skeleton) and route every column through the existing tenant-aware formatter.	A euro shop watching its own product page flash a dollar sign and its admin list show '\$' and '€' in the same table reads the app as broken. Consistent currency from first paint earns trust.
8	Products & Catalog	State consequences on every destructive confirmation and name what's lost - Product delete, role delete, automation delete, the one-click 'Remove samples' bulk delete, and the refund popup all use a generic 'Are you sure?'. Name the item and spell out the loss ('permanently delete Premium Business Cards including its sizes, options, pricing rules and images, and remove it from your storefront'; 'remove access for the 3 team members using this role'; 'refund \$120.00 to the card via Authorize.Net '). Restate the action on the button ('Delete Product', 'Record \$X Payment').	Owners cleaning up their catalog or roles today wipe hours of configured work, lock out working staff mid-shift, or double-refund a customer with one careless click. Stated consequences let them act confidently and stop costly mistakes.
9	Customers & Activity	Show engagement columns and actionable empty states on the Customers page by default - The seeded default view omits Last Order, Total Spent, Orders, Last Login (the cells already render); add them to the seeded view plus a back-fill for older tenants. Replace '(N/A)' detail placeholders with 'No email added - click to add', and self-heal a missing view instead of showing 'Invalid view' on a blank table.	An owner opening Customers to find their best repeat customers sees a plain address book with no spend/order data and a cryptic config icon - or an 'Invalid view' error that reads as lost data. Surfacing the engagement data they were promised, the moment the page loads, is the whole point of the feature.

Biggest impact (larger, most worth-paying-for)

#	Feature	Change	Client benefit
1	Tenant Onboarding & Setup	Auto-create the first store at signup and make onboarding a finishable, honest path - No store exists after registration, so the 'Get your store ready' checklist points at a store that doesn't exist - every step silently reports incomplete and links go nowhere. Auto-create one primary store (named after the business) and run <code>StoreOnboardingService::seedAll()</code> at approval, count only non-demo products toward 'add your first product', and seed honest neutral defaults instead of fake '10,000+ Orders Delivered'/'4.9 rating' stats and dead facebook.com social links that would otherwise go live verbatim.	A new owner's first session today is 'figure out why the buttons are broken' on a dashboard whose checklist can never complete, with a risk of publishing fake trust numbers. Instead it becomes 'add my logo and a product' on a working, seeded store - the difference between a trial that converts and one that bounces.
2	Invoices, Payments & Accounting	Make payments actually work end-to-end: connect Stripe correctly, show the webhook URL, add Test Connection, and record real payments - Close the payment trust gaps as a set: wire the Stripe invoice button into the working multi-gateway abstraction (today it POSTs to a non-existent route and dead-ends the customer); show a copy-able per-store webhook URL with a 'paste this in Stripe or paid orders stay Pending' helper; add a 'Test connection / Send test' button so bad keys surface before a real checkout; block activating a half-configured or sandbox-in-production gateway; warn the owner in admin when NO payment method is connected; and make 'Mark as Paid' open a Record Payment dialog (amount, method, date) that creates a real payment row and feeds the accounting totals/CSV (which today exclude all invoice payments).	Every one of these is a place where an owner believes they can take money but silently can't - a dead Stripe link, a stuck-on-Pending order, a rejected key found only at checkout, a green invoice with no payment record, or an accounting export that omits all cash and cheque. Getting paid reliably and reconciling the books is the core promise of a commerce SaaS; this is what makes it worth paying for.
3	Pricing Engine & Calculator	Make the pricing engine usable by a shopkeeper: guided goal-first rules, correct defaults, and a real calculator - The pricing engine is built for power users and quietly mis-prices for everyone else. Add a guided 'What do you want to do?' picker (bulk discount / rush fee / price by square foot) that pre-loads the right template; relabel strategies and the Adjustment-vs-Replacement choice in business terms ('Add to the price' / 'Set the whole price') with worked examples and correct per-type defaults (so a bulk tier doesn't silently double the price); add a simple inline 'quantity discount' table on the Pricing tab so common	Pricing is the heart of a print shop and today it's a minefield: leaving a tier on the default 'Adjustment' silently doubles the price, the strategy names don't map to any goal, and the calculator that

#	Feature	Change	Client benefit
		bulk pricing never touches the rules engine; label tier columns 'Price each' vs 'flat price for the batch' with a sample row; and wire the calculator to the real engine. Mirror admin ↔ storefront per PRICE_CALCULATOR_SYNC_GUIDELINES.	should catch the mistake lies. A guided, plain-language pricing flow with correct defaults means owners price right the first time instead of discovering errors from a customer complaint.
4	Roles & Permissions (Team Access Control)	Pre-fill built-in staff roles with sensible permissions so new hires aren't locked out - All seven built-in roles are seeded with ZERO permissions, so a new Designer/Sales Rep logs in to a blank nav that reads as a broken app - and the owner gets no prompt that they were supposed to configure the role first. Seed sensible defaults per role (Designer: jobs/tasks editable, quotes visible; Sales Rep: quotes/customers editable, invoices visible; PM: visibility across orders/jobs/customers), keep them editable, tell the owner the new hire gets an email to set their own password, fix the edit-role screen that says 'Create New Role' and greys out the name, and stop offering access tiers the backend silently ignores (mark admin-only modules 'Admin only').	The first time an owner adds a staff member is a make-or-break moment; today that person is locked out of everything with no clue why, and the owner can't even correct a typo'd role name or trust that the toggles they set do anything. Working defaults and an honest editor make team access something a non-technical owner can actually use.
5	B2B / Business Accounts	Make the B2B promise hold across the storefront and let owners onboard companies in one sitting - A logged-in B2B buyer sees full B2C prices on every product card because StorefrontProductResource returns raw base_price - the catalog visibly contradicts 'your prices are 15% lower' until they open the configurator. Apply CompanyPricingResolver on the listing path (the cache key already includes :company:{id}), show the negotiated price with the original struck through; let the owner create a brand-new customer directly from the Contacts tab (today search-only dead-ends on anyone who's never shopped); replace the error-prone signed manual-adjustment field with a Charge/Credit toggle and live plain-language preview; surface a 'set a store up for business first' guided path; and clearly mark budget/role fields as informational-today so owners don't expect un-enforced spend caps.	B2B is a premium, higher-revenue capability, and today it undermines the owner's own sales pitch (wrong prices on the catalog), forces a clunky 'go self-register first' onboarding, and invites backwards ledger entries. Making the negotiated price show everywhere and letting an owner stand up a corporate account during the sales call is what makes B2B genuinely sellable.
6	PDF / Artwork	Close the artwork/PDF quality and print-readiness gaps so jobs don't fail on press - Customer-uploaded files (up to 200MB, no	In a print business, a low-res file caught after

#	Feature	Change	Client benefit
	Pipeline	checks) appear in the order as a bare download link with no quality info - the owner finds a 72-DPI RGB or wrong-size file only after starting production. Surface a print-readiness badge (green 'Print-ready' / amber 'Check before printing' / red 'Not print-ready') driven by the preflight service, with a 'Quality not checked' chip when the service is off. Label the designer 'Download PDF' by quality (or route it to the 300-DPI print-grade endpoint) so nobody ships a blurry web-grade file as the deliverable, and make the Print File Checker and preview-503 states explain themselves with retry instead of raw server errors.	plates are made is wasted ink, reprints, and an angry customer. A readiness badge on the order screen and an honestly-labelled print-grade PDF turn 'discover the problem in production' into 'catch it before you start' - directly protecting the owner's margin and reputation.

Per-feature recommendations (full appendix)

- Products & Catalog (6)
- Pricing Engine & Calculator (6)
- Orders & Print Jobs (6)
- Quotes (6)
- Invoices, Payments & Accounting (7)
- Customers & Activity (6)
- B2B / Business Accounts (6)
- Roles & Permissions (Team Access Control) (7)
- Notifications & Automation (6)
- Action Center (6)
- Tenant Onboarding & Setup (6)
- Integrations & Payment Gateways (6)
- Storefront Themes & CMS (6)
- Storefront Cart & Checkout (6)
- Designer Studio (6)
- AI Product Builder (6)
- PDF / Artwork Pipeline (6)
- Dashboard & Reporting (6)

Products & Catalog

- **Show human product-type names instead of raw enum values everywhere they appear** -  High effort S

- *Today:* The product list (nuxt/app/pages/products/index.vue line 147 table badge and line 365 grid badge) renders `row.product_type || row.type` verbatim, so a shop owner sees 'printing', 'ecommerce', 'tshirt', 'hybrid' across their whole catalog. These are internal database codes, not words a print-shop owner uses. The same internal style also leaks into pricing-rule type badges (e.g. 'Area Based', 'Quantity Based').
 - *Change:* Add a single shared label map (Printing -> 'Print Product', tshirt -> 'Apparel / T-Shirt', ecommerce -> 'Physical Product', service -> 'Service', hybrid -> 'Print + Physical') and render it in both the table and grid badges, and reuse the same map in the General tab dropdown labels so the create-time choice and the list badge match word-for-word. The dropdown in General.vue (lines 149-156) already has friendlier-ish labels but they still say 'Ecommerce'/'Hybrid' - align all three surfaces to the plain-language map.
 - *Why it helps the owner:* A store owner scanning their catalog to find 'that mug I sell' today reads a column of lowercase developer codes and cannot tell a physical mug ('ecommerce') from a printed banner ('printing'). Instead they would read 'Physical Product' vs 'Print Product' and instantly know which row is which without guessing what 'hybrid' means.
- **Make the delete confirmation state exactly what is lost and restate the action on the button - ●**
High · effort S
 - *Today:* Utils/ConfirmationDialog.vue renders a generic 'Are you sure you want to delete this product?' (line 37-39) with a plain 'Delete' button. It never tells the owner that the product's options, sizes, pricing rules, and gallery go with it, that it is removed from the storefront, or that it cannot be undone. This directly violates the [CLAUDE.md](#) destructive-action rule.
 - *Change:* Pass the product name into the dialog and render a consequence line: "This will permanently delete 'Premium Business Cards', including its sizes, options, pricing rules and images, and remove it from your storefront. This cannot be undone." Change the confirm button label from 'Delete' to 'Delete Product'. Apply the same to UtilsDeleteEntity used in the table so both paths match.
 - *Why it helps the owner:* A store owner cleaning up their catalog today clicks Delete on a generic 'Are you sure?' and may not realize they just wiped every paper-type option and quantity discount they spent an hour configuring. Instead the dialog names the product and spells out what disappears, so they delete confidently and never lose pricing work by accident.
 - **Replace the one-click 'Remove samples' bulk delete with a counted, named confirmation - ●**
High · effort S
 - *Today:* The demo-products banner (index.vue lines 41-50) fires `removeAllDemoProducts` on a single click and deletes every sample product via `Promise.all` (line ~448) with no confirmation and no undo. A misclick on a red button wipes all the pre-built, ready-to-customize sample products at once.
 - *Change:* Gate 'Remove samples' behind the same consequence dialog: "Remove all 6 sample products? These pre-configured examples (Business Cards, Banners, ...) will be permanently deleted. You can always add your own from scratch." with a 'Remove Samples' confirm button. Keep the count and ideally list the names being removed.

- *Why it helps the owner:* A new owner exploring the catalog the first day often clicks the prominent red 'Remove samples' to tidy up - and today instantly loses the realistic, pre-priced examples that were the easiest way to learn how a product is built. Instead they get one confirmation showing what and how many will go, so they can change their mind before destroying their best learning material.
- **Hide the 'Slug' and 'Option Key' technical identifiers and rename the 'Gen' button** - ● High · effort M
 - *Today:* General.vue line 11-14 shows a prominent 'Slug' field ('Leave blank to auto-generate'); the option editor (Upsert.vue) shows an 'Option Key' field with hint 'Unique identifier for this option'; and the SKU field has an unexplained 'Gen' button (General.vue line 21). 'Slug', 'Option Key', and 'Gen' are developer terms a print-shop owner cannot decode.
 - *Change:* Collapse the Slug field into an 'Advanced' disclosure (or remove from the form and auto-generate silently from the name, exposing it only as a read-only 'Web address' under SEO). Auto-generate the Option Key silently and remove it from the option form entirely. Rename the SKU button from 'Gen' to 'Generate' with a tooltip 'Create a SKU for me'.
 - *Why it helps the owner:* An owner adding their first product today hits a required-looking 'Slug' box and a cryptic 'Option Key' and either freezes or types garbage into a field that controls their public URLs. Instead the form shows only fields they understand (Product Name, Price, Type), the system handles the web address and keys behind the scenes, and 'Generate' clearly offers to make a SKU for them.
- **Add a guided 'Quantity discount' shortcut so simple bulk pricing doesn't require the pricing-rules engine** - ● Med · effort L
 - *Today:* The most common thing a print shop wants - 'cheaper per unit when you order more' - currently means leaving the Pricing tab, opening the dedicated pricing-rules form, and choosing among developer-framed types (Fixed, Percentage, Formula, Area Based, Conditional) plus an 'Applies To' selector and a numeric 'Priority' field. The mental model is built for power users, not a shop owner who just wants '500 cards = \$40, 1000 = \$70'.
 - *Change:* On the Pricing tab (PricingHub.vue), add a prominent 'Add a quantity discount' card that opens a simple inline table: quantity from / to and price (or % off), with an 'Add tier' row. Generate the underlying Quantity Based rule from it automatically - hiding priority, Applies To, and formula syntax. Keep the full rules page as an 'Advanced pricing' link for the rare complex case.
 - *Why it helps the owner:* A store owner who wants to reward bigger orders today must learn what 'Area Based', 'Conditional', and rule 'Priority' mean just to set a 10%-off-at-1000 tier, and many will give up or price it wrong. Instead they fill a plain quantity-and-price table right where they set the base price, and their bulk discount works without ever seeing a formula or a priority number.
- **Let Gallery and Inventory auto-save the draft instead of blocking with a 'Save the product first' wall** - ● Med · effort M
 - *Today:* Gallery and Inventory tabs show a blocking 'Save the product to enable gallery uploads / inventory management' alert when the product is unsaved, with no inline save action. A new owner's instinct is often to add photos first, so they jump to Gallery, hit a dead-end message, and must

navigate back to General, save, then return - all subsequent tabs are also disabled via the opacity-50 + pointer-events-none guard.

- *Change:* When a user opens a save-gated tab on an unsaved product, either (a) auto-create the draft in the background on first interaction, or (b) replace the blocking alert with an inline 'Save product to start uploading images' button that saves the General tab and stays on Gallery. Either way, never leave the owner to figure out that the fix lives on a different tab.
- *Why it helps the owner:* An owner adding their first product naturally clicks 'Gallery' to upload the product photo first - and today hits a wall telling them to save, with no button to do it, so they bounce between tabs confused about why nothing works. Instead the draft saves itself (or one inline button saves it) and they upload the photo right where they expected to.

Pricing Engine & Calculator

- **Make the admin Pricing Calculator actually calculate (kill the fake stub or hide it)** - ● High · effort M
 - *Today:* The 'Test your pricing rules' calculator on the pricing-rules guide page (nuxt/app/components/admin/product/shared/PricingCalculator.vue) looks fully functional - width/height/quantity/base-price inputs and a 'Calculate Price' button - but calculate() at lines 177-205 is a stub. It waits 500ms via setTimeout, then always returns { final_price: inputs.base_price, rules: [] }. It never calls the API, so it ignores every tier, area rate, formula, and combination the owner set up, and even shows a reassuring green 'No rules applied / base price is being used' panel as if that were the real answer.
 - *Change:* Replace the stub with a real call to the existing storefront calculator. Wire calculate() to POST the product UUID + inputs to an admin-side endpoint backed by the same ProductPricingCalculator::calculate() the storefront uses (app/Services/Pricing/ProductPricingCalculator.php), and render the returned breakdown array (the same line items the customer sees). Until that endpoint is wired, do NOT ship the widget visibly - gate it behind a flag so owners never see a calculator that lies. Also replace the hardcoded '\$' in its formatPrice() (line 208) with the tenant currency formatter.
 - *Why it helps the owner:* A shop owner who sets up '100+ flyers = \$0.08 each' and wants to confirm 250 flyers costs \$20 before going live must today either trust a calculator that silently answers with the un-discounted base price (false confidence - they ship a wrong price), or abandon it and go to the live storefront, pick options, and read the price panel. Instead they could type 250 into the calculator on the same page where they built the rule and see the real \$20 with the tier line itemized.
- **Rename 'Adjustment vs Replacement' to a plain-language choice with a worked consequence** - ● High · effort M
 - *Today:* On the rule form (pricing-rules/form.vue lines 159-178) the 'Rule Mode' radio offers 'Adjustment' and 'Replacement' with a help tooltip written for developers: 'Replacement sets the subtotal to a specific value - only the highest-priority replacement applies.' It never states the business outcome. An owner who builds a Quantity Based tier rule and leaves it on the default 'Adjustment' gets the tier price ADDED on top of the base subtotal instead of replacing it - producing a price roughly double what they expect - and no error is raised.

- *Change:* Relabel the two options in business terms: 'Add to the price (use for fees, surcharges, discounts)' and 'Set the whole price (use for tier/bulk prices and fixed overrides)'. Replace the tooltip with one concrete worked example for the rule type currently selected, e.g. for a Quantity Based rule: 'Base price \$0.10 + your tier \$0.08. Add to the price = \$0.18/unit. Set the whole price = \$0.08/unit. For bulk tiers you almost always want Set the whole price.' Default Quantity Based and Fixed Price rules to 'Set the whole price', and Percentage/Formula/Area to 'Add to the price'.
- *Why it helps the owner:* A shop owner offering bulk pricing today must understand that a tier is technically a 'Replacement' or their quantity discount stacks on the base and prints double - a mistake invisible until a customer complains. Instead the default would already be correct for a tier, and the choice would read in the language of their actual goal.
- **Replace strategy-type developer labels with goal-first labels and add a 'what do you want to do?' guided picker** - ● High · effort M
 - *Today:* The strategy dropdown (form.vue lines 494-499) lists 'Fixed Price', 'Percentage', 'Formula', 'Area Based', 'Quantity Based', 'Conditional' - named after the engine's strategy classes, not the owner's intent. An owner who wants '10% off for orders over 50' has no obvious entry: it could be Percentage, or Quantity Based with tiers. There is no path from the business goal to the right type; they must read the right-hand help panel to reverse-engineer it.
 - *Change:* Relabel the options goal-first: 'Bulk / quantity discount' (quantity_based), 'Percentage markup or discount' (percentage), 'Price by size / area, e.g. banners' (area_based), 'Fixed price for this option set' (fixed), 'Custom formula (advanced)' (formula), 'Price for a specific option combination' (conditional). Above the dropdown, add a short one-line picker - 'What do you want to do?' - with the three most common goals (bulk discount, rush fee, price by square foot) as clickable chips that select the matching type AND pre-load the corresponding template from the existing 8 templates.
 - *Why it helps the owner:* A shop owner trying to add a rush-order surcharge today must read each strategy's help text to learn it is a 'Percentage' or 'Fixed' rule; instead they click a 'Rush order fee' chip and the form arrives pre-shaped for that exact task.
- **Fix the dollar-sign flash and the two-formatter currency disagreement so non-USD stores look correct** - ● High · effort S
 - *Today:* Two hardcoded-USD bugs hit non-dollar stores. (1) frontstore/app/utils/productHelpers.ts formatProductPrice() (lines 148-151) hardcodes '\$' + amount.toFixed(2); it is the fallback shown on the product page before the first calculation returns, so a euro or rupee store flashes '\$0.00' for a beat - looking broken to the customer. (2) The admin product list (products/index.vue) mixes formatMoney() (tenant-currency-aware) at lines 166/171/175 with a local formatPrice() at line 529 that hardcodes Intl.NumberFormat('en-US',{currency:'USD'}), so two columns in the same table can show different currency symbols for the same store.
 - *Change:* Make formatProductPrice() take the store currency symbol from useThemeSettings() (the same source ProductPriceSummary already uses correctly) instead of a literal '\$', or render a neutral skeleton placeholder instead of a fake \$0.00 until the real result arrives. In products/index.vue, delete the local USD-hardcoded formatPrice() and route every price column through the existing tenant-aware formatMoney() with the store symbol.

- *Why it helps the owner:* A shop owner running their store in euros today watches their own product page flash a US dollar sign before the real price loads, and sees their admin product list show '\$' in one column and '€' in another - both read as a broken/untrustworthy app to them and their customers. Instead every price shows their actual currency, consistently, from first paint.
- **Label quantity tiers and area units in shop terms and pre-fill one example row** - ● Med · effort M
 - *Today:* Quantity tier rows (form.vue lines 263-277) use raw column names 'From Quantity', 'To Quantity', 'Unit Price', 'Total Price (optional)' with an empty grid and no example. The crucial Unit-Price (per item) vs Total-Price (flat for the whole range) distinction is buried in a tooltip; picking the wrong column produces a price off by a factor of the quantity with no warning. Separately, the area unit dropdown labels read 'Square Feet (sq_ft)' but after selection the field surfaces the internal code 'sq_ft' rather than 'per sq ft'.
 - *Change:* Rename the tier columns to 'Order at least', 'Up to', 'Price each' and 'Or flat price for the batch (optional)', and seed the empty tiers grid with one greyed example row (e.g. 1 / 49 / \$0.10 / blank) shown as a placeholder so the owner sees the shape. Add an inline note on the row: 'Fill EITHER Price each OR flat price - not both.' For area units, drop the internal codes from the visible label entirely ('Square feet', 'Square inches', ...) and render the chosen unit back to the owner as 'per sq ft' wherever the rate is echoed.
 - *Why it helps the owner:* A shop owner pricing 500 business cards at 8 cents each today can accidentally type 0.08 into 'Total Price' and unknowingly charge 8 cents for the whole batch (or type a batch price into 'Unit Price' and charge 500x) with nothing flagging it; instead plain 'Price each' vs 'flat price for the batch' labels plus a sample row make the right field obvious before they ever save.
- **Add guardrails for conflicting rule priority so an ignored rule never fails silently** - ● Med · effort M
 - *Today:* Priority is a raw number the owner edits both by drag-reorder on pricing-rules/index.vue AND in a separate Priority field on form.vue - so editing a rule after dragging can silently contradict the dragged order. Worse, two 'Set the whole price' (replacement) rules at the same priority resolve to only one winning silently; the other rule the owner carefully built simply never fires, with no warning anywhere.
 - *Change:* Remove the manual Priority number field from the form and make drag-order on the list the single source of truth (the list already writes priority on reorder). On the list page, detect when two or more 'Set the whole price' rules could both apply to overlapping conditions and show an inline amber warning on the losing rule: 'This rule is overridden by [other rule] and will never apply. Reorder or change its mode.' - matching the integration-warning banner pattern in [CLAUDE.md](#).
 - *Why it helps the owner:* A shop owner who creates both a 'Premium material override' and a 'Bulk tier' as full-price rules today gets one of them silently ignored and can only discover it by noticing a wrong price on a live order; instead the offending rule carries a plain-language warning telling them exactly which rule wins and how to fix it.

Orders & Print Jobs

- **Make the order-list status colours match what the owner actually set** - ● High · effort S

- *Today:* The order list (nuxt/app/components/order/List.vue, getStatusColor at lines 453-468) uses a hardcoded colour switch: 'open' renders RED, and any status with no case (including the paused 'on_hold' flag and every custom status the owner created) falls through to gray. So a shop owner who deliberately colour-coded 'Open' as blue in their status settings, and who set up their own statuses, sees the list paint them red or grey instead. The list silently contradicts the detail page, which correctly reads the tenant's hex colours from the status API.
- *Change:* Delete the hardcoded getStatusColor switch and drive the list badge from the same per-status hex colour the detail page already pulls from GET /order-statuses (apply it via inline style on the badge, with a sensible neutral fallback only when a status has no colour set). Keep the human label coming from statusLabel(). Add a distinct treatment for 'on_hold' (e.g. amber) so a paused order is visually different from a cancelled one.
- *Why it helps the owner:* A store owner who set 'Open' to blue and named a custom 'Awaiting Artwork' status in pink expects the order list to show those exact colours, but today the list shows red and grey and looks broken; instead the list would mirror their own colour scheme everywhere, so they can scan the board and trust the colours mean what they decided.
- **Stop the 'Jobs Late' KPI from counting finished jobs** - ● High · effort S
 - *Today:* In app/Http/Controllers/Api/Job/PrintJobController.php (getKpiDetails, line 282) the 'Jobs Late' count is PrintJob::whereDate('due_date','<',now())->count() with no status filter at all. A job that was delivered last month but had a past due date still adds to 'Jobs Late'. The number a shop owner glances at to decide what is on fire is inflated by work that is already done.
 - *Change:* Exclude terminal statuses (Fulfilled/Completed/Delivered and Cancelled, resolved by JobStatus id the same way the In-Progress KPI already resolves completed status ids) from the jobsLate query, so it counts only past-due jobs that are still open. Apply the identical fix anywhere 'Jobs Late' is computed for the calendar/Kanban badge.
 - *Why it helps the owner:* A shop owner who opens /jobs in the morning and sees 'Jobs Late: 12' today can't tell how many actually still need rescuing versus how many are old finished jobs, so the number is useless for triage; instead 'Jobs Late' would show only genuinely-late open jobs, so the owner knows exactly how many fires to put out before lunch.
- **Give the print-job detail page a real error state instead of a blank screen** - ● High · effort S
 - *Today:* In nuxt/app/components/job/view.vue the data-fetch catch block (lines 800-802) only does console.error and leaves the page empty. If a shop owner's session expired or the network hiccupped, the job page renders completely blank: no message, no retry, nothing to click. This also violates the mandatory loading/empty/error rule in [CLAUDE.md](#) section 0.
 - *Change:* Add an error state to the job detail page mirroring the order detail page: on fetch failure set an error flag, show a plain-language panel ('We couldn't load this job. Check your connection and try again.') with a Retry button that re-runs the fetch, and a 'Back to Jobs' link. Keep the existing skeleton for the loading phase.
 - *Why it helps the owner:* A shop owner whose login timed out while checking a job today just sees a white screen and assumes the job was deleted or the app is broken; instead they would see a clear

message and a Retry button, so they recover in one click instead of calling support or re-creating the job.

- **Hide the raw database alias from the order list's Total column settings and filters** - ● Med · effort S
 - *Today:* The grand-total column uses the key 'items_sum_r_o_u_n_damount_tax2' (List.vue lines 292, 295, 539). The displayed cell label is 'Total', but the moment a shop owner opens the column-visibility/Views config or a sort/filter control, this raw aggregate alias leaks into the UI as the field identifier - exactly the kind of internal token [CLAUDE.md](#) section 0 forbids.
 - *Change:* Decouple the human-facing column from the storage key: keep the API field name internally but give the column a stable, plain key/label of 'total' / 'Total' wherever it surfaces in the column picker, Views modal, and sort/filter UI. Map back to the raw aggregate only at the data-read layer so nothing user-visible ever shows 'items_sum_r_o_u_n_damount_tax2'.
 - *Why it helps the owner:* A store owner customising which columns appear, or sorting by order value, currently sees a garbled developer string and second-guesses whether they're touching the right field; instead they'd only ever see 'Total', so configuring their order view feels safe and finished.
- **Run the order detail status badge through the plain-language label map** - ● Med · effort S
 - *Today:* The status stepper header on the order detail page renders the raw DB status (e.g. {{ order.status }}) directly. A tenant whose status key is 'void' sees the word 'void' in the header badge, even though statusColor.ts already maps void -> 'Cancelled' and the order list uses that mapping. The same order reads 'Cancelled' in the list but 'void' on its own detail page.
 - *Change:* Wrap the detail-page status badge (and any other place that prints order.status raw) in the existing statusLabel() helper from nuxt/app/utils/statusColor.ts, so 'void' shows as 'Cancelled', 'on_hold' as 'On Hold', etc., matching the list. Keep the tenant's custom hex colour as-is.
 - *Why it helps the owner:* A shop owner who cancelled an order sees it labelled 'Cancelled' in the list but 'void' on the order's own page today, which makes them wonder if it's two different states; instead both screens say 'Cancelled', so the order's status is unambiguous wherever they look.
- **Surface unread store messages to customers with a badge instead of a collapsed section** - ● Med · effort M
 - *Today:* On the storefront order detail (frontstore/app/pages/profile/orders/[id].vue) the 'Updates & Messages' section is collapsed by default with no count, so a customer who just got a message or proof from the shop has zero visual cue. They'd have to scroll down and expand it to discover the shop is waiting on them - so replies stall and the shop owner ends up chasing by phone.
 - *Change:* Add an unread-message/new-update count badge on the 'Updates & Messages' header (driven by the existing History/Messages thread data), and auto-expand the section when there is an unread store message or a newly attached proof. Mark as read once viewed so the badge clears.
 - *Why it helps the owner:* A shop owner who messages a customer asking 'approve this proof?' today often hears nothing because the customer never noticed the collapsed section; instead the customer sees a clear 'New message' badge and the panel opens itself, so proofs get approved faster and the owner stops playing phone tag.

Quotes

- **Alert the owner the moment a storefront quote request arrives** - ● High · effort M
 - *Today:* When a visitor submits the /request-quote form (RequestQuoteForm.vue -> StorefrontQuoteController@store), the new quote lands in the /quotes list with status 'new' and source 'storefront', but nothing tells the owner. There is no badge, no email, no Action Center entry, no sound. The ExpiringQuotesRule only fires when a quote is 7 days from expiry, never when one comes in. A non-technical owner who isn't habitually refreshing the Quotes list will simply never see the request, and the customer who was promised a reply 'within 24 hours' gets silence.
 - *Change:* Add a 'New quote requests' rule to the Action Center (mirror ExpiringQuotesRule.php as a NewStorefrontQuotesRule that selects quotes where source='storefront' AND status='new' AND not yet opened, severity 'info'/'warning', link /quotes/{id}), AND send the owner a notification email/in-app notification on StorefrontQuoteController@store using the existing notification mechanism. The Quotes nav item should also show an unread-count badge for storefront requests not yet acted on.
 - *Why it helps the owner:* A store owner trying to win a job today has to remember to open the Quotes list and eyeball it for new rows; if they forget for a day the lead goes cold. Instead the request pops into the same Action Center feed they already check each morning and pings their inbox, so they reply while the customer is still shopping.
- **Fix the status colours so owners can scan which quotes need action** - ● High · effort S
 - *Today:* In nuxt/app/components/quote/List.vue getStatusColor() (lines 334-349) only maps new/draft (blue), open (red), paid (green) and void (gray). The real lifecycle is new -> draft -> open -> accepted -> expired/canceled, so 'accepted', 'expired' and 'canceled' all fall through to the gray fallback and look identical. An owner scanning the list cannot tell at a glance which quotes were won, which lapsed, and which were killed - everything that matters most is the same grey. Worse, 'open' (a quote actively awaiting the customer) is shown in alarm-red as if it were a problem.
 - *Change:* Rewrite getStatusColor to cover the actual enum: accepted = green (won), open = amber/yellow (waiting on customer), new/draft = blue (still being prepared), expired = orange (lapsed, may be revivable), canceled/void = gray. Pull these from the same statusLabel()/status colour source the rest of the app uses so they stay consistent, and use the project's statusColor.ts dot convention rather than ad-hoc strings.
 - *Why it helps the owner:* A store owner reviewing their pipeline today cannot tell a won quote from a dead one because both are grey, so they re-chase customers who already accepted or waste effort on lapsed quotes. With distinct colours they instantly see green = paid me, amber = chase the customer, orange = offer to renew.
- **Give the storefront customer a reference number and tracking link after they request a quote** - ● High · effort M
 - *Today:* RequestQuoteForm.vue submitQuote() shows only a toast ('Quote request submitted successfully! We'll get back to you within 24 hours.') and then resetForm(). The customer gets no quote number, no confirmation email, and no way to look up the request. They cannot reference it when they call, and the store owner has no shared handle to discuss it either. Logged-in customers also can't find it under /profile/quotes until the owner manually progresses it.

- *Change:* On successful submit, return the generated `quote_number` from `StorefrontQuoteController@store` and show a confirmation panel (not just a toast): 'Thanks! Your request is Quote #00123. We'll email you a price within 24 hours.' plus a 'View my requests' link to `/profile/quotes` for logged-in customers. Send the customer an automatic acknowledgement email with the same reference. Replace the hardcoded '24 hours' copy so it reflects the chosen delivery timeline.
- *Why it helps the owner:* A store owner whose customer phones to ask 'did you get my request?' currently has nothing to match them against and has to dig through the list by name. With a reference number on both sides, the customer says 'Quote #123' and the owner finds it instantly, and the auto-acknowledgement stops the owner from fielding 'did it go through?' calls at all.
- **Rename and explain the 'Client Page Link' and clarify when to use it vs Email PDF** - ● Med · effort S
 - *Today:* The quote detail page exposes a 'Client Page Link' (to `/quotes/payment/{id}`) and a separate 'Email PDF' button. 'Client Page Link' is a developer's name; a non-technical owner has no idea this is the page where the customer accepts and pays, nor when to copy it versus when to hit Email PDF. The two actions look unrelated even though they're two ways to do the same thing (get the customer to accept).
 - *Change:* Relabel 'Client Page Link' to 'Copy customer accept-and-pay link' with a one-line helper: 'Share this if you'd rather send the quote yourself (WhatsApp, SMS) instead of emailing it.' Group it directly beside 'Email PDF' under a single 'Send to customer' heading, and on the Email PDF flow note 'This email already includes the accept-and-pay link' so the owner understands the two are equivalent and never sends both confusingly.
 - *Why it helps the owner:* A store owner who wants to send a quote over WhatsApp today sees a cryptic 'Client Page Link' and doesn't realise that's the thing to paste; they fall back to email even when the customer prefers chat. With a plain label and grouping they confidently pick the channel their customer actually uses.
- **Hide raw status mechanics from quote creation and the Accounting 'Unposted' label** - ● Med · effort M
 - *Today:* The New Quote modal (`Add.vue`) forces the owner to pick from a generic 'Status' dropdown (`quote_status: new/draft/open`) with no guidance - a non-technical owner cannot know the difference and may set 'open' before the quote even has line items. Separately the detail page permanently shows 'Unposted' under Accounting with no explanation, leaving the owner wondering if they forgot a step. Both expose internal bookkeeping concepts the owner shouldn't have to reason about.
 - *Change:* Remove the Status dropdown from the default (collapsed) New Quote form and always create as 'Draft' - status should advance automatically (Draft until 'Email PDF'/share, which already flips it to 'open' via `sendToCustomer()`). Leave manual status only in the advanced 'Show All Fields' section for power users, with plain descriptions ('Draft - still preparing', 'Open - sent, waiting on customer'). Replace the bare 'Unposted' accounting label with a tooltip-explained plain phrase such as 'Not yet in your books (becomes an invoice when the customer accepts)'.

- *Why it helps the owner:* A store owner creating their first quote today is asked to choose a status they don't understand and stares at 'Unposted' wondering what they did wrong. Instead the quote just starts as a Draft and advances itself, so the owner only thinks about the customer and the price - never the plumbing.
- **Remove or wire up the dead quote surfaces and the fake customer-facing data** - ● Med · effort L
 - *Today:* Several quote surfaces are non-functional or show fake data that erodes trust. The `/quote.vue` cart-based builder's `submitQuote()` only `console.logs` and never hits the API, so it silently does nothing. The frontstore `/quote` calculator shows hardcoded contact info ('+1 (555) 123-4567', 'quotes@printflow360.com', 'Mon-Fri 9AM-6PM EST') and a hardcoded 'Your Recent Quotes' list (Business Cards/Flyers/Brochures at fixed prices) that is not the customer's real data. The `/profile/quotes` list also shows 'Paid' and 'Void' filter tabs for statuses the workflow never produces, so customers click them and see an unexplained blank.
 - *Change:* Either gate these behind a feature flag until finished or fix them: make `/quote.vue` submit to the real quote endpoint (or remove the page); replace the hardcoded contact block with the tenant's real store settings (phone/email/hours) and the dummy 'Recent Quotes' with the customer's actual `GET /frontstore/customer/quotes` data (with proper empty state); and drop the 'Paid'/'Void' tabs from `/profile/quotes` to match the real lifecycle (or relabel to states that exist, e.g. 'Accepted'/'Expired').
 - *Why it helps the owner:* A store owner expects every page their customer sees to be theirs; today a customer can land on a quote page showing a US phone number that isn't the shop's and prices that aren't theirs, which makes the shop look broken or fraudulent. Removing the fake data and dead buttons means the owner never has to apologise for a page they didn't know existed.

Invoices, Payments & Accounting

- **Make "Mark as Paid" record the actual payment, not just flip a status** - ● High · effort M
 - *Today:* The "Mark as Paid" action (`POST /invoices/{id}/mark-as-paid`, `invoices/[id].vue` line 269) only sets `status='paid'`. It shows no confirmation, never asks how much was paid or by what method, and creates no payment row. The owner who just received \$500 cash clicks it, the invoice goes green, but the payment history stays empty and "Amount Paid" stays \$0 - there is no record of when or how they were paid.
 - *Change:* Replace the silent status flip with a small "Record Payment" dialog that pre-fills the full Balance Due as the amount, defaults the date to today, and requires a payment method (Cash / Cheque / Bank Transfer / Card). On confirm, create a real payment row (`paymentable_type = Invoice`) AND set the status to paid. Keep the amount editable so partial payments leave the invoice 'open' with a correct Balance Due. The confirm button reads "Record \$X.XX Payment".
 - *Why it helps the owner:* A store owner who takes a cash payment at the counter today gets a green invoice but no proof of payment in the history. Instead, one click captures the amount, method, and date - so when the customer disputes it next month, the owner can show exactly what was paid and how.
- **Fix or hide the broken Stripe invoice-payment path** - ● High · effort L

- *Today:* On the customer payment-select page (payment/select/[id].vue line 106), choosing Stripe routes to /invoices/payment/stripe/[id], which POSTs to /stripe/create-checkout-session/{id} - a route that exists in no route file. The customer gets a raw “Failed to initialize Stripe payment.” with no recovery. An owner with Stripe enabled believes invoice payment works; it silently dies the moment a real customer tries to pay.
- *Change:* Wire the Stripe invoice button into the working multi-gateway abstraction (CheckoutPaymentService / PaymentGatewayRegistry) that already drives storefront Stripe redirects, reusing the same session-creation flow that webhooks (UpdatePaymentAndOrder) already mark paid. If that cannot ship immediately, gate Stripe out of the invoice gateway list (filter it from activeGateways here) so it never appears as a payable option until it works - never show a method that errors on click.
- *Why it helps the owner:* A store owner who emails a customer a Stripe pay link today sends them into a dead error page and looks unprofessional. Instead the customer completes payment in Stripe’s checkout and the invoice auto-marks paid, exactly like storefront orders already do.
- **Include admin-entered invoice payments in the Accounting totals and export** - ● High · effort M
 - *Today:* AccountingController::tenantPaymentsQuery() joins exclusively through orders, so every payment with paymentable_type = Invoice (cash/cheque/bank transfer entered via the admin New Payment form) is excluded from the Accounting page KPIs, the This Month/This Year revenue, the payment-method breakdown, and the CSV export. The owner sees those payments inside each invoice but never in the summary.
 - *Change:* Broaden the accounting payments query to union order-based payments with invoice-based payments (status='paid'), keyed by the same tenant scope, so all collected money appears in the KPI cards, method breakdown, transaction log, and CSV. Add the invoice number / customer as the source label on invoice rows so the log stays readable.
 - *Why it helps the owner:* A store owner who exports the Accounting CSV to hand their accountant today gives them numbers that silently omit every cash and cheque payment - the totals are wrong. Instead the export reflects every dollar collected, so their books reconcile.
- **Remove the hardcoded “Unposted” badge from the invoice detail** - ● High · effort S
 - *Today:* invoices/[id].vue (lines 427 and 624) renders a permanent orange “Unposted” label with no backing database field and no way to ever change it. There is no ‘post’ action and no explanation. A non-technical owner reads a permanent orange warning on every invoice and assumes something is broken or unfinished.
 - *Change:* Delete the “Unposted” label entirely (both occurrences). If a real accounting state is wanted later, replace it with a meaningful, backed status with a tooltip - but ship the removal now so no invoice carries a meaningless orange flag.
 - *Why it helps the owner:* A store owner glancing at an invoice today sees an alarming orange ‘Unposted’ tag and either worries or wastes time hunting for how to ‘post’ it. Instead the invoice shows only states that mean something they can act on.
- **Make the customer pay-link clearly shareable and not an admin URL** - ● Med · effort M

- *Today:* The “Open Payment Page” link points to the admin app URL `/invoices/payment/select/{id}`, which the owner is expected to copy and paste to the customer by hand. There is no “Copy link” affordance, no indication it is an admin-side URL, and while `hasActiveGateway` is still loading the link area is simply blank with no feedback.
 - *Change:* Replace the bare link with a “Copy payment link” button that copies a public, tokenized payment URL (storefront/standalone host, not the admin app) and shows a “Link copied - ready to send to your customer” toast. Render a skeleton/disabled state with ‘Checking payment methods...’ while gateways load instead of a blank gap, and when no gateway is active show the existing amber ‘configure a gateway’ guidance inline.
 - *Why it helps the owner:* A store owner who wants a customer to pay an invoice today must hand-copy an admin URL that may dump the customer into the staff panel. Instead they click Copy, paste it into an email or text, and the customer lands on a clean public payment page.
- **Stop showing a card form for Cheque, and give the invoice status a sensible default** - ● Med · effort M
 - *Today:* Two creation/flow defaults trip up first-timers: (1) selecting ‘Cheque’ on the payment-select page routes to the [Authorize.Net](#) card-entry form with `?method=cheque` ignored (`select/[id].vue` line 109), so a cheque-paying customer is shown a card number/CVV form; (2) the New Invoice modal forces the owner to pick status `new/draft/open` with no default or tooltip.
 - *Change:* For Cheque, route to a simple instructions screen (your cheque payee/mailling or bank-transfer details + ‘Mark this invoice as awaiting cheque’) instead of the card form. For new invoices, default the status to ‘Open’ (ready to send/collect) and add one-line plain-language hints under each option (‘Draft - not finished yet’, ‘Open - sent, awaiting payment’).
 - *Why it helps the owner:* A first-time owner creating invoice number one today must guess between `new/draft/open` with no guidance, and a cheque-paying customer is shown a credit-card form. Instead the invoice defaults to the obvious ‘Open’, and choosing Cheque shows where to send the cheque - no wrong forms, no guessing.
 - **State the consequences in the refund confirmation** - ● Med · effort S
 - *Today:* `RefundConfirmationPopup.vue` asks only “Are you sure you want to refund this payment?” with no amount, no statement of where the money goes (back to the card automatically vs. processed manually), and a generic confirm button - violating the destructive-action rule in [CLAUDE.md](#) section 0.
 - *Change:* Show the refund amount, the payment method/last-4, and a plain-language consequence (‘This will refund \$120.00 to the customer’s card via [Authorize.Net](#). This cannot be undone.’ or, for manual methods, ‘This marks the payment refunded - you must return the money to the customer yourself.’). Label the button “Refund \$120.00”.
 - *Why it helps the owner:* A store owner refunding a payment today can’t tell from the dialog how much is leaving or whether the system actually moves the money. Instead they see the exact amount and whether it auto-refunds or needs a manual hand-back, so they don’t double-refund a customer.

Customers & Activity

- **Show the engagement columns by default instead of hiding them behind a column-config icon** -  High · effort S
 - *Today:* The default view seeded for every tenant (TenantOnboardingService.php line 213) only includes 7 columns: company_name, account_number, tags, primary_contact_name, primary_contact_email, primary_contact_number, actions. Last Login, Last Order, Type, Total Spent, and Orders are all absent. A store owner opening the Customers page for the first time sees a plain address book and none of the engagement data the feature was built to provide. To see it they must find the unlabeled table-cells icon, guess what it does, and toggle each column on.
 - *Change:* Add last_order_at, total_spent, orders_count, and last_login_at to the seeded default view column list in TenantOnboardingService.php, and add a one-time migration/back-fill that adds these columns to existing tenants' default view rows that pre-date the activity feature. The list already renders all of these cells correctly (List.vue lines 208-227); only the seeded default omits them.
 - *Why it helps the owner:* A store owner who opens Customers today to answer 'who are my best repeat customers and who's gone quiet?' sees an address book with no spend, order, or login data and has to discover a cryptic icon to surface it; instead they would see total spend, order count, last order and last login the moment the page loads, with zero configuration.
- **Fix the raw 'Manual' sub-label under each customer name to read 'Added by store'** -  High · effort S
 - *Today:* The always-visible sub-label inside the Company Name column (List.vue line 145) renders {{ row.customer_type || 'guest' }} with CSS capitalize, so a contact the owner added themselves shows as 'Manual'. The codebase already has customerTypeBadge() in customerType.ts that maps the internal 'manual' value to the human label 'Added by store' (and 'guest' → 'Guest', etc.), but this sub-label bypasses it and prints the raw database string. The owner sees the word 'Manual' with no way to know what it means.
 - *Change:* In List.vue line 145, replace {{ row.customer_type || 'guest' }} with customerTypeBadge(row.customer_type).label so the sub-label reuses the same plain-language mapping as the dedicated Type column. This also makes the inline label and the optional Type column show identical wording for the same customer.
 - *Why it helps the owner:* A store owner scanning their list today sees a contact they personally entered labelled 'Manual' and cannot tell it apart from a developer term; instead they would see 'Added by store', immediately understanding this person can't log in until invited and isn't a real storefront sign-up.
- **Replace the '(N/A)' detail-page placeholders with click-to-fix empty states** -  High · effort S
 - *Today:* The customer detail page uses developer-style fallbacks: 'Primary Name (N/A)' (line 404), 'Primary Email (N/A)' (line 414), 'Primary Phone (N/A)' (line 428), and 'Tags (N/A)' (line 290). [CLAUDE.md](#) section 0 explicitly bans this pattern and calls for an actionable empty state. The fields are already inline-editable, but '(N/A)' gives the owner no signal that the blank is fixable or how.
 - *Change:* Swap each '... (N/A)' string for an actionable placeholder tied to the existing inline-edit affordance, e.g. 'No name added - click to add', 'No email added - click to add', 'No phone added -

click to add', and for tags 'No tags yet - click to add'. Render these in muted text so they read as a prompt, not data.

- *Why it helps the owner:* A store owner viewing a half-complete contact today sees 'Primary Email (N/A)' and assumes the system is broken or that nothing can be done; instead they would see 'No email added - click to add' and know exactly how to complete the record on the spot.
- **Turn the four KPI cards into engagement-actionable counts** - ● Med · effort M
 - *Today:* The four KPI cards are Total Customers, Active Customers, Archived Customers, and New This Month. Archived Customers is near-useless as a headline metric, and none of the cards reflect the engagement data the feature captures. An owner wanting to act on 'who never logged in' or 'who hasn't ordered' has to manually build a filter rather than see the number surfaced and clickable.
 - *Change:* Replace the 'Archived Customers' KPI with a 'Never Logged In' count (customers where `last_login_at` is null and source is not 'admin'), and make each KPI card click through to the list pre-filtered to that segment (the list already supports Last Login date-range and status filtering via the funnel modal). Keep Total, New This Month, and Active; move Archived behind the existing Active/Archived toggle where it already lives.
 - *Why it helps the owner:* A store owner who wants to re-engage quiet customers today must understand the filter modal and construct a last-login condition by hand; instead they would see 'Never Logged In: 23' on load, click it, and land on exactly that list ready to email or call.
- **Explain what 'Login as Customer' does before and after it runs, and guide recovery when no store is configured** - ● Med · effort S
 - *Today:* Clicking 'Login as Customer' (list row icon and detail-page button) opens the storefront in a new tab with a 30-minute HMAC token (`CustomerController@loginAs` line 735) but no on-screen explanation that the owner is now impersonating the customer or that the session expires. If the primary store's frontstore URL can't be resolved (no primary store configured), the owner gets a dead link with no guidance on what to fix.
 - *Change:* On a successful loginAs, show a plain-language toast such as 'Opening the storefront as this customer in a new tab. This temporary session expires in 30 minutes.' When the store URL can't be resolved, replace the failed link with an actionable error toast: 'No storefront is set up for this customer's store yet. Set one up in Settings > Stores to use Login as Customer.' with a link to that path, matching the unconfigured-integration pattern in [CLAUDE.md](#) section 0.
 - *Why it helps the owner:* A store owner clicking 'Login as Customer' to reproduce a checkout problem today either gets a silently-opened tab they don't understand or a broken link with no explanation; instead they would get a clear confirmation of what's happening and, on failure, a direct instruction on what to configure.
- **Recover gracefully when a tenant has no seeded customer view instead of showing 'Invalid view' on an empty table** - ● Med · effort M
 - *Today:* The list depends on a default view row seeded at onboarding. A store onboarded before the customer default view existed hits an 'Invalid view' error toast on load and an empty table, with no path to recover without backend intervention. For a non-technical owner this reads as the whole Customers page being broken.

- *Change:* When no valid view is returned, have the list fall back to a built-in default column set (the same 7-to-11 columns) and silently self-heal by creating the missing default view row for that tenant, rather than surfacing an 'Invalid view' toast and a blank table. Show the table normally; log the repair server-side.
- *Why it helps the owner:* A store owner on an older tenant who opens Customers today sees an error and an empty screen and assumes their customer data is gone; instead the page loads their customers normally and quietly fixes the missing-view configuration behind the scenes.

B2B / Business Accounts

- **Show the negotiated price on product listing cards, not just inside the configurator** - ● High · effort M
 - *Today:* StorefrontProductResource (app/Http/Resources/Api/Storefront/StorefrontProductResource.php:21) returns raw base_price with no per-company discount, so a logged-in B2B buyer browsing the catalog sees normal store prices on every product card. The discounted price they will actually pay only appears once they open the product configurator. The shop owner has promised a client 'your prices are 15% lower' and the catalog visibly contradicts that promise.
 - *Change:* Thread the company_account_id context already used by ProductPricingCalculator into the listing path so StorefrontProductResource applies CompanyPricingResolver to base_price for logged-in B2B customers. Show the discounted price on the card with the original struck through and a small 'Your business price' badge. Keep guest/B2C cards unchanged. The cache key already includes :company:{id} (CacheApiResponse), so per-company prices won't leak.
 - *Why it helps the owner:* A store owner who just told their corporate client 'you get 15% off everywhere' can today be undermined the moment that client browses, because the catalog shows full price and generates 'why am I being overcharged?' support emails. Instead the client sees their agreed price from the first page they land on, and the owner never has to explain the discrepancy.
- **Let the owner create a brand-new customer directly from the Contacts tab** - ● High · effort M
 - *Today:* The Add Contact flow (nuxt/app/pages/business-accounts/[id].vue:482) only searches existing customers via api.searchCustomers and offers no way to create a person who has never shopped before. If a B2B buyer has never registered, the search returns nothing and the owner hits a silent dead end with no instruction on what to do next.
 - *Change:* When the contact search returns no match, surface a 'Add a new person to this company' button in the empty state that opens a small inline form (name, email, role, department, allow-pay-on-account). On submit, create the Customer pre-linked to this company (account_type=b2b, company_account_id set) and, if the company has allowed_email_domains, validate the email against them with the existing plain-language message. Optionally send them a set-password invite.
 - *Why it helps the owner:* A store owner onboarding 'Globex Corp' with five named buyers today must ask each buyer to go self-register on the storefront first, then come back and search for them one by one. Instead the owner types in all five people on the spot during the sales call and the account is ready to order immediately.

- **Replace the signed manual-adjustment field with a Charge/Credit choice** - ● High · effort S
 - *Today:* The Account & Credit tab's manual adjustment uses a signed-number convention (negative = charge the company, positive = credit them) with only small helper text. A non-accountant store owner correcting a billing mistake will frequently enter the wrong sign, posting a backwards ledger entry that then needs a second adjustment to undo.
 - *Change:* Replace the single signed input with two controls: a 'Type' toggle ('Charge the company more' / 'Give the company credit') and an always-positive amount field. Show a live plain-language preview: 'This will increase Globex Corp's balance owing by \$50.' before they confirm. Keep the note required. Convert to the signed ledger amount under the hood so the append-only ledger is unchanged.
 - *Why it helps the owner:* A store owner fixing a \$50 overcharge today must remember whether to type +50 or -50, and getting it wrong doubles the error and the cleanup. Instead they pick 'Give the company credit', type 50, read 'reduces what they owe by \$50', and confirm with no accounting knowledge required.
- **Surface a 'No business stores yet - set one up' path before the Add Account modal** - ● High · effort S
 - *Today:* B2B activation depends on having a Business/Both store, but the only guidance is an amber warning buried inside the Add Business Account modal (Form.vue:15-19) that the owner sees only after clicking Add and finding the store dropdown empty with a disabled Create button. The connection 'I must first set a store's type to Business' is never offered proactively from the Business Accounts section.
 - *Change:* On the Business Accounts list page, when the tenant has zero business-facing stores, replace the empty state with a guided card: 'To sell to companies, first set a store up for business. Choose which store sells to businesses → ' linking directly (absolute path) to that store's edit page with the 'Who will buy from this store?' field. Also show this as the modal's body instead of a disabled form when no eligible store exists.
 - *Why it helps the owner:* A store owner who wants to start selling to a company today clicks Add Business Account, finds an empty dropdown and a greyed-out button, and is left guessing what's wrong. Instead they get a one-click path to the exact setting that unlocks the feature, and reach a working create form on the next screen.
- **Tell the owner the role and department-budget fields are informational today** - ● Med · effort S
 - *Today:* The Departments tab accepts a monthly budget that is captured but never enforced at checkout, and contact Roles (Buyer/Manager/Account Admin) are shown prominently but today only gate who can view the statement. A store owner who sets a \$2,000 marketing budget expects orders to stop at that limit, and someone marked 'Account Admin' is expected to have admin powers - neither is true, with nothing in the UI saying so.
 - *Change:* Add an inline helper under the budget field: 'Recorded for your reference and reporting. Orders are not yet blocked when a budget is reached.' Add a small tooltip/helper next to the Role selector: 'Controls who can view the company statement on the storefront. Ordering and approvals are not yet limited by role.' Plain text, no roadmap jargon.

- *Why it helps the owner:* A store owner relying on a department budget to cap a client's spend today believes orders will be stopped at the limit and only finds out they weren't after an overspend. Instead they know upfront the figure is for reporting, and set expectations with their client accordingly.
- **Show a proper loading skeleton for the Account & Credit tab and flag the private-store default -**
 - Med · effort S
 - *Today:* The Account & Credit tab is lazy-loaded on first click but has no credit-specific skeleton, so the first click briefly shows a blank panel that reads as 'no credit set up' to a non-technical owner. Separately, Business stores default to login-only (require_login) but that default is set silently in Store.requiresStorefrontLogin() with nothing telling the owner their storefront is now private until a customer complains they cannot browse.
 - *Change:* Add the standard skeleton/spinner to the Account & Credit tab while its data loads (matching the other four tabs). On the store edit page, when store type is set to Business, show an inline note next to the type field: 'Business stores are private by default - only logged-in customers can browse. You can change this under Store Settings → Require login to access this store.' with a link to that toggle.
 - *Why it helps the owner:* A store owner opening a client's credit tab today sees a blank panel and assumes credit isn't configured, and separately discovers their public storefront went login-only only when a walk-in customer says they can't see prices. Instead the credit numbers load behind a clear skeleton, and the private-store consequence is stated at the moment they choose 'Business'.

Roles & Permissions (Team Access Control)

- **Pre-fill each built-in role with the permissions its job actually needs -**
 - High · effort M
 - *Today:* All seven built-in roles (Designer, Sales Rep, Project Manager, etc.) are seeded with ZERO permissions. TenantOnboardingService::seedRolesAndPermissions() assigns nothing to non-admin roles, so the moment an owner adds a 'Designer' staff member, that person logs in and sees only the Dashboard - a blank nav that reads as a broken app, with no clue that the owner was supposed to configure the Designer role first.
 - *Change:* Seed sensible default permissions per built-in role at onboarding (e.g. Designer = Editable on Print Jobs/Tasks + Visible on Quotes; Sales Rep = Editable on Quotes/Customers + Visible on Invoices; Project Manager = Visible across orders/jobs/customers). Keep them editable afterward. Add a small 'Suggested for this role' note in the edit screen so the owner sees these are starting defaults they can change. This is the \$0 'sensible defaults so nobody has to configure' rule applied directly.
 - *Why it helps the owner:* A shop owner who hires a new designer today must know - with zero prompting - to first open Settings > Team > Roles > Designer and tick the right boxes, or the designer is locked out of everything; instead they could add the designer and have them immediately able to see jobs and tasks on first login, exactly what 90% of shops want.
- **Tell the owner the new staff member gets an email with their password -**
 - High · effort S

- *Today:* The Add User form (team/user/Add.vue) collects Name, Email, Phone, Role and a Save button - with no mention anywhere that the backend auto-generates a 15-char password and emails it via SendNewUserEmailJob. The owner has no idea the person will receive login credentials by email, so they either try to hand-set a password or tell the new hire a password that doesn't exist.
 - *Change:* Add a plain-language helper line under the Email field and an inline confirmation on save: 'We'll email [name] a link to set up their login. You don't need to share a password.' On success, replace the generic toast with 'Invitation sent to jane@shop.com - they'll receive login details by email.'
 - *Why it helps the owner:* An owner adding their first sales rep today is left guessing how that person logs in and may phone them a made-up password; instead they could read one sentence on the form, hit Save, and confidently tell the hire 'check your email'.
- **Fix the edit-role page so it stops saying 'Create New Role' and lets the name be corrected** - ● High · effort S
 - *Today:* In team/role/Upsert.vue the heading on line 196 is the hardcoded string 'Create New Role' regardless of mode, and the name input on line 200 is :disabled="isEditMode". So when an owner clicks the pencil to adjust the Sales Rep role, the page title says 'Create New Role' and the name field is greyed out - it looks like they're about to make a confusing duplicate, and a typo'd role name can never be fixed.
 - *Change:* Make the heading conditional: 'Edit {{ title }} Role' vs 'Create New Role'. Remove the hard :disabled on the name input in edit mode (or allow rename with a confirm), so a mislabeled role like 'Salse Rep' can be corrected. Also delete the two console.log statements on lines 42 and 156 that dump permission internals into the browser console.
 - *Why it helps the owner:* An owner editing the Sales Rep role today sees 'Create New Role' and reasonably backs out fearing they'll create a second role, or is stuck forever with a typo in a role name; instead they could see 'Edit Sales Rep Role', trust they're in the right place, and fix the spelling.
- **Stop swallowing role save/load errors - show a plain message and a retry** - ● High · effort S
 - *Today:* Every catch block in this feature only console.error()s: fetchRoleData (Upsert.vue line 87), savePermissions (line 178), fetchRolesData (List.vue line 32), and fetchRoles (Add.vue line 32). If the permissions list fails to load or a save fails, the owner sees absolutely nothing - no toast, no banner. They click Save again and again, or assume the app froze. This directly violates the \$0 mandatory error-state rule.
 - *Change:* On every catch, show a plain-language error toast with a recovery action: on save failure 'We couldn't save this role. Please check your connection and try again.' (keep the form filled); on load failure render an inline 'Couldn't load permissions - Retry' button instead of a dead screen. Never leave a catch as console-only.
 - *Why it helps the owner:* An owner on a flaky shop wifi who hits Save and nothing happens today has no idea if the role saved or not and may corrupt their setup by re-clicking; instead they could see 'couldn't save, try again', press retry, and know exactly where they stand.
 - **Add loading skeleton and a helpful empty state to the Roles list** - ● Med · effort S

- *Today:* In team/role/List.vue the loading ref only drives the 'Load More' button (line 104), so on first open the bordered list area is blank white until data arrives - looks broken on a slow connection. There is also no empty state: if nothing has loaded the user just sees an empty box with no message. Both violate the §0 mandatory loading + empty state rules.
 - *Change:* Show 3-4 skeleton rows while the first fetch is pending, and when the list is genuinely empty show a helpful message with the next action: 'No roles yet. Click "Add New Role" to create one, or use the built-in roles above.' Reserve the same height so the layout doesn't jump.
 - *Why it helps the owner:* An owner opening Settings > Team on a slow connection today sees a blank white box and assumes team access is broken; instead they could see loading placeholders that resolve into their roles, or a clear prompt telling them what to do next.
- **Warn how many staff are affected before deleting a role** - 🟡 Med · effort M
 - *Today:* Deleting a role (UtilsDeleteEntity in List.vue, label 'role') shows the generic 'Are you sure you want to delete this role?' - it never says how many team members are currently assigned (the list already loads role.users_count). An owner could wipe the access of several active staff with no warning, violating the §0 'destructive actions must state consequences' rule.
 - *Change:* Make the delete confirmation specific using the member count already available: 'Deleting "Sales Rep" will remove access for the 3 team members currently using it. They won't be able to log in until you assign them another role. This cannot be undone.' Restate the action on the button ('Delete Role', not 'Yes'). If users_count > 0, ideally require reassigning them first.
 - *Why it helps the owner:* An owner cleaning up roles today can accidentally lock out three working sales reps mid-shift with a one-click 'Yes'; instead they could see exactly who is affected and what breaks before confirming, and re-home those staff first.
 - **Stop showing access tiers the backend will silently ignore (Orders and other admin-only modules)** - 🟡 Med · effort M
 - *Today:* The role editor lets the owner grant Editable/Visible access to modules like Orders, but Orders is hardcoded to admin-only both in the backend route (middleware permission:is_admin) and the nav (Header.vue line 151 permissions:['is_admin']). So an owner can carefully give a Project Manager order access, save it, and nothing happens - the module stays invisible with no explanation. Several other modules (CMS, Catalog, Marketing, Accounting) behave the same way.
 - *Change:* In the role editor, mark modules that can't currently be delegated as 'Admin only' (disabled tier with a tooltip: 'Only Admins can access Orders right now'), instead of offering Editable/Visible toggles that the backend ignores. Better still, where the data model allows it, actually honor order.* permissions so the toggle works - but at minimum never present a control that silently does nothing (§0 'no half-built UI').
 - *Why it helps the owner:* An owner trying to let their project manager see incoming orders today spends time configuring permissions that have zero effect and then assumes the app is buggy; instead they could immediately see 'Orders is Admin only' and not waste effort, or actually grant the access and have it work.

Notifications & Automation

- **Make the storefront customer notification toggles actually work (or remove them)** - ● High · effort M
 - *Today:* In `frontstore/app/pages/profile/settings.vue` (lines 70-107) the customer Notifications tab shows three real-looking toggles - Order Updates, Promotions, Newsletter - but they are pure decorative HTML: no `v-model`, no `@change`, no API call. A customer who unchecks 'Promotions' or 'Order Updates' sees the switch move and assumes it saved, yet nothing is persisted and they keep getting every email. For a print-shop owner this becomes a support complaint ('I turned off marketing emails and still get them') with no setting they can point to.
 - *Change:* Wire the three toggles to a real customer preference store. Bind each toggle to a `v-model`, call a PATCH endpoint (e.g. `/storefront/profile/notification-preferences`) on change, persist the choices on the customer record, and have the outbound email path (ActionExecutor + `SendAppNotification` for customer-facing mail) honour the 'Promotions'/'Newsletter' flags before sending marketing-type emails (order-status emails should stay mandatory and the toggle for those should be removed or labelled 'always on'). Add a 'Saved' toast on change and a loading/disabled state while saving. If wiring the backend is out of scope short-term, the toggles must be hidden, not shown as functional.
 - *Why it helps the owner:* A shop owner whose customer asks 'stop sending me promotions' can today only apologise - the preference is silently discarded. Instead the customer flips one switch, it sticks, and the owner stops fielding 'why am I still emailed' complaints and stays compliant with unsubscribe expectations.
- **Stop the email-channel notification bug that drops subscribed recipients** - ● High · effort S
 - *Today:* In `app/Jobs/Notification/SendAppNotification.php` the email recipient resolution (line 96) calls `getNotificationUsers(..., 'push')` instead of 'email'. This means staff who are correctly subscribed to an event by role but have their personal push toggle off (`notifications_enabled = false`) never receive the EMAIL either - email is silently gated by a push-only flag. A store owner who set up 'email the Sales Rep when an invoice is paid' gets no error and no email; they just quietly miss payments.
 - *Change:* Fix line 96 to resolve recipients with channel 'email' and ensure the email path is gated by an email-specific preference (or no per-user gate), not by the push `notifications_enabled` flag. Add a Feature test asserting that a role-subscribed user with push disabled still receives the email notification for that event.
 - *Why it helps the owner:* An owner relying on 'email me when a proof is approved so I can start production' is today missing some of those emails with zero visibility into why. After the fix, every recipient they configured in Settings > Notifications actually gets the email, so production doesn't stall on a notification that never arrived.
- **Replace the permanently greyed-out SMS toggle with an honest, explained state** - ● High · effort S
 - *Today:* In `nuxt/app/pages/setting/notifications.vue` (lines 50-62) every one of the 37 event rows shows an SMS toggle hard-disabled with `opacity-40` and zero explanation. A non-technical owner

clicks it, nothing happens, and they conclude either the app is broken or that they're missing something they paid for. It's a confusing dead control repeated 37 times.

- *Change:* Either (a) hide the SMS column entirely until SMS sending is wired, or (b) keep it visible but add a tooltip / inline helper on hover and tap ('Text message alerts are coming soon' or 'SMS requires a connected text provider - Set up in Settings > Integrations') so the disabled state is self-explanatory. Per the project's unconfigured-integration pattern, point to where to enable it rather than leaving a silent grey switch.
- *Why it helps the owner:* An owner scanning notification settings today wastes time poking a dead SMS switch and may open a support ticket. Instead they instantly understand SMS isn't on yet and where it'll live, and move on to configuring the channels that do work.
- **Group the 37 flat event rows by category and hide rare edge-case events** - ● Med · effort M
 - *Today:* Settings > Notifications renders all 37 event types as one undifferentiated flat list, labelled by a dot-to-space transform (formatEventLabel, line 140-141). A store owner sees 'Invoice Created', 'Invoice Restored', 'Quote Restored', 'Inquiry ...' all jumbled together with no sense of which matter day-to-day versus edge cases like 'Invoice Restored'. They can't find the three events they actually care about.
 - *Change:* Group rows under collapsible plain-language section headers by module (Invoices, Quotes, Orders, Production Jobs, Proofs, Customers, Inquiries) using the already-computed eventModules map. Within each group, surface the common operational events first and collapse rare/admin events (Restored, etc.) under a 'More events' expander. Improve the label map so technical-sounding events read in shop language (e.g. 'invoice.paid_in_full' → 'Invoice Fully Paid').
 - *Why it helps the owner:* An owner who just wants 'tell my production manager when a proof is approved' today has to scan 37 rows to find it. Grouped and shop-worded, they jump straight to the Proofs section, flip one row, and trust they've covered the case that matters - instead of giving up unsure they got it right.
- **Add a confirmation dialog with consequences before deleting an automation** - ● Med · effort S
 - *Today:* In nuxt/app/components/automation/List.vue, deleteAutomation (line 228) fires the DELETE request immediately on clicking the red 'Delete' link - no confirmation, no warning. An automation may have run hundreds of times ('ran N times' counter) and be the thing emailing every customer a payment thank-you. A misclick silently destroys it and its run history, and the owner may not notice until customers stop getting emails.
 - *Change:* Gate deleteAutomation behind a confirmation modal that states the consequence in plain language and restates the action on the button: e.g. 'Delete "Thank-you on payment"? This automation has run 142 times. Customers will no longer be emailed automatically when they pay. This cannot be undone.' Confirm button reads 'Delete Automation', not 'Yes'. Match the destructive-confirm pattern used elsewhere in the admin.
 - *Why it helps the owner:* A shop owner tidying up their automations today can wipe out a live customer-facing rule with one accidental click and no safety net. With a clear consequence-stating confirm, they see exactly what stops happening before they commit, so a tidy-up never silently breaks their customer emails.

- **Surface silently-failing automations and triggers instead of `run_count = 0`** - ● Med · effort L
 - *Today:* Several automations can be enabled yet silently never do anything: the default 'In Production' / 'Ready for Pickup' rules reference hardcoded global job-status IDs (job-27, job-64) that may not match a tenant's statuses, so ConditionValidator returns false and nothing fires; the task_completed trigger has no ConditionValidator branch (fires unconditionally); and assign_task / request_payment actions are null-matched in ActionExecutor and do nothing. The owner just sees an enabled automation with 'ran 0 times' and no clue why, or worse, emails firing for the wrong reason.
 - *Change:* Three concrete fixes: (1) Make the seeded automations resolve job statuses by tenant-specific name/slug at onboarding rather than hardcoded IDs, and if a referenced status is missing, mark the automation as 'Needs setup' in the list with a plain note ('This rule points to a status that doesn't exist in your store - pick one to activate it'). (2) Remove task_completed, assign_task, and request_payment from the trigger/action pickers until they're implemented (gate behind the feature flag pattern) so owners can't build a rule that silently does nothing or fires unconditionally. (3) In the automation list, when an enabled rule has run_count = 0 after being created N days ago, show a subtle 'Hasn't run yet - check the condition' hint.
 - *Why it helps the owner:* An owner who set up 'email the customer when their order is ready for pickup' today sees it enabled, trusts it, and never learns it's firing into a void because of a mismatched status ID - customers never get told their job is ready. With status-by-name resolution and a 'Needs setup' flag, the rule either works or visibly tells them what to fix, so the pickup email they rely on actually goes out.

Action Center

- **Turn the email summary ON by default and surface it in plain sight** - ● High · effort S
 - *Today:* The email digest is OFF by default and buried two clicks deep: the owner must navigate to /action-center, then open the 'Manage alerts' slideover, then find the 'Email summary' section. A non-technical owner who never discovers it will simply never receive overdue-invoice or late-job warnings on the days they don't log in - which is exactly when they most need them.
 - *Change:* Default the store-wide digest to ON (daily at 07:30) for new stores, and on first load of /action-center show a one-line confirmation banner at the top ('We email you a daily summary of what needs attention. Change this in Manage alerts.') with an inline 'Turn off' link. Keep the slideover for fine-tuning, but the default and the banner mean discovery is no longer required.
 - *Why it helps the owner:* A shop owner who only logs in twice a week and is trying to avoid missing an overdue invoice must today stumble onto a hidden slideover and turn on emails they don't know exist; instead the summary already lands in their inbox every morning, so a late job or unpaid invoice reaches them whether or not they log in.
- **Replace developer severity labels with shopkeeper language everywhere** - ● High · effort S
 - *Today:* The full page shows tab labels and per-row badges reading 'Critical', 'Warning', and 'Info' (action-center/index.vue lines 158-163 and 223-225). 'Critical' and 'Info' are developer terms, and they are inconsistent with the email digest template, which already uses the friendlier 'Needs attention' / 'Good to know'. A non-technical owner reading 'Info' next to a dormant-customer alert has no idea whether it matters.

- *Change:* Map severities to plain language in the UI: critical -> 'Urgent', warning -> 'Needs attention', info -> 'Good to know'. Apply the same map to the tab labels, the row badges, and the dashboard widget badge so the in-app wording matches the email exactly. Keep the colour dots (red/amber/blue) as the at-a-glance signal.
- *Why it helps the owner:* An owner scanning the feed to decide what to do first must today translate 'Critical' vs 'Info' in their head and then sees different words again in the email; instead 'Urgent' / 'Needs attention' / 'Good to know' read the same on screen and in the inbox, so they instantly know an overdue invoice ('Urgent') outranks a quiet customer ('Good to know').
- **Give the dashboard widget a real error state instead of silently showing zero** -  High · effort S
 - *Today:* DashboardActionCenterWidget.vue has no error handling: fetchData() wraps the call in try/finally with no error ref, so if the API call fails the widget falls through to the 'You're all caught up' empty state (items.length === 0). The owner sees a reassuring green check and 'Nothing needs your attention' when in reality the data never loaded - they could have three overdue invoices and believe everything is fine.
 - *Change:* Add an `error` ref set in a catch block (and on `!res?.ok`), and render a third state in the widget: an amber line 'Couldn't check what needs attention' with a 'Try again' button calling fetchData() - mirroring the pattern already implemented on the full page (index.vue lines 201-205). Never let a failed load render as the all-caught-up state.
 - *Why it helps the owner:* An owner glancing at their dashboard on login to confirm nothing is on fire must today trust a green checkmark that can be a lie when the request fails; instead a visible 'Couldn't check - Try again' tells them the list is unverified, so they don't walk away from a silent overdue invoice believing they're caught up.
- **Add a labelled Action Center entry to the sidebar with a live count** -  High · effort S
 - *Today:* There is no sidebar link to /action-center (confirmed: only Header.vue and the dashboard widget reference it). The only ways back are the unlabelled bolt badge in the header - which has no visible tooltip on touch devices - and the dashboard widget title. An owner who dismisses the widget or doesn't recognise the bolt icon has no obvious path to the feature.
 - *Change:* Add a named sidebar item 'Action Center' (bolt icon) that links to /action-center, with the same live count badge the header already polls from /api/v1/action-center/count (red for critical, amber otherwise). Place it near the top, by Dashboard, since it's a daily-attention surface.
 - *Why it helps the owner:* An owner who wants to re-check their to-do list later in the day must today remember that a small unlabelled lightning icon in the header is the way in; instead they see a clearly labelled 'Action Center' in the main menu where they already look for everything else, with a number showing how many items are waiting.
- **Make low-stock and quiet-customer alerts say what to do, not just state a number** -  Med · effort M
 - *Today:* The low-stock subtitle reads 'Reorder threshold: 10' - 'threshold' and a bare number are technical and give no instruction. Separately, the at-risk-customer rule fires for every customer with no order in 90+ days, which floods the feed with 'Good to know' items that bury urgent ones, and its View link just opens the customer record with no suggested next step.

- *Change:* Rewrite the low-stock subtitle to an instruction: 'Only 3 left - you reorder when it drops below 10. Update this in the product's stock settings.' For quiet customers, cap the rule output low (e.g. top 5 highest-value lapsed customers rather than 50) so they never crowd out urgent items, and change the row action label from a generic 'View' to 'Send a re-order reminder' / 'View customer' pointing at the customer's contact/re-engage action.
- *Why it helps the owner:* An owner who sees a low-stock alert must today decode 'threshold: 10' and guess whether to act; instead they read 'Only 3 left - reorder below 10' and know immediately. And instead of scrolling past dozens of dormant-customer notices to find the one overdue invoice, they see only the few lapsed customers worth chasing, each with a clear 'send a reminder' action.
- **Offer flexible snooze and a way to see what's been snoozed** - ● Med · effort M
 - *Today:* The three-dot menu offers only a single fixed 'Snooze for 7 days' option (index.vue line 146), and once snoozed an item vanishes with no list of what was snoozed or when it returns. An owner who wants to defer a job alert until its actual due date, or who forgot what they snoozed, has no recourse - the alert silently reappears in a week regardless of relevance.
 - *Change:* Expand the snooze menu to 'Until tomorrow', 'For 3 days', 'For a week', and 'Until a date...' (date picker), and add a 'Snoozed' filter tab (alongside All/Urgent/etc.) listing snoozed items with their return date and an 'Un-snooze' action. The existing alert_dismissals snooze storage already records the snooze expiry, so the data is there to display.
 - *Why it helps the owner:* An owner who snoozes a 'proof awaiting sign-off' alert because they just chased the client must today accept a rigid 7-day reappearance and can never review or undo it; instead they snooze it to the exact follow-up date and can open a 'Snoozed' tab to see everything they've parked and when it'll come back.

Tenant Onboarding & Setup

- **Auto-create the first store at registration so the owner never lands on a dead-end dashboard** - ● High · effort M
 - *Today:* No store is created during sign-up. AuthController::register() creates only the tenant and seeds back-office defaults; StoreOnboardingService (which seeds the homepage, pages, sample products, shipping methods, and offline payment) only runs when the owner manually discovers Store Management > Add Store and submits the create form. Until they do that, the dashboard 'Get your store ready' checklist is broken: resolveStore() returns null, so checkLogoStep/checkShippingStep/checkPaymentGatewayStep all silently report 'incomplete', the 'Configure Shipping' link has no store ID to build a URL from, and there is no 'Create your first store' prompt anywhere on the dashboard. A non-technical owner is told to configure shipping but the link goes nowhere.
 - *Change:* After the tenant is created and approved (auto-approve path immediately, manual-approve path inside AccountStatusService::approveTrial), automatically create one primary store named after the business and run app:new-store-created so StoreOnboardingService::seedAll() seeds the full storefront. Remove the requirement that the owner manually create their first store; keep 'Add Store' available only for adding additional stores. The checklist then resolves a real store on first login and every step link works.

- *Why it helps the owner:* A shop owner who just signed up today logs in to a dashboard whose checklist can never be completed because half the steps point at a store that does not exist yet, and nothing tells them to create one. Instead, they would log in to a working store with a real homepage and a checklist whose every step opens the correct page, so their first session is 'add my logo and a product', not 'figure out why the buttons are broken'.
- **Stop counting the seeded demo products as the owner's 'Add your first product' step** - ● High · effort S
 - *Today:* checkProductStep() in useSetupChecklist.ts marks 'Add your first product' complete as soon as /products total > 0, but TenantOnboardingService seeds 4 demo (is_demo) products at registration. So a brand-new owner who has added nothing sees that step pre-ticked and a false '1 of 5 complete', while their real catalogue is empty. The products page separately shows an amber 'sample products' banner urging them to remove those very products - the two surfaces contradict each other.
 - *Change:* Change the product step to count only non-demo products: pass a filter (e.g. ? exclude_demo=1 or is_demo=0) to the /products check so the step stays incomplete until the owner adds or publishes a real product, or edits a sample into a real one. Keep the demo products for storefront preview, but do not let them satisfy the setup milestone.
 - *Why it helps the owner:* An owner trying to understand how much setup is left sees the checklist claim a product is already added when they have not touched the catalogue, so they may skip the most important step and go live with only sample products. Instead the step stays open until they have a genuine product, matching the 'remove samples' banner and guiding them to the one task that actually makes their store sellable.
- **Replace the seeded placeholder storefront content that would otherwise go live verbatim** - ● High · effort M
 - *Today:* StoreOnboardingService seeds customer-facing content that is wrong-by-default and easy to miss: the homepage Stats block shows hardcoded '10,000+ Orders Delivered', '2,847 Orders This Week', '4.9 Average Rating'; the footer social links point to generic platform homepages (facebook.com, instagram.com); and the hero banner has no image, falling back to an empty placeholder background. A non-technical owner who does not edit these will publish fake trust numbers and dead social links to real customers.
 - *Change:* Seed honest neutral defaults instead of fake specifics: omit or hide the Stats block until the owner enters real numbers (or seed it empty with editable zero/'-' values and a clear 'Add your stats' prompt in admin), set footer social fields to empty with placeholder helper text 'Add your Facebook link' rather than a live generic URL so empty ones simply do not render on the storefront, and either seed a tasteful default hero image or surface an inline admin prompt on the hero block ('Add a hero image - recommended 1600x600') so the empty hero is obviously a to-do, not a finished section.
 - *Why it helps the owner:* An owner who previews their store and thinks it looks finished will unknowingly publish '4.9 Average Rating' they never earned and social icons that send customers to facebook.com's homepage. Instead the starter store contains nothing false; anything still blank visibly reads as 'fill this in', so they cannot accidentally go live with fake stats or dead links.

- **Make the pending-approval wait self-serve instead of an email-only black box** - ● Med · effort M
 - *Today:* With manual approval (`auto_approve_signups=false`) the owner is redirected to `/auth/pending-approval`, a holding page with four 'while you wait' tips, no way to check status, and no link into the dashboard even for read-only browsing - they can only wait for an email. Worse, subscription creation is commented out in `register()` and only happens at approval time, so a manually-approved tenant can reach the dashboard with no active subscription and hit opaque 402 responses from `EnsureTenantHasActiveSubscription` instead of a clear message.
 - *Change:* On the pending-approval page add a 'Check approval status' button that re-fetches the account status and, once approved, routes straight into the dashboard, plus a 'Take a look around' read-only link so they can explore while waiting. Ensure approval always creates the trial subscription before the owner can act (it does via `approveTrial`, so the gap is the manual path reaching the dashboard pre-subscription); have `EnsureTenantHasActiveSubscription` return a plain-language 'Your free trial starts as soon as we approve your account - we will email you' message keyed on `TRIAL_REQUESTED` instead of a bare 402.
 - *Why it helps the owner:* An owner waiting for approval today has no idea whether minutes or days remain and cannot do anything but refresh their inbox; if they slip through early they hit cryptic 402 errors. Instead they can press 'Check status' to get in the moment they are approved, browse their seeded store while waiting, and never see a raw payment error during the trial.
- **Fix the 'Onboard' redirect to explain in plain language what business detail is missing** - ● Med · effort S
 - *Today:* `auth.ts` redirects any user with status 'Onboard' to `/setting/account` with the toast 'Please update business details to use system.' - developer phrasing, and the destination is the full account settings form with no indication of which fields are required or why they were sent there. A non-technical owner sees a generic settings page and a vague instruction with no clear finish line.
 - *Change:* Change the toast to plain language ('Finish setting up your business details to unlock your dashboard') and, on `/setting/account` when arriving in the Onboard state, show a short banner at the top naming exactly what is still needed and a single 'Save and continue' button that flips the user out of Onboard status and returns them to the dashboard, rather than leaving them in an open-ended settings form.
 - *Why it helps the owner:* An owner sent to the settings page today reads broken-English developer instructions, does not know which of the many fields matter, and has no obvious way to get back to the dashboard. Instead they see exactly the two or three fields to fill, save once, and are taken straight to their store - a finishable task instead of a confusing detour.
- **Show the full plan feature list and a clear empty/error state in the registration plan picker** - ● Low · effort S
 - *Today:* Step 1 of `/register` renders each plan from `/public/plans` but the feature list shows only `plan.features.description[0]` (the first element), so the owner sees a single line and cannot tell what each tier actually includes. If `/public/plans` is empty the form skips silently to step 2 with no explanation, and there is no visible error/empty state for the plan fetch.

- *Change:* Render the full plan.features list (or the documented set of included features) per plan card with checkmarks instead of just the first line, and when no plans are returned show a brief 'You'll start on our standard free trial' note rather than silently skipping the step, plus a plain-language retry state if the fetch fails.
- *Why it helps the owner:* An owner choosing a plan today sees one cryptic line per tier and cannot judge what they are paying for, or the plan step vanishes with no word; instead they see a clear feature comparison and always understand which plan (or default trial) they are starting on before they commit.

Integrations & Payment Gateways

- **Show the webhook URL with a one-click copy and "why this matters" note on every payment gateway** -  High · effort M
 - *Today:* The webhook URL (`https://api/v1/webhooks/payments/{gateway_key}`) is never shown anywhere in the admin UI. To make Stripe or Razorpay actually confirm payments, the owner must already know to copy this URL from documentation and paste it into the gateway's own dashboard. If they miss it or paste the wrong one, orders silently sit on 'payment pending' forever with no explanation, and the owner has no way to discover what went wrong.
 - *Change:* In the payment gateway configure form (`PaymentGatewayCard.vue` and the integrations form for payment-category services), add a clearly labelled, read-only 'Payment confirmation URL' field that renders the live per-store webhook URL with a copy button and a plain-language helper: 'Paste this into your Stripe dashboard under Developers > Webhooks so we know when a customer has paid. Without it, paid orders may stay marked as Pending.' Only render it for gateways that use webhooks (redirect drivers), not for offline/Cheque.
 - *Why it helps the owner:* A store owner setting up Stripe today must already know a webhook exists, find the URL in docs they won't read, and reconstruct their own store host by hand; instead they copy one labelled URL straight from the gateway card and paste it where the helper tells them to, so paid orders actually flip to Paid instead of getting stuck on Pending.
- **Add a "Test connection" / "Send test" button after saving credentials** -  High · effort L
 - *Today:* There is no way to confirm credentials work after saving. After entering Stripe keys (or SMTP, or a shipping carrier) and clicking Save, the only feedback is 'settings saved successfully' - which only means the form stored the text, not that the keys are valid. The owner learns the keys are wrong only when a real customer's checkout fails or a real order email never arrives.
 - *Change:* On the configure form footer for each integration, add a 'Test connection' button next to Save (for email: 'Send test email to me'; for payment: a sandbox auth/ping via the existing driver registry; for shipping: a sample rate quote). Show a plain result inline: green 'Connection works - these credentials are valid' or red 'These credentials were rejected by Stripe. Double-check your Secret key and Mode (live vs test).' Disable the activate toggle's 'success' state messaging until a test has passed, or at least nudge them to test.
 - *Why it helps the owner:* A shop owner who fat-fingers a Stripe secret key today only finds out when their first real customer can't pay; instead they click 'Test connection', see it fail immediately with a hint about which field is wrong, and fix it before a single order is lost.

- **Block activation of a half-configured gateway and warn on sandbox-in-production** - ● High · effort M
 - *Today:* The activate toggle on both the Integrations index (toggle(item)) and the gateway card turns a service 'on' regardless of whether required fields are filled, and Mode (sandbox/live) is just a bare select with no warning. An owner can flip Stripe to Active with missing keys, or leave it in 'sandbox' mode with live keys - and the gateway then appears as a real payment option at checkout where every transaction silently fails.
 - *Change:* Prevent toggling an integration to Active while its configuration_status is 'incomplete' - instead of activating, show a toast/inline message 'Stripe still needs your Secret key before it can go live' and open the configure form. When a payment gateway is set to Active in 'sandbox'/'test' Mode, show a persistent amber banner on the card and at checkout-config level: 'This gateway is in Test mode - real customers will not be charged. Switch to Live when you're ready to take real payments.'
 - *Why it helps the owner:* An owner who toggles Stripe on before finishing setup, or forgets it's in test mode, today exposes a broken or fake payment option to real customers with no warning; instead they're stopped at the toggle with a plain reason, and a standing 'Test mode' banner makes the live/sandbox mistake impossible to miss.
- **Merge the duplicate Payment Gateways page into the Integrations hub (or make it a filtered view) and fix its deep links** - ● High · effort M
 - *Today:* Payment gateways can be configured from two unrelated places: Settings > Integrations (the canonical hub) and Settings > Payment Gateways (/setting/payment-gateway), which is not in the side-nav, has no breadcrumb back, and is only reachable via deep links from Invoices/Quotes. An owner who sets up Stripe in one place may never realise the other exists, and the two can drift out of sync in their mind.
 - *Change:* Make /setting/payment-gateway a thin redirect (or alias) to the Integrations hub pre-filtered to the Payment category (activeCategory='payment'), and repoint the 'Set up a payment gateway →' deep links on Invoice and Quote pages to that same filtered hub URL. Keep one canonical place to manage payment gateways. If the simpler card layout is preferred for payments, render it inside the hub under the Payment section rather than on a separate orphan page.
 - *Why it helps the owner:* A shop owner who clicks 'Set up a payment gateway' from an unpaid invoice today lands on an orphan page with no way back and no idea it duplicates the Integrations hub; instead every path leads to the one Payments area, so they never wonder whether they configured Stripe 'in the right place'.
- **Replace the raw service_key subtitle and unhide/repair the Authorize.Net dead-end** - ● Med · effort S
 - *Today:* PaymentGatewayCard.vue line 17 prints the internal service_key ('stripe', 'razorpay', 'authorizenet') as the card subtitle - a developer identifier a shop owner reads as a bug. Separately, Authorize.Net is offered as a checkout option but its embedded driver only shows 'inline checkout is not yet available - please choose another payment method', so an owner who activates it as their only gateway sends every customer into a dead-end.

- *Change:* In `PaymentGatewayCard.vue`, replace the `service_key` subtitle with a human descriptor (the category label 'Payment provider', or the template's short description), never the raw key. For [Authorize.Net](#): since the embedded flow is unfinished, either hide it from the catalog/checkout entirely behind a feature flag, or mark its card 'Coming soon - not yet available for live checkout' and prevent activating it, so it can never silently become a customer's only option.
- *Why it helps the owner:* A shop owner reading 'authorizenet' under a gateway name today assumes something is broken; and one who picks [Authorize.Net](#) as their gateway dead-ends every customer. Instead the card reads in plain language, and the unfinished gateway can't trap them or their buyers.
- **Make the configuration status bar tell owners exactly which field is missing, with a direct fix link** - ● Med · effort M
 - *Today:* The Configuration bar shows colour-coded badges (Configured / Partial / Incomplete) and a percentage-style width, but never names the specific missing required field. An owner who sees 'Incomplete' or 'Partial' has to click Configure and visually hunt the form for the blank required input, with no idea how many fields remain.
 - *Change:* Have `IntegrationResource.php` return the list of missing required field labels (it already counts them server-side), and on the card render them in plain language under the status bar: 'Still needed: Secret key, Webhook signing secret' with a 'Finish setup →' button that deep-links into the configure form. Update the 'X need setup' header badge tooltip to say what 'setup' means ('credentials still missing - customers can't use these yet').
 - *Why it helps the owner:* An owner staring at a 'Partial' Stripe card today must open the form and scan every field to find the one blank box; instead the card tells them 'Still needed: Webhook signing secret' and one click takes them straight to finishing it.

Storefront Themes & CMS

- **Make Typography, Layout, and Buttons reachable from the section tabs** - ● High · effort S
 - *Today:* In `theme.vue` the sticky section nav (`sections` array, lines 699-709) lists Preset, Footer, Products, Banners, Homepage, Category, Cart, Social, and Custom Code - but NOT Typography, Layout, or Buttons. Those three sections render in the page (they exist in `openSections` and in the HTML) and hold core look-and-feel controls (fonts, heading scale, border radius, container width, button style, shadow), yet there is no tab that jumps to them. An owner can only reach them by scrolling past the Preset picker and stumbling onto them.
 - *Change:* Add three entries to the `sections` array immediately after Preset: `{ id: 'typography', label: 'Fonts & Text' }`, `{ id: 'layout', label: 'Layout' }`, `{ id: 'buttons', label: 'Buttons' }`, each with the matching icon and keywords, so they appear as clickable tabs and the existing scroll-to/active-highlight behaviour works for them like every other section.
 - *Why it helps the owner:* A store owner who wants to change their storefront's heading font or make corners less rounded today has to scroll blindly past the theme picker and hope they find the right controls; instead they could click a clearly labelled 'Fonts & Text' or 'Layout' tab and land directly on those settings, the same way they reach Banners or Cart.
- **Replace music-theory 'Heading Scale' labels with plain visual descriptions** - ● High · effort S

- *Today:* The Heading Scale dropdown (theme.vue lines 211-212) offers `['minor-second', 'major-second', 'minor-third', 'major-third', 'perfect-fourth']` as both the values AND the labels. These are typographic ratio names that mean nothing to a print-shop owner - there is no hint that they control how much bigger headings are than body text.
- *Change:* Keep the same stored values but render human labels in the USelectMenu: 'Subtle (headings slightly larger)', 'Gentle', 'Balanced - recommended', 'Bold (clear size jumps)', 'Dramatic (very large headings)'. Map label → value so saved data is unchanged, and mark the current default (`major-third`) as 'Balanced - recommended'.
- *Why it helps the owner:* An owner who wants their product page headings to stand out more today sees five cryptic terms like 'perfect-fourth' and has to guess; instead they could pick 'Bold (clear size jumps)' and understand exactly what they'll get without trial-and-error refreshing of the storefront.
- **Let owners set banner autoplay in seconds, not milliseconds** -  High · effort S
 - *Today:* The banner autoplay field (theme.vue line 399-400) is labelled 'Autoplay Interval (ms)' and is a raw number input with `step=500`, defaulting to 6000. A non-technical owner does not know that 6 seconds means typing 6000, and may enter '6' expecting six seconds - producing a banner that flips every 6 milliseconds.
 - *Change:* Change the label to 'Seconds between slides', display the value divided by 1000 (so 6000 shows as 6), accept whole/half seconds, and convert back to milliseconds on save. Add helper text 'How long each banner stays on screen before sliding to the next.'
 - *Why it helps the owner:* An owner setting up a rotating homepage banner today must know the hidden rule that the unit is milliseconds and multiply by 1000; instead they could type '6' for a six-second slide and trust it does what it says.
- **Cross-link Banner content and Banner behaviour so owners find both halves** -  Med · effort S
 - *Today:* Banner images and text (the content) live in Banner Management, while banner autoplay/height/animation live in Theme Settings > Banners - two completely separate screens with no mention of each other. An owner who changes a banner image and wants it to rotate faster has no way of knowing the speed control is on a different page entirely.
 - *Change:* Add a short banner at the top of Theme Settings > Banners: 'These settings control how banners behave. To change banner images and text, go to Banner Management →' (linking to the store's Banners page). Add the reverse note on the Banner Management list: 'To change autoplay speed, height, and animation, go to Theme Settings > Banners →'. Both links absolute and store-scoped.
 - *Why it helps the owner:* An owner who just uploaded a new hero image and wants it to auto-rotate must currently discover, by luck, that the speed setting lives on an unrelated screen; instead each screen points them straight to its other half so they can finish the job in one sitting.
- **Guard the Custom Code section and warn before it can break the storefront** -  Med · effort S
 - *Today:* The Custom Code section (CSS, head scripts, body scripts) is shown to every owner with no warning. A non-technical owner who pastes the wrong thing there can break their live storefront, and the UI shows no caution or recovery path.

- *Change:* Collapse Custom Code behind an 'Advanced - for developers' disclosure that is closed by default, with a plain-language note: 'These boxes are for code from a developer or an analytics tool (like Google Analytics). Pasting the wrong thing here can break how your store looks. If you're not sure, leave these empty.' Keep the existing fields unchanged underneath.
- *Why it helps the owner:* An owner exploring theme settings today can stumble into raw code boxes and accidentally paste something that breaks their storefront; instead the section is clearly fenced off as developer-only and they are warned before touching it, so the 90% who never need it are never at risk.
- **Show whether a page is live to customers in the Delete Page confirmation, and protect against a missing preset** - ● Med · effort M
 - *Today:* Two silent gaps: (1) the Delete Page modal names the page and warns 'cannot be undone' but never says whether the page is currently published and visible to shoppers, so an owner can't tell if deleting will immediately break a link customers are using. (2) Only 'Default' and 'Aurora' presets exist; if a store still has a retired preset id ('elegant'/'classic') saved, the storefront silently loads no preset CSS with no visible signal in Theme Settings.
 - *Change:* In the Delete Page confirmation, add a line driven by the page's status: if published, 'This page is LIVE - customers can see it now. Deleting it will immediately show a Not Found error to anyone visiting its link.'; if draft, 'This page is a draft and is not visible to customers.' Separately, in Theme Settings, if the saved preset id is not one of the registered presets, show an inline amber notice on the Preset section: 'Your saved theme is no longer available, so your store is using the basic style. Pick a theme below to fix this.' and default the selection to Default.
 - *Why it helps the owner:* An owner deleting an old page today can't tell whether they're about to break a link customers are actively visiting; instead the modal tells them plainly whether it's live. And an owner whose store quietly lost its theme styling sees a clear 'pick a theme to fix this' prompt instead of an unexplained plain-looking storefront.

Storefront Cart & Checkout

- **Warn the owner in the admin when their store has no payment method (and link to fix it)** - ● High · effort M
 - *Today:* If no payment gateway is connected, checkout silently shows customers an amber box: "The store hasn't set up any payment options yet. Please contact the store owner..." (frontstore/app/components/checkout/CheckoutPayment.vue:10-11). The owner gets NO warning anywhere in admin - no banner on the dashboard, store settings, or theme editor. They only find out when a customer complains they can't pay. The box also has no link, violating [CLAUDE.md](#)'s 'guide to where to configure' rule.
 - *Change:* Two coordinated changes. (1) In the admin: add a persistent amber alert on the Store Settings page header and the dashboard Action Center - "Customers can't pay yet: no payment method is connected. Connect Stripe, Razorpay, or enable Cheque / Local Pickup → Integrations" linking to the absolute Integrations path. (2) In the storefront box (CheckoutPayment.vue): when the viewer is the logged-in store owner, swap the dead-end copy for the same actionable line with a link to Integrations; keep the polite 'contact the store owner' copy for customers.

- *Why it helps the owner:* A store owner who just finished setting up products and shared their store link today has no idea their store can't take a single order - they discover it from an angry customer. Instead they'd see a clear red flag the moment they open admin, with a one-click path to connect payments before they lose a sale.
- **Replace developer terms 'single-page / accordion / multi-step' with plain labels + a one-line preview of each** - ● High · effort S
 - *Today:* The checkout layout picker in Theme Editor (nuxt/.../theme.vue:541-542) is a bare dropdown of raw values ['single-page', 'multi-step', 'accordion'] with no description. 'Accordion' and 'multi-step' are developer words a print-shop owner cannot picture, so they either leave the default blindly or pick at random without understanding what their customers will see.
 - *Change:* Change the options to plain labels with a one-sentence description each: 'All on one page - fastest, everything visible at once (recommended)', 'Step by step - guides the customer through a few short pages', 'Collapsible sections - one section open at a time, tidy on phones'. Render them as labelled radio cards (or a select with help text), not raw kebab-case strings. Keep the stored values unchanged so nothing breaks.
 - *Why it helps the owner:* A store owner deciding how their checkout should feel today is shown three meaningless engineering words; instead they read a plain description of what each does and can choose confidently - e.g. picking step-by-step because most of their buyers are on phones.
- **Gather all checkout controls into one 'Checkout' settings area instead of splitting them between Features and Theme Editor** - ● High · effort M
 - *Today:* Checkout behaviour is scattered: guest checkout, request-a-quote, shipping requirement, and contact mode live under Store Settings → Features (settings.vue, alongside reviews/B2B/uploads), while checkout layout style, free-shipping threshold, and option preview limit live under Theme Editor → Cart (theme.vue). An owner who wants to 'set up checkout' has no single place to go and must hunt across two different admin pages.
 - *Change:* Create one 'Cart & Checkout' settings group (a section or sub-tab) that surfaces all of these in one scroll: guest checkout, request-a-quote, login requirement, shipping requirement, contact mode, layout style, free-shipping threshold, and option preview limit. The layout/threshold fields can remain theme-backed under the hood, but present them together here so the owner sees every checkout decision in one view.
 - *Why it helps the owner:* A store owner setting up checkout for the first time today has to know that guest checkout is under 'Features' but layout is under 'Theme Editor → Cart' - two pages, no signpost. Instead they open one 'Cart & Checkout' area and configure everything about how customers buy in a single sitting.
- **Default guest checkout ON so first-time buyers aren't silently turned away** - ● High · effort S
 - *Today:* Guest checkout (quick_order_enabled) defaults to false in the backend config, while the theme defaults object (theme.vue:1229) sets guest_checkout: true - a confusing split-brain default. The effective behaviour for most owners is that a new store forces account creation, so a first-time buyer who just wants one batch of business cards hits a login wall and abandons. The owner never chose this and gets no nudge that it's costing them sales.

- *Change:* Make guest checkout ON by default for new stores (align the backend `quick_order_enabled` default to true, matching the theme default that already says true), and resolve the two defaults to a single source of truth. Add a short helper line under the toggle: 'Off means customers must create an account before buying - this turns away many first-time shoppers.'
- *Why it helps the owner:* A store owner who assumes 'of course people can just buy' today unknowingly forces sign-up and loses first-time customers with no warning; instead the sensible default lets buyers check out immediately, and the helper text makes the trade-off clear if they ever want to require accounts.
- **Make the coupon field reachable on mobile checkout instead of buried in the order-summary drawer** - ● Med · effort S
 - *Today:* The coupon input lives only inside `CheckoutSummary.vue`, which on mobile is collapsed into the 'Order summary' drawer at the top of the page. A phone customer holding a promo code the owner advertised has to know to tap 'Order summary' to reveal it - many won't, so the discount the owner promoted never gets applied and they look unreliable.
 - *Change:* On mobile, surface a visible 'Have a promo code?' link/field in the main checkout flow (near the totals or place-order button), expanding inline - not hidden behind the summary drawer. Reuse the existing coupon validation logic; only the placement changes so the field is discoverable without opening the drawer.
 - *Why it helps the owner:* A store owner who runs a '10% off' promo today watches mobile customers fail to find where to enter the code and pay full price (or abandon, feeling tricked); instead the code field is right there in the checkout flow, so the discount they advertised actually gets used.
- **Turn the 'Failed to lock designs' checkout error into plain language with a working retry** - ● Med · effort S
 - *Today:* When the pre-submission POST `/api/designer/designs/lock-for-checkout` step fails, the customer sees a toast: 'Failed to lock designs. Please try again.' A non-technical buyer has no idea what 'locking a design' means or how to retry meaningfully - and the store owner gets a stuck customer at the very last step, the most expensive place to lose an order.
 - *Change:* Replace the message with plain, action-oriented copy: 'We couldn't finish saving your artwork for this order. Please tap Place Order again - if it keeps happening, refresh the page or contact us.' Make the retry actionable (re-attempt the lock automatically on the next Place Order press rather than leaving the button in a dead state), and surface the store's contact email/phone if it fails twice so the customer has a path forward.
 - *Why it helps the owner:* A store owner loses a ready-to-buy customer at the final click today because a cryptic 'lock designs' error gives the buyer nothing to act on; instead the buyer reads what to do next and can retry or reach the shop, so a near-complete order doesn't evaporate.

Designer Studio

- **Show "Customers can't personalise this yet" right on the product, with a one-click fix** - ● High · effort M

- *Today:* To let customers personalise a product, the owner must open the product, scroll to the 'Designer' accordion in the Settings tab, flip the 'Enable Designer' quick-toggle, then dig into the separate 'Studio' sub-tab to switch on Mini Studio. Nothing on the product list or product header tells them the designer is currently off, and the Designer section just says 'Enable the Designer toggle above' without pointing to which toggle. A non-technical owner has no way to know personalisation is missing or where to switch it on.
 - *Change:* Add a plain-language status line at the top of the product editor and a small badge on each product list row: when `enable_designer` is false, show 'Customers can't personalise this product yet' with an 'Turn on online designer' button that flips `enable_designer` (and offers to also enable Mini Studio). When it is on, show 'Customers can personalise this online' in green. In the Designer section, replace 'Enable the Designer toggle above' with an inline 'Turn on online designer' button so the owner fixes it in place instead of hunting for the toggle.
 - *Why it helps the owner:* A shop owner who just added 'Premium Business Cards' and wants buyers to type their own name and number can see at a glance that personalisation is off and switch it on with one click from the product itself, instead of guessing across two tabs and a sub-tab.
- **Replace raw 'System error' and the missing 'message.exportError' key with plain-language messages and a retry** - ● High · effort S
 - *Today:* When a PDF preview export fails, the designer shows the literal `i18n` key 'message.exportError' because that key is missing from `designer/src/plugins/i18n/lang/en.ts`, and the API error handler in `designer/src/utls/request.ts` (lines 290 and 336) falls back to the hardcoded string 'System error'. A customer designing their flyer sees developer text with no idea what to do next, and the shop owner gets a confused support call.
 - *Change:* Add the missing 'exportError' and 'previewPdfReady' entries to `en.ts` with shopper-friendly copy, and change the two 'System error' fallbacks in `request.ts` to a plain message like 'Something went wrong on our end. Please try again in a moment.' Pair every one of these with a visible 'Try again' button that re-runs the export/save, so the customer is never stuck on a dead end.
 - *Why it helps the owner:* A customer finishing their business-card design and hitting a momentary export hiccup sees 'We couldn't generate your preview - try again' with a button, completes their order, and the shop owner never fields a 'your site is broken' message about cryptic text.
 - **Reassure the customer their design is saved before forcing them to log in** - ● High · effort S
 - *Today:* On the Design Options page, when a guest clicks 'Add to Cart' and quick-order is off, `handleAddToCart` immediately pops a `LoginModal` with no explanation. The customer who just spent ten minutes designing has no idea why login is suddenly required or whether the design they just made will survive logging in - a common moment to abandon.
 - *Change:* Before opening the login modal, show one line of reassurance inside it: 'Your design is saved. Sign in or create a free account to add it to your cart - you won't lose your work.' Since the design already has a `design_id` persisted to the backend, this is true and the modal already supports re-running `handleAddToCart` on success.
 - *Why it helps the owner:* A first-time customer who designed a custom t-shirt and hits the login wall is told their work is safe and why the account is needed, so they sign up and complete the order

instead of closing the tab - directly protecting the shop owner's sale.

- **Rename 'Material Library' to 'Logos & Graphics' in the Designer Studio hub** - ● Med · effort S
 - *Today:* The Designer Studio hub at /setting/designer labels the cliparts card 'Material Library' (setting/designer/index.vue line 40). A print-shop owner looking to upload a logo or decorative graphic for customers to drop into designs would not connect 'Material Library' to clipart, and the internal category name 'clipart' leaks through the URL and API.
 - *Change:* Rename the card title from 'Material Library' to 'Logos & Graphics' (or 'Graphics & Cliparts') with a one-line subtitle 'Logos, icons and graphics customers can add to their designs.' Keep the underlying clipart route/API unchanged so nothing breaks; this is a label and description change only.
 - *Why it helps the owner:* A shop owner who wants to make their company logo available inside the designer scans the hub, recognises 'Logos & Graphics', and finds it immediately instead of overlooking a 'Material Library' card that means nothing to them.
- **Tell the customer their design is being prepared instead of showing a blank space** - ● Med · effort M
 - *Today:* After saving from the designer, design-options.vue polls up to 30 seconds for the preview image. The DesignPreviewPanel does render a spinner while isPreviewLoading is true, but it only mounts when artworkMethod==='design' AND (hasDesignPreview OR isPreviewLoading); if the design has no preview yet and a timing edge leaves isPreviewLoading false, or after the 12-attempt window quietly ends, the left column collapses and the customer sees no preview and no message - reading as 'my design was lost'.
 - *Change:* Always reserve the preview column when a design_id is present, and make the panel show an explicit 'Preparing your design preview...' state during polling and a 'Preview is taking a little longer - your design is saved, you can keep configuring your order' message (with a manual 'Refresh preview' button) if the 30-second poll ends without an image, instead of silently stopping.
 - *Why it helps the owner:* A customer who configures size and quantity while their preview renders is never faced with an empty box that looks like their design vanished, so they trust the order and the shop owner avoids 'where did my design go' complaints.
- **Warn the admin before they lose an unsaved template when the designer tab is closed** - ● Med · effort M
 - *Today:* Creating a template opens the full designer SPA in a new browser tab carrying admin_token/template_id/return_url, with no in-context instruction. If the owner accidentally closes that tab before saving, the template silently doesn't save and they get no warning - they return to the template list assuming their work was kept.
 - *Change:* In the designer SPA, add a beforeunload guard that prompts 'You have unsaved design changes - leave anyway?' when the canvas has unsaved edits, and on the template list page show a short inline note when launching ('Your template opens in a new tab. Click Save before closing it.'). On return via return_url, show a clear 'Template saved' confirmation so the owner knows it worked.

- *Why it helps the owner:* A shop owner building a flyer template who closes the wrong tab is stopped and reminded to save, instead of losing twenty minutes of layout work and having no idea why their template never appeared in the list.

AI Product Builder

- **After AI creates the product, ask ‘Add to your store now?’ instead of leaving it invisible to customers** - ● High · effort M
 - *Today:* AiProductCreatorService creates the Product with status=‘active’ but never inserts a store_products pivot row, so the product is invisible on the storefront. The result screen’s action card (product-builder.vue lines 305-333) says only ‘Edit the product to add images, adjust pricing, set up rules, and publish it’ and never mentions store assignment. A non-technical owner sees ‘Product created successfully’, assumes it’s live, and is baffled when customers can’t find it.
 - *Change:* On the result screen, add a primary ‘Add to my store’ button (or a checkbox ‘Make this visible on my storefront’ defaulted on) that calls the existing store_products assignment endpoint for the owner’s primary store. Until it’s assigned, show a plain-language status pill on the result card: ‘Saved to your catalog, but not on your website yet.’ Once assigned, change it to ‘Live on [Store Name]’. Wire the same assignment into AiProductCreatorService::create() as the default for single-store tenants.
 - *Why it helps the owner:* A store owner who just described ‘Black hoodie S to 2XL’ and saw ‘created successfully’ today must somehow know to open the editor, scroll to a ‘Stores’ section, tick their store, and re-save before any customer can buy it – with no hint that step exists. Instead they click one ‘Add to my store’ button (or it’s pre-ticked) and the hoodie is actually for sale.
- **Replace the developer ‘AI confidence: 73%’ line with a plain next-step the owner can act on** - ● High · effort S
 - *Today:* The success banner (product-builder.vue lines 171-174) prints ‘AI confidence: {{ result.confidence }}%’ next to the AI’s raw reasoning sentence. A print-shop owner has no mental model for an ‘AI confidence score’ and no idea what to do if it reads 35%. Worse, the create threshold is only 30, so genuinely shaky products (confidence 30-50) are created and presented with the same celebratory green banner as a confident 95.
 - *Change:* Drop the raw percentage from the UI. Map confidence to a plain verdict: >=70 -> green ‘Looks complete – review and add your artwork’; 40-69 -> amber ‘I made some guesses – please double-check the print size and options before publishing’; 30-39 -> amber ‘I wasn’t very sure about this one. Check the size, options, and price carefully.’ Keep the numeric value only in the AI Usage log, where it’s a diagnostic for support.
 - *Why it helps the owner:* An owner who gets back a 38%-confidence product today sees a green ‘success’ badge and a number that means nothing, so they publish a half-wrong product. Instead they see ‘I wasn’t very sure – check the print size before publishing’ and know exactly what to verify.
- **Make the result screen show the store’s real currency and humanise the type and pricing labels** - ● High · effort M

- *Today:* AiBuilderResultResource line 76 hardcodes 'currency' => 'USD', and product-builder.vue lines 203 and 294 hardcode a '\$' symbol – so a GBP or INR shop sees the wrong currency on every AI result. The type badge (line 193) renders the raw enum ('ecommerce', 'hybrid') with CSS capitalize, and the pricing label (line 296) shows 'pricing_suggestion' with only one underscore replaced ('hybrid_area_quantity' -> 'hybrid area quantity'). All three are developer values leaking to a non-technical owner.
 - *Change:* Return the store's configured currency code from the resource (use the store currency already threaded elsewhere) and format both prices through the admin money formatter instead of a literal '\$'. Map the type enum to plain labels: printing->'Print Product', tshirt->'Apparel', ecommerce->'Merchandise', service->'Service', hybrid->'Print + Merchandise'. Map pricing_suggestion to a sentence: quantity_based->'Price changes with quantity', area_based->'Price changes with print size', hybrid_area_quantity->'Price changes with size and quantity', flat->'Single fixed price'.
 - *Why it helps the owner:* An owner running a UK shop who creates an A4 poster today sees '\$12.00' and a badge reading 'Hybrid' – neither matches their world. Instead they see '£12.00', 'Print + Merchandise', and 'Price changes with size and quantity', and can trust what they're reading at a glance.
- **Surface the auto-applied 60/40 cost-and-margin split on the result, instead of setting it silently -**
 - Med · effort S
 - *Today:* AiProductCreatorService hardcodes cost_price = base_price x 0.6 and profit_margin = 40 for every product, and nothing on the result screen tells the owner this happened. The pricing card (lines 289-301) shows only the base price. An owner whose real material cost is, say, 80% of price is now carrying a wrong margin in their books and has no idea it was invented for them.
 - *Change:* In the pricing card, show a small editable summary: 'Estimated cost: [cost] · Your profit: [margin]%' with helper text 'We estimated this – update it to match your real costs.' and an inline link to the pricing section of the editor. Keep 60/40 as the default, but make it visible and one click to correct.
 - *Why it helps the owner:* A shop owner whose hoodies cost more than the AI assumed has, today, a silently wrong profit margin that only surfaces when their numbers don't add up months later. Instead they see 'We estimated your cost at X – update it' right on the result and fix it in seconds.
 - **Add an 'AI Product Creator' entry point on the Products list page, where owners actually go to add products -**
 - Med · effort S
 - *Today:* The only entry point is Header.vue's Catalog -> AI dropdown section (line 181); the Products list page has no mention of it. A non-technical owner trying to add a product opens the Products page and looks for an 'Add' button – 'AI' in a dropdown is not where their intuition sends them, so the feature goes undiscovered.
 - *Change:* On the Products list page, next to the existing 'Add Product' button, add a secondary 'Create with AI' button (sparkles icon) linking to /ai/product-builder. On the empty state of the Products list, feature it prominently: 'Describe a product and let AI set it up for you.'
 - *Why it helps the owner:* An owner with zero products today stares at an empty Products list and a single manual 'Add Product' form, never realising they could just type 'Black hoodie S to 2XL'.

Instead the AI option sits right where they're already standing, at the moment they're trying to add a product.

- **Make the setup wall self-explanatory and tie the loading bar to real progress** - 🟡 Med · effort M
 - *Today:* Two separate gaps hit non-technical owners. (1) The not-ready banner (lines 34-47) just shows the API message plus a 'Settings -> Integrations' link, with no plain-language explanation of what Anthropic/OpenAI is, where to get a key, or roughly what it costs – most owners stop here. (2) The loading animation (startAnimation, lines 382-390) advances on a fixed 2s timer to 90% and the final 'Save' step never checks until the call returns; if the AI takes longer than ~8s the owner sees a stalled bar and assumes it broke.
 - *Change:* (1) Replace the bare banner with a short plain-language card: 'This feature uses an AI assistant (like ChatGPT or Claude) to set up products for you. You'll need a free account key from one of them – here's how (2 minutes).' with a step-by-step link, so the owner isn't dropped onto an integrations page cold. (2) Keep the orb animation but cap the fake progress at ~85% and, once it lingers, swap the caption to a calm 'Still working – complex products can take a few more seconds...' so a slow call never reads as a freeze; jump to 100% only on the real response.
 - *Why it helps the owner:* An owner who's never heard of 'Anthropic' hits a red banner today and quits before typing anything; and an owner with a detailed prompt watches a bar freeze at 90% and wonders if it crashed. Instead they get a plain explanation of what to connect and why, and a loading state that reassures rather than alarms when the AI takes its time.

PDF / Artwork Pipeline

- **Stop the Print File Checker from looking broken when its server flag is off** - 🔴 High · effort S
 - *Today:* /ai/preflight (nuxt/app/pages/ai/preflight.vue) renders a fully polished drag-and-drop UI with an active 'Check File' button, but every check returns the raw 503 'Preflight checking is not enabled on this server' because PDF_SERVICE_FEAT_PREFLIGHT is absent from .env and defaults to false. A shop owner who finds this tool, drags in a file, and hits that developer error will conclude the whole product is broken - there is no explanation, no 'not available' label, and no link to fix it.
 - *Change:* On page load, call a lightweight capability probe (e.g. reuse pdf-service:status, or have the controller return a 'preflight available: false' health flag) BEFORE showing the drop zone. When unavailable, replace the upload card with the project's standard amber 'integration not configured' panel: heading 'Print File Checker isn't switched on yet', plain text 'This tool needs your print-file service turned on by your administrator before it can check files.', and a link to the PDF service settings/status page. Hide or disable the 'Check File' button entirely (with a tooltip stating why) rather than letting it fire and 503. When available, show the existing UI unchanged. Never surface the raw 503 string to the user.
 - *Why it helps the owner:* A store owner who opens the Print File Checker today uploads a file and gets a cryptic 'not enabled on this server' error, leaving them thinking the feature - and possibly the platform - is broken. Instead they would immediately see a clear note that the tool just needs switching on, plus exactly where to go, so they never waste time or lose trust.
- **Show a print-readiness badge on customer-uploaded files in the order detail** - 🔴 High · effort M

- *Today:* In `nuxt/app/pages/orders/[id].vue` (lines ~1037-1047) the 'Customer Files' section renders each uploaded artwork file as a bare download link labeled by filename or 'Artwork file', with zero quality information. Files up to 200MB are accepted at upload with no DPI/color-mode/size check (StorefrontArtworkUploadController is Laravel-only, no processing). The owner only discovers a 72 DPI RGB or wrong-size file after downloading it and opening it in their print software - at which point the order may already be in production.
- *Change:* When the preflight/pdf-service is available, run a check on each uploaded file (asynchronously at upload time, or on-demand from the order page) and render a small status badge next to each file row in the Customer Files list: green 'Print-ready', amber 'Check before printing - low resolution / RGB', red 'Not print-ready - too low resolution'. Make the badge click-through to the full preflight report. When the service is off, show a neutral 'Quality not checked' chip instead of nothing, so the absence is explicit rather than silently implying the file is fine.
- *Why it helps the owner:* A shop owner processing an order today has to download every customer file and open it in Illustrator/Acrobat just to find out one is too low-res to print - often after they've already started the job. Instead they would glance at the order screen, see a red 'Not print-ready' badge, and email the customer for a better file before any plates or ink are wasted.
- **Label the designer 'Download PDF' output by quality and warn when it is web-grade** -  High · effort M
 - *Today:* When `PDF_SERVICE_FEAT_PDF_EXPORT` is false (its production default), the designer's 'Export' / 'Download PDF' button silently routes through `legacyExportFile()` and produces a 72-96 DPI RGB `dmpdf` file. Nothing in the UI distinguishes this from the 300 DPI CMYK Node.js output. A customer or owner downloads it, sends it to a printer, and gets a blurry web-quality result with no warning it was unsuitable.
 - *Change:* Have `exportFile()` return which path produced the file (legacy web-grade vs print-grade) and reflect it in the download UI. For print-grade output, label the button/result 'Download Print-Ready PDF (300 DPI)'. For the legacy path, either relabel to 'Download Preview PDF (web quality - not for printing)' and surface a one-line note 'This file is for on-screen preview only. For a print-ready file, ask the store to enable high-quality export.', or, preferably, point the customer-facing 'Download PDF' action at the existing high-quality `previewPdf` Node.js endpoint so customers always get print-grade output. Never hand over a web-grade PDF that visually claims to be the deliverable.
 - *Why it helps the owner:* A store owner who tells a customer 'just download your design as a PDF and we'll print it' is today handing out blurry 72 DPI files that look fine on screen and fail on press, generating reprints and complaints they can't explain. Instead the file is clearly labeled by quality - or is genuinely print-ready - so what the customer downloads is what the press can actually use.
- **Turn the designer preview 503 into a recoverable, explained state** -  Med · effort S
 - *Today:* The `previewPdf` endpoint (behind 'Update Preview' / 'Download PDF') always calls the Node.js pdf-service with no fallback, and on outage returns HTTP 503 'PDF service is currently unavailable. Try again in a moment.' shown as a bare toast. The user gets no retry control, no indication whether it's temporary, and the preview area is left in whatever state it was - reading as 'broken' for a non-technical owner.

- *Change:* In the designer, catch the 503 specifically and render an inline recovery state in the preview slot (not just a toast): 'We couldn't build your print preview right now. This is usually temporary.' plus a visible 'Try again' button that re-fires the request, and an auto-retry with a short countdown ('Retrying in 5s...'). Reserve the preview space so the layout doesn't jump. Distinguish a transient outage (offer retry) from a hard misconfiguration (after N failures, escalate to 'If this keeps happening, the store's print service may need attention' with a settings link for admins).
- *Why it helps the owner:* When the print service hiccups, a shop owner clicking 'Update Preview' today just sees a red error flash and a frozen panel, with no idea if they did something wrong or whether to keep trying. Instead they'd see a calm 'temporary, retrying...' message with a Try Again button, so a momentary blip doesn't read as the designer being broken.
- **Let store owners customise invoice/quote/proof PDFs without touching code** - ● Med · effort L
 - *Today:* Invoice, quote, order, and proof PDFs are rendered by dompdf from hard-coded Blade templates (InvoiceService::generateInvoicePdf, ProofService, OrderService). Logo size, accent color, and footer text (terms, return policy, contact line) are fixed in code, so a non-technical owner has no way to brand or adjust the documents their customers receive.
 - *Change:* Expose a small 'Document Appearance' section in store settings that feeds the existing Blade templates: upload/scale logo, pick an accent color, and edit footer/terms text, with a live 'Preview Invoice' button that renders a sample through the same dompdf path. Wire these as template variables into the existing invoice/quote/proof Blade views - no new PDF engine, just data-driven values with sensible defaults pre-filled from existing store info so it works out of the box.
 - *Why it helps the owner:* A shop owner who wants their invoices and proofs to carry their logo and 'Payment due in 14 days' footer today simply can't - the documents are locked in code and they'd have to file a support request. Instead they could upload their logo, set their accent color, and type their terms once, and every customer-facing PDF reflects their brand.
- **Confirm imposition render failures surface a plain-language status in Job Watch** - ● Low · effort S
 - *Today:* The imposition flow returns asynchronously via a Node.js callback; on render failure the ImpositionController logs the error and POSTs a JSON body, but it is unconfirmed whether the /imposition Job Watch UI (which polls every 15s) shows the owner a readable failure. If it leaves a stuck/blank status, a production-staff user waits indefinitely on a press sheet that will never arrive.
 - *Change:* Ensure the callback's error case maps to a distinct 'failed' status that Job Watch renders as a clear red row: 'Imposition failed to render - please try again or check the source files.' with a 'Retry' action, instead of leaving the job spinning or blank. Stop the 15s polling once a terminal failed/done status is received so the row doesn't appear to hang forever.
 - *Why it helps the owner:* A production user who kicks off a gang-run imposition and walks away today may come back to a job that silently never finished, with no way to tell if it's still working or dead. Instead they'd see a clear 'failed - retry' status the moment it breaks, so they can re-run it rather than waiting on output that's never coming.

Dashboard & Reporting

- **Make the three Reports buttons actually do something - or remove them** - ● High · effort S
 - *Today:* In `nuxt/app/components/dashboard/ReportsWidget.vue` the 'Sales Report', 'Customer Report', and 'Inventory Report' buttons spin for a fake 1-second `setTimeout`, then render as clickable rows with a > chevron, empty icon strings, no `@click`, no `to=`, and no route. They go nowhere. A non-technical owner sees three official-looking report buttons, clicks one expecting a report, and nothing happens - no page, no message, no 'coming soon'. This reads as a broken product on the very first screen after login.
 - *Change:* Wire each button to the data the dashboard already aggregates instead of inventing new report pages. 'Sales Report' → `nuxt-link` to a pre-filtered `/invoices?status=paid` view (or `/orders`) which already exists; 'Customer Report' → `/web-leads` or the customers list; 'Inventory Report' → the products/stock list. Replace the fake `setTimeout` with real navigation. Fill the empty icon fields with real heroicons (`i-heroicons-banknotes`, `i-heroicons-users`, `i-heroicons-cube`). If a real destination genuinely does not exist for one of them, do not render that button at all (gate it) rather than shipping a dead row.
 - *Why it helps the owner:* A store owner who wants to see 'how did sales go this month' today clicks Sales Report and is met with silence, concluding the feature - and possibly the product - is broken. Instead, one click takes them straight to their paid invoices, the exact data they were after, turning three dead buttons into three real shortcuts.
- **Give every metric widget a real error and empty state instead of an eternal spinner** - ● High · effort M
 - *Today:* `InvoiceWidget`, `QuoteWidget`, `MetricTabs`, `PaperTrailWidget`, and `WebLeadsWidget` all handle a failed API call with only `console.error()`. On failure `InvoiceWidget.vue` sets `loading=false` but `progressItems` stays empty, so the owner sees an empty table with column headers and no rows; `MetricTabs` shows zeros. There is no message, no retry, and no signal that something went wrong - a blank widget is indistinguishable from 'you have no data'.
 - *Change:* Add an explicit error ref to each widget's catch block. When the call fails, render a plain-language inline message with a Retry button, e.g. 'We couldn't load your invoice summary. Check your connection and try again.' with a button that re-runs the fetch. When the call succeeds but returns zero rows, show a true empty state ('No invoices yet - they'll appear here once you create your first one.'), not an empty table. Distinguish loading vs error vs empty as three separate states per the [CLAUDE.md](#) mandate.
 - *Why it helps the owner:* A store owner glancing at the dashboard to check 'are any invoices unpaid' currently can't tell whether an empty Invoices widget means everything is settled or the data failed to load - a dangerous ambiguity for money. Instead they get either a clear 'all clear, nothing yet' or a 'this failed, tap to retry', so they always know which is true.
- **Fix the Top Products chart to show the store's real currency, not a hardcoded dollar sign** - ● High · effort S
 - *Today:* `TopProductsBarChart.vue` hardcodes the Y-axis title to 'Revenue (\$)' and formats every value as '\$' + value (lines 132 and 135), and the tooltip prepends '\$' too. A shop owner configured in GBP

or EUR sees pounds of revenue labelled with dollar signs, while the revenue KPI card directly above it (DashboardOrderRevenueWidget) correctly uses CurrencyHelper::settingsForStore(). The same screen shows two different currencies for the same money.

- *Change:* Pass the store currency symbol into the chart the same way the revenue widget gets it (CurrencyHelper / store settings), and replace the three hardcoded '\$' occurrences - Y-axis title text, the yaxis labels formatter, and the custom tooltip - with that symbol. Also reword the Y-axis title from the technical 'Revenue (\$)' to plainly 'Revenue (£)' etc., matching the card above.
 - *Why it helps the owner:* A UK or EU shop owner reviewing their best-selling products today sees dollar signs on a chart for money they bank in pounds, making them distrust every number on the dashboard. Instead the chart speaks their currency, matching the card right above it, so the whole financial picture reads as one consistent, trustworthy view.
- **Rename 'Overview' and either make the tab selection do something or drop it** - ● Med · effort M
 - *Today:* MetricTabs.vue titles the panel 'Overview' (line 8), which tells a non-technical owner nothing. Worse, clicking the Design Proofs / Website leads / Quotes tabs only flips a local selectedMetric ref (lines 25/33/45) that drives nothing visible - the LazyDashboardTopProductsBarChart below never reacts to the selection. The tabs look interactive and selectable but selecting one changes nothing, so the control feels broken.
 - *Change:* Rename the header from 'Overview' to something a shop owner understands at a glance, e.g. 'Where your work is coming from' or simply 'New This Week'. Then resolve the dead selection one of two ways: either make selecting a tab actually re-scope the chart/content below it, or remove the selected-state styling entirely so the three cards read as plain navigational stat tiles (which is what they functionally are - each links to its list page). Do not leave a toggle that visibly highlights but changes nothing.
 - *Why it helps the owner:* A store owner clicking 'Quotes' at the top of their dashboard expecting the view to focus on quotes today gets a highlighted tab and zero change, making them think the screen is frozen. Instead the header tells them what they're looking at in plain words, and the cards behave honestly - either filtering the view or simply jumping to the quotes list.
 - **Clarify the 'Design Proofs' card so zero doesn't read as 'no proofs'** - ● Med · effort S
 - *Today:* The Design Proofs stat card in MetricTabs counts only unlinked Proof records (proofable_type/proofable_id IS NULL), but the label just says 'Design Proofs' with no qualifier. An owner who has many proofs attached to jobs but none unlinked sees '0' and reasonably concludes they have no design proofs at all - the number means something narrower than the label promises.
 - *Change:* Either change the label to describe what is actually counted (e.g. 'Unassigned Proofs' or 'Proofs awaiting a job') with a small tooltip 'Design proofs not yet linked to an order or job', or change the underlying count to match the broad label and total all proofs. Pick whichever matches what an owner most needs to act on; if the intent is 'proofs that need attention', the 'Unassigned Proofs' label plus tooltip is the honest fix.
 - *Why it helps the owner:* A shop owner checking whether any artwork still needs sorting sees '0 Design Proofs' and assumes nothing is outstanding, when in fact several unassigned proofs may be

sitting unhandled. A precise label plus tooltip means the number finally matches what they think it means, so nothing slips through.

- **Tell mobile users that dashboard customisation exists (on a bigger screen)** - ● Low · effort S

- *Today:* Drag-to-rearrange is fully disabled below 768px with the drag handles simply hidden and no explanation. A shop owner running their business from a phone has no idea the board is customisable at all, and an owner who customised it on desktop may be confused why it won't budge on mobile.
- *Change:* On mobile, show a one-line, dismissible hint near the dashboard heading - e.g. 'Tip: open your dashboard on a computer to rearrange these panels.' Keep it subtle and store the dismissal in localStorage like the setup checklist already does, so it appears once and goes away.
- *Why it helps the owner:* A store owner who lives on their phone never discovers they can tailor their dashboard, or fights a board that won't drag, with no clue why. A single plain-language tip tells them the feature exists and where to use it, instead of leaving the capability invisible.

Make What We Have Better - Feature Improvement R&D

Source: readme/research/feature-rd/FEATURE_IMPROVEMENT_RD_2026-06-01.md

IN SHORT — KEY TAKEAWAYS

This document is a health check of 13 features already in the product. A team of research agents read the real code, found where things go quietly wrong for store owners, and ranked what to fix first. It matters because several features are currently lying to owners — the screen says “Saved” or “Success” while the real work silently did not happen.

The main parts explained simply:

- **The biggest problem: success messages that lie** — In several places, an owner does something (saves a price, creates a product, sends an email), sees a green “Success” message, and walks away — but nothing actually happened behind the scenes. The owner can’t tell, has no error to look at, and often only notices later when money is missing or a customer complains. This is called “confirmed-then-discarded” and it is the top priority to fix.
- **Two verified bugs (checked line by line in the code)** — (1) The “Markup” profit field on a product looks like it saves — the green preview box even updates — but the value is silently thrown away every time. The product keeps selling at cost. (2) The AI Product Builder says “Product created successfully” with a tick and a confidence score, but the product is never attached to the owner’s shop, so it never appears in the store. Re-running the prompt piles up invisible duplicates.
- **Six patterns that repeat across the whole product** — The same kinds of mistakes appear in many different features: success messages that are not true; blank screens that look broken when they are just empty; developer-only words showing up where plain English belongs; destructive actions (like sending an email blast or cancelling a job) with no warning about what will be lost; third-party connections (payment gateways, email providers) that accept wrong credentials without any test; and two separate tools doing the same job when one would be simpler.
- **A four-wave fix plan** — Wave 1 is the fastest: fix the phantom Markup field, make “Mark as Paid” actually send the receipt, fix the AI Builder product assignment, and add a recipient-count confirmation before bulk emails go out. Wave 2 builds small shared helpers (plain labels, empty/error states, confirm modals, integration test buttons) that improve every screen at once. Wave 3 tackles bigger work: real onboarding, a working pricing tester, and a friendly email editor. Wave 4 removes duplicate tools.
- **Per-feature findings** — Every one of the 13 modules (Products, Pricing, Orders, Designer, B2B, Storefront Builder, AI Builder, Action Center, Integrations, Payment Gateways, Onboarding, Analytics, Email/SMS) has a single sharpest pain point identified and a ranked list of fixes ready to pick up.

What to do with this: Start with Wave 1 — five small fixes that stop the product from lying to owners. Then build the four shared building blocks in Wave 2, which repair dozens of screens in one go. Treat “the success message must be true” as a rule that every new feature must pass before it ships.

Date: 2026-06-01 **Goal:** Not new features - make our **existing** features easier, clearer, and more useful for our **current** non-technical store owners. **Method:** Multi-agent R&D council. For each of 13 core features: an audit agent **read the real code** (file + line), then a two-person council (a non-technical-owner advocate + a UX/support-load reducer) debated the friction, then a chair found the cross-cutting patterns and ranked fixes. 40 agents, ~2.1M tokens, grounded in this codebase. **Verification:** The two headline bugs below were hand-verified against the actual source before publishing this doc. Treat every other `file:line` citation as a strong lead to confirm before fixing - but the audit read real code, not guesses.

The one finding that matters most

Our biggest problem isn't missing features - it's that several features quietly LIE to the owner. The screen says "Saved" / "Success" / "Active" while the real work silently didn't happen. For a non-technical shopkeeper this is the worst possible failure: positive feedback followed by invisible data loss, with no error to investigate and no way to self-diagnose. Every one of these is a guaranteed support ticket - and the kind that destroys trust.

We call this pattern "**confirmed-then-discarded**" and it shows up across the whole product. Fixing these comes before everything else.

✓ Two confirmed-in-code examples (verified by hand)

1. The phantom Markup field (Products) - worst bug in the platform The owner types a profit markup, watches the big green "Effective Price" box update to confirm it, clicks Save, sees "Pricing saved successfully" - **and the value is silently thrown away.** The product keeps selling at base price (i.e. at cost).

- `PricingHub.vue:47` binds `form.markup` ; line 239 computes `base * (1 + markup/100)` for the green box.
- `AdminProductTabs.vue:625` sends `markup` in the save payload.
- But `StoreProductRequest.php:39` and `UpdateProductRequest.php:40` only accept `profit_margin` - `markup` is never validated, so Laravel **drops it before it touches the database.**
- The owner cannot diagnose this. They just see lower revenue and never connect it to a field that "looked saved."
- **Fix:** wire the UI field to the real `profit_margin` end-to-end (create + update + savePricing payloads, and seed it back on load), OR delete the field and the green box entirely. Never ship the in-between state.

2. AI Product Builder creates products that vanish (verified) "Product created successfully" with a green check and a confidence score - but `AiProductCreatorService.php:47` creates the product as `ACTIVE_STATUS` and **never assigns it to the owner's store** (no `syncProductStores` / `stores()` -

`>attach / store_id`). So it's "live" but attached to nothing, and never appears in the shop. Owners assume the tool is broken and re-run the prompt, piling up orphan duplicates.

- **Fix:** auto-assign the new product to the current store, and create it as **Draft** ("Saved as a draft - review the price and add a photo, then publish") so an AI-guessed price/photo never goes live by accident.

The 6 recurring patterns (fix once, help everywhere)

These show up across many features. Each one, built as a **shared building block**, repairs dozens of screens at once - the highest leverage per hour of work.

Pattern 1 - Confirmed-then-discarded (the success message must be TRUE)

Where: Products (Markup), Orders (Mark-as-Paid never fires the receipt), Payment Gateways/Integrations ("Active" on a half-configured gateway that never collects money), AI Builder ("created" but unassigned), Email Campaigns (0-recipient send marked "sent"), Storefront Builder ("Save as Template" succeeds but can never be re-inserted; "Set as Homepage" does nothing). **Fix:** Make every on-screen confirmation tell the truth. Wire fields to real saved values, route status changes through the lifecycle methods that fire automations, block "Active" until a real Test Connection passes, auto-assign AI products, never report a send/save as successful when nothing went out. **Make "the success message must be true" a hard review gate.**

Pattern 2 - Missing error/empty states that read as "all clear" or "broken"

Where: Action Center (a failed dashboard widget shows the green "all caught up" check), Job Kanban (failed load = empty board = "my jobs vanished"), Storefront pages list (blank table looks identical to a network error), Analytics, most data views. **Fix:** Apply the house three-state rule - **loading skeleton / helpful empty / plain-language error + Retry** - to every data view. Build **one shared empty-state + error-state component pair** and drop it into each list/board/widget. Never render the success/empty state on a failed fetch.

Pattern 3 - Raw technical output leaking into owner-facing labels

Where: Designer layer list (`textbox` , `svgimage` , `V1StGXR8_Z`), Product cards (a badge literally reads `none / fixed`), Orders (priority shown as a bare `9`), Payment Gateways (`authorizenet` under the brand name), B2B catalog (product UUID instead of name), AI Builder (raw `ecommerce / quantity_tier`), Storefront editor (`yourstore.com` , slug chips, raw `rem` values). **Fix:** Build small shared label helpers - `getLayerLabel()` , `humanizePriority()` , a product/pricing-type map, a status-color util - and use them everywhere the raw value shows today. Near-zero risk, platform-wide payoff, and it enforces the existing house standard.

Pattern 4 - Destructive actions with no stated consequences and no undo

Where: Email Campaigns (one click blasts the whole list, no count, no confirm in the authoring form), Job Kanban (a one-pixel mis-drag into "Cancelled" silently kills a live job), Action Center ("Dismiss" permanently hides a still-overdue invoice with no undo), list deletes (bare browser confirm). **Fix:**

Standardize on the consequence-stating confirm modal with a restated action button (“Send to 1,240 people”, “Cancel Job”, “Delete Page”), show the real number/name affected, and add one-tap **Undo** where reversible.

Pattern 5 - Integrations that fail silently and can't be self-verified

Where: Integrations & Payment Gateways (no per-field help, no Test Connection, webhook URL surfaced nowhere, can activate while incomplete), B2B (gated by a `.env` flag the owner can't touch), Action Center & Email (silently inert when no gateway/provider is configured). **Fix:** Three shared moves - (a) seed plain-language help text + real placeholders + a “where to find this” link for every credential field; (b) add a real **Test Connection** per driver and block “Active” until it passes; © surface the **webhook URL** with a Copy button. Reuse the house “unconfigured integration → explain + link to setup” pattern everywhere.

Pattern 6 - Two divergent UIs / two parallel systems for the same task

Where: Pricing rules (a weaker modal editor vs the full-page editor), Email/SMS (two unrelated campaign systems), Onboarding (two register routes + a legacy fake-numbers dashboard), Products pricing (base price + markup duplicated across General and the Pricing hub). **Fix:** Pick one canonical implementation per task, delete/redirect the other, and keep single-source-of-truth data (one home for base price + margin). Fewer surfaces means future fixes can't miss a copy.

The roadmap - 4 waves, easiest-and-most-damaging first

Wave 1 - Stop the silent lies (days, not weeks)

Small fixes that each kill a whole category of undiagnosable ticket. Ship in this order:

1. **Products** - fix or remove the phantom Markup field.
2. **Orders** - make “Mark as Paid” actually fire the receipt automation (route through `markAsPaid()` / `finalize()` / `markAsVoid()` + `STATUS_CHANGED`, not a raw status write).
3. **AI Builder** - auto-assign the product to the store + save as Draft.
4. **Email Campaigns** - Send-Now confirm with a real recipient count; refuse to send to a 0-person audience.
5. **Action Center** - give the dashboard widget a real error state so a failed load stops showing the green “all caught up.”

Together these restore the basic promise: *when the screen says it worked, it worked.*

Wave 2 - Ship the shared building blocks (1–2 weeks, highest leverage)

Build four small reusable pieces, apply platform-wide:

- (a) Plain-language label/status helpers (layer names, priority words, product/pricing-type maps, status colors).

- **(b)** Standard empty-state + error-state-with-Retry pair in every list, board, and widget.
- © Consequence-stating confirm modal + Undo for destructive actions.
- **(d)** Integration help-text seed + Test Connection + visible webhook URL + “can’t activate while incomplete” guard.

Wave 3 - The big experience investments (multi-week)

Now that trust is restored and the product is de-jargoned:

- **Onboarding** - auto-create the owner’s first store at signup (fire the storefront-seeding pipeline), default to auto-approved trial, show a focused first-run setup screen instead of the empty 10-widget BI dashboard. *(Root cause of the worst first impression: today no store exists after signup and nothing tells the owner to create one.)*
- **Pricing** - replace the **mock** `PricingCalculator` (it fakes a result, never calls the API) with the real `calculate-pricing` endpoint, embedded in the rule editor, previewing the **unsaved draft rule live**. *(Today the module tells owners “test your rule before activating” but sends them to a stub that always says “no rules applied” - so they save live rules blind.)*
- **B2B** - replace the developer-only `B2B_MODULE_ENABLED` `.env` gate with an owner-facing setting + a guided explainer. *(An owner who paid to sell to businesses sees no menu and no explanation.)*
- **Email** - a friendly rich-text/template editor + “Send a test to myself” instead of a raw HTML textarea and a product-ID textarea.
- **Integrations/Payments** - a real per-driver Test Connection that blocks going Active until it passes; persistent “Test mode” banner.

Wave 4 - De-duplicate and document (ongoing cleanup)

Collapse the two pricing-rule editors into one, unify the two campaign systems, remove the duplicate register route and legacy fake-numbers dashboard, bring size/option editing inline, refresh stale designer + AI docs.

Guiding rule throughout: treat “the success message must be true” and “every data view needs loading + empty + error states” as non-negotiable review gates, so these patterns stop re-entering the product as new features ship.

Per-feature highlights

Each feature’s single sharpest friction + its top “do now” fixes (impact /10).

Products

- **Sharpest:** the phantom Markup field (verified above).
- Replace silently greyed-out tabs on a new product with a clear “save the basics first to unlock” step (*imp 9*); block Create until SKU + Subcategory are filled with inline errors (*imp 8*); humanize pricing/type

badges - never render `none` / `fixed` (*imp 7*); one “How this product is priced” summary at the top of the Pricing tab, de-duplicating base price + margin into one home (*imp 8*).

Pricing & Price Calculator

- **Sharpest:** the module tells owners to “test your rule before activating,” but the calculator they’re sent to is a **mock that always says ‘no rules applied’** - so they save live rules blind.
- Replace the fake calculator with the real one (*imp 10*); preview the **unsaved** rule, not just saved ones (*imp 9*); make Formula authorable with insert-variable chips + a “Check formula” button (*imp 8*); hide Priority/Replacement under “Advanced” with sensible defaults (*imp 7*).

Orders & Print Jobs

- **Sharpest:** “Mark as Paid” never fires the receipt/notification automations the owner set up (raw status write bypasses `markAsPaid()`).
- Make Mark-as-Paid fire the receipt (*imp 10*); confirm before a drag cancels a real job (*imp 9*); color order badges by real status so Paid/Void/Refunded are distinguishable (*imp 8*); show job priority as a word (Rush/High/Normal/Low), not a bare number (*imp 7*).

Designer Studio

- **Sharpest:** the layer list speaks developer - `textbox` / `svgimage` / `V1StGXR8_Z` - so nobody can tell which row is their logo.
- One `getLayerLabel()` so every row shows a human name (*imp 9*); ensure **buyers** get readable names too (*imp 8*); rename the “Material” tool tab to “Shapes” (*imp 6*); on mobile, “Preview” should show the design, not download PNGs (*imp 7*).

B2B / Corporate Accounts

- **Sharpest:** the only enable switch is a `.env` flag - an owner who bought B2B sees no menu and no explanation.
- Let owners turn B2B on from Settings (*imp 10*); put real numbers + a next step in the over-credit-limit checkout error (*imp 8*); show real product names (not UUIDs) in the restricted-catalog list (*imp 7*); link the “no business store” warning straight to the store-type setting (*imp 6*).

Storefront Builder

- **Sharpest:** owners can’t do the one thing they came for - “make THIS my homepage” - and “Save as Template” silently does nothing (the saved template can never be re-inserted).
- Stop promising “Save as Template” until it round-trips (*imp 9*); add a plain “Set as Homepage” button + badge (*imp 9*); add empty + error states to the pages list (*imp 7*); let owners upload images instead of pasting a URL (*imp 7*).

AI Product Builder

- **Sharpest:** “Product created” but it’s never assigned to a store, so it vanishes (verified above).

- Auto-assign to the current store (*imp 10*); create as Draft and say so (*imp 9*); fix the success wording + the empty Designer trap (*imp 7*); guard against duplicate orphan products on re-submit (*imp 6*).

Action Center

- **Sharpest:** the trust feature has two silent ways of saying “nothing’s wrong” when something is - a failed widget load shows green “all caught up,” and the severity tabs filter only the current page while the count is whole-feed.
- Real error state on the widget (*imp 9*); make Critical/Warning/Info tabs filter the whole feed server-side and never falsely say “all caught up” (*imp 9*); confirm permanent Dismiss + one-tap Undo (*imp 8*).

Integrations

- **Sharpest:** paste a wrong credential, see “Saved,” flip to Active, walk away - nothing tests it, no help text, failure is invisible until checkout.
- Fill in help text + placeholders + “where do I find this?” per field (*imp 9*); stop an unconfigured integration from switching On (*imp 9*); add a Test Connection with plain pass/fail (*imp 10*); show the webhook URL with a Copy button (*imp 8*).

Payment Gateways

- **Sharpest:** the webhook URL is shown nowhere and the Webhook Secret is marked optional, so customers pay real money while orders sit “pending” forever.
- Show the webhook URL + Copy button + one instruction per card (*imp 10*); point the onboarding checklist at the dedicated gateway page (*imp 8*); make the gateway un-saveable into a broken state (webhook secret required) (*imp 9*); make “Test mode” impossible to miss (amber banner + checkout badge) (*imp 8*).

Onboarding

- **Sharpest:** after signup no store exists and nothing tells the owner to create one - they land in an empty BI dashboard with a checklist that can never complete.
- Auto-create the first store at signup (*imp 10*); default new signups to auto-approved trial (*imp 9*); focused first-run screen, not the empty command-centre (*imp 9*); can’t-skip “Set up your store” step (*imp 8*).

Analytics Dashboard

- **Sharpest:** the numbers lie in the owner’s own terms - INR/GBP stamped with , one store’s revenue next to all-stores’ customer counts, “avg per paid order” that includes drafts.
- Show every figure in the store’s own currency (*imp 9*); scope Customer cards to the store being viewed (*imp 8*); make “Avg Order Value / per paid order” actually true (*imp 7*); honest “vs last month” at the zero baseline (*imp 6*).

Email & SMS Campaigns

- **Sharpest:** one click can email the entire customer base with no confirmation and no recipient count - the exact “sent to everyone by mistake” disaster, and irreversible.
- Last-chance confirm stating the real recipient count (*imp 10*); always show recipient count + refuse empty audiences (*imp 9*); product picker by name instead of a raw product-ID textarea (*imp 8*); “Send a test to myself” that matches the real inbox email (*imp 8*).

Generated by a code-grounded multi-agent council. The two headline bugs (phantom Markup, AI-product-not-assigned) were verified by hand against the source. Other `file:line` citations are strong leads - confirm before fixing. Per the house testing rule, each bug fix needs a test that reproduces the bug before the fix passes.

Make What We Have Better - Feature Improvement R&D, Round 2 (Wave-5)

Source: readme/research/feature-rd/FEATURE_IMPROVEMENT_RD_2026-06-02.md

IN SHORT — KEY TAKEAWAYS

This document is a deep check on 13 features the platform already has. A team of 40 automated reviewers read the real code to find places where the screen says “Saved” or “Done” but nothing actually happened. It matters because store owners make real business decisions based on what the screen tells them — and right now, several screens are quietly lying.

The main parts explained simply:

- **The biggest problem found** — Across almost every feature, the screen shows a success message even when the action failed, the data was never saved, or the change had serious side-effects (like emailing a customer) with no warning first. This is called a “silent lie.”
- **Fixes that were thought done but weren’t** — A list of things the team believed were already fixed. Most were only half-done or never actually wired up. For example: a pricing calculator that exists in code but is shown nowhere, a “new signup auto-approved” setting that was never saved to the database, and an onboarding checklist that marks steps complete using fake demo data.
- **Patterns that keep repeating** — Nine recurring problems found across all 13 features, such as: success toasts that fire even on failure, delete buttons with no “are you sure?” warning, raw technical words (like `void` , `textbox` , `service_key`) shown to store owners, and error screens that were coded but never connected so failures look like empty pages.
- **Tier 0 — ship today fixes** — Ten small changes (a few hours each) that stop the most damaging lies: the Kanban board not rolling back a failed status change, “Save & Back” sending you away even when saving failed, analytics showing all-zeros instead of an error message, and the AI Builder opening the wrong page because it passes an internal number instead of the correct store ID.
- **Tier 1–3 — upcoming fixes** — Bigger improvements: making severity filters in the Action Center actually work, enforcing that two pages can’t share the same web address, letting owners test that their payment gateway really works before going live, and replacing raw database IDs in email campaigns with human-readable product names.

What to do with this: Start with the Tier 0 list — these are hours of work that stop owners from trusting broken confirmations. Then tackle the “re-do list” of fixes that were marked done but aren’t, since those are the most dangerous because no one is watching them.

Date: 2026-06-02 **Goal:** Same as Round 1 - not new features, make our **existing** features easier, clearer, and more useful for **current** non-technical store owners. **What’s different this round:** Round 1 (`FEATURE_IMPROVEMENT_RD_2026-06-01.md`) found a class of “confirmed-then-discarded” lies and waves 1–4 shipped fixes. This round **first verified what those fixes actually did** (many were “strong leads,” not confirmed), **then** went deeper to find what’s still hard. **Method:** Multi-agent council, 40 agents, ~2.1M tokens, grounded in real code. Per feature: an agent ① read the current code + the

relevant wave commit to mark each prior claim *fixed* / *partial* / *open* with `file:line` evidence, ② did a fresh deep usability audit of what's still painful, ③ a two-person council (owner-advocate + support-load-reducer) debated and ranked fixes. A chair then found the cross-cutting patterns. **Trust note:** every `file:line` below came from an agent reading real code, but confirm before you fix - and per the house rule, every bug fix needs a test that reproduces the bug first.

The one finding that matters most

Waves 1–4 fixed the *visible* lies and left the *write-path* lies intact. Across all 13 features the sharpest remaining friction is some variant of: the UI says “Saved,” “Sent,” “Activated,” “Configured,” or “You’re all caught up,” while the underlying action silently failed, reached nobody, or committed a destructive change the owner never confirmed. We added scaffolding (badges, status fields, empty states) but in several places the actual save, mount, migration, or guard was never wired.

The two most damaging concrete instances - both cheap to fix:

- **Kanban silent-state lie** - a Kanban drop commits a status change (which can **email the customer** and notify the team) with no confirm; on success it only `console.log('success')` (`KanbanBoard.vue:579`), and on a `200` -with- `ok:false` it leaves the card moved on screen while the DB never changed (`KanbanBoard.vue:581`), no rollback.
- **Products “Save & Back” lie** - `saveGeneralAndBack` (`AdminProductTabs.vue:613-619`) navigates to the list whenever `!isSubmitting` , but `isSubmitting` is reset in `finally` *before* the check, so the guard is **provably always false** - the owner is bounced to the list believing a failed (e.g. duplicate-SKU) save succeeded.

Prior “fixes” that were incomplete or false (re-do list)

These were believed done after waves 1–4 - the highest trust-risk items because nobody is watching them anymore. Re-verify each rather than trusting the commit message.

Prior claim	Reality (verified this round)	What's actually needed
Pricing calculator “mock” fixed	The component was rewired to the real engine but is dead code, mounted nowhere - while <code>guide.vue:735</code> still tells owners to use it.	Mount it on the pricing-rules screen.
New signups auto-approved trial	False by default. <code>auto_approve_signups</code> column was never migrated → resolves null → false; every signup lands in pending-approval.	Migration + cast + default-on.
Onboarding “checklist that can never complete” fixed	Re-broken in the default no-store state: <code>resolveStore()→null</code> disables 4 of 5 steps, reviving the dead-end.	Add a gated “Create your store” step 0.
Analytics currency fixed (a13bb391)	Partial - only the main dashboard widget; the dedicated Store Analytics page is untouched , still en-US/USD everywhere.	Currency sweep on the analytics page + pricing API + AI builder.
Order badges colored by real status	Partial - tenant override added, but the fallback <code>getStatusColor()</code> has no paid/void/refunded case → all gray out of the box; job table still prints raw <code>void</code> .	Delegate both <code>getStatusColor</code> s to <code>statusColor.ts</code> ; render <code>statusLabel()</code> .
AI Builder orphan-product fixed	Half - now store-assigned, but the “Open in Designer” button passes the numeric PK where a store UUID is required (<code>AiProductBuilderController.php:64</code>), so the studio silently fails. Also no idempotency on re-submit.	Pass <code>getCurrentStore()</code> - <code>>uuid</code> ; add dedup.
B2B over-credit error has numbers + next step	Partial - the figures live on <code>CreditLimitExceededException</code> but are never interpolated into the message; no recovery step.	Interpolate available/attempted + recovery copy.
Integrations per-field help shipped	Plumbing present, content absent - <code>:help</code> is bound but 0 of ~40 seeded attributes carry a <code>description</code> .	Seed descriptions + “where to find this.”
Save-as-Template (Storefront Builder)	Open - write-only dead end; <code>getBlockTemplates</code> is never imported, nothing re-inserts a saved template.	Close the round-trip.
Designer layer-naming / “Material” tab / mobile Preview	Mostly open - manual rename added, but no auto-derivation from content; rows still show <code>textbox</code> /nanoid; “Material” still mislabeled; mobile Preview still downloads PNGs.	Auto-name from content; rename tab; in-editor Preview modal.
All four Payment Gateway claims	All open - <code>git log --since=2026-06-01</code> touched no gateway file. Webhook URL unshown, secret optional, no activation guard, sandbox mode invisible at checkout.	See Tier 0–2 gateway items.

Cross-cutting patterns that survived waves 1–4

Each appears in many features - which is why one-off patches keep missing them. Fix the *pattern* once.

A. Success/empty states that are actually silent failures. Orders/Jobs (`KanbanBoard.vue:570-590`), Products (`AdminProductTabs.vue:613-619`), Integrations (green “Activated” on blank keys, `index.vue:434-443`), Payment (“saved” ≠ “works”), Email (empty audience → “Sent to 0”, `SendEmailCampaignJob.php:41-48`), Analytics (failed load = all-zero dashboard, error UI unreachable at `index.vue:55-59`), Action Center (“Critical (5)” → green “all caught up”), AI Builder (Designer URL passes PK not UUID).

B. Destructive actions with no confirmation / stated consequence (a direct `CLAUDE.md` §0 violation). Kanban drop-to-Cancelled + one-click Archive (`KanbanBoard.vue:275,408`); Designer layer/image deletes; B2B `removeContact/removeDepartment/removePriceRow` (direct API calls *while account-delete right above them has a full modal*, `business-accounts/[id].vue:697,766,854`); Action Center permanent Dismiss; Storefront Builder list-delete uses native `confirm()` while the editor has a styled modal.

C. Raw identifiers / jargon / untranslated keys leaking to owners. Email product picker is a textarea of raw DB IDs (`Upsert.vue:86-88`); Products header `ecommerce` , Options raw `pricing_type` , “Slug”; Orders raw `void` ; Designer `textbox` /nanoid rows + literal `message.exportError` toast (missing i18n keys); Integrations/Payment raw `service_key` ; Analytics raw enum statuses.

D. The error state is the one always skipped. Teams added loading + empty but swallowed the error path into `console.error` : Jobs (`job/List.vue:611-613`), Storefront pages list (`index.vue:337-339`), Analytics, Designer clipart (`IllustrationTab.vue:104-105`).

E. Confirmed-then-discarded / “looks done but isn’t wired” (the Round-1 class, still recurring). Save-as-Template write-only; `emitUpdate` hard-codes `data_binding: undefined` so any edit wipes a block’s binding (`BlockSettingsEditor.vue:333-341`); the real PricingCalculator is mounted nowhere; quantity-tier rules can’t save (field-name mismatch `form.vue:564` vs `StorePricingRuleRequest.php:36-38`).

F. Currency hardcoded to \$ /USD (the known currency-flow gap). Analytics page (`index.vue:347-351,122,441,472,484`), pricing calculate-price API (`ProductController.php:473,526-528`), AI Builder (`product-builder.vue:204,295`).

G. Two parallel code paths for one concept (they drift and disagree). Pricing inline modal vs full-page editor; Email old `Campaign` vs new `EmailCampaign` ; Orders list `void-Cancelled` vs detail modal raw; Products base-price/margin in General + PricingHub.

H. Guards that don’t gate (incompleteness *shown* but not *enforced*). Integrations `configuration_status` computed but `toggle()/update()` never block activation; Payment `webhook_secret` still `required:false` ; Onboarding `auto_approve_signups` never migrated.

I. Whole-feed vs current-page mismatch (counts lie relative to rows). Action Center tab badges from whole-feed `meta.counts` while rows are one page; B2B restricted-catalog names capped at `per_page:25` → fall back to “Product.”

Wave-5 roadmap - easiest-and-most-damaging first

■ Tier 0 - one-line / hours, stops active lies or page crashes. Ship immediately.

1. **Honor the server result in** `updateJobStatus` + roll back on `ok:false` (*Orders/Jobs, hours*) - `KanbanBoard.vue:570-590` ; `showSuccess` / `showError` instead of `console.log` .
2. **Fix Save & Back silent navigation** (*Products, hours*) - `submit()` returns a success boolean; only `navigateTo(list)` on true.
3. **Null-guard** `job/List.vue:getStatusColor` (*Orders/Jobs, hours*) - `(status||'').toLowerCase()` stops a page-blanking crash (`List.vue:424`).
4. **Stop the empty-audience “Sent to 0”** (*Email, hours*) - status `failed` / `sent_to_none` with a plain reason (`SendEmailCampaignJob.php:41-48`).
5. **Make** `loadSummary` / `loadChart` **surface failures** (*Analytics, hours*) - set `error.value=true` so the existing error UI renders instead of a fake all-zero dashboard.
6. **Scope B2B suspension to ALL payment paths** (*B2B, day - top severity*) - `isActive()` guard on the gateway-paid checkout (`StorefrontCheckoutController.php:459` reads only `company_account_id`).
Add a feature test.
7. **Block activation of incomplete integrations + gateways** (*Integrations + Payment, day*) - completeness check in `IntegrationService::toggle()/update()` ; tie `is_active=true` to `webhook_secret` . One pattern fixes both features' “green Activated, can't work” lie.
8. **AI Designer URL: pass store UUID not PK** (*AI Builder, hours*) - `AiProductBuilderController.php:64` → `getCurrentStore()->uuid` .
9. **Header status badge: “Active”/“Hidden”, not invented “Draft”**; reuse `productTypeLabel()` (*Products, hours*) - `AdminProductTabs.vue:15,534-539` .
10. **Add the three missing Designer i18n keys** (*Designer, hours*) - `exportError` / `saveError` / `previewPdfReady` so toasts stop showing literal `message.*` .

■ Tier 1 - hours-to-a-day, closes a blocked core task or a destructive trap.

11. **Quantity-tier save mismatch** (*Pricing, hours*) - align field names end-to-end + `StorePricingRuleRequest extends BaseRequest` ; the most common print rule can't save today. Add a backend test.
12. **Empty-state “Create Custom Rule” → real navigation** (*Pricing, hours*) - bind to the same `getProductPath(.../form)` ; delete the undefined `addRule` .
13. **Server-side severity filtering** (*Action Center, day*) - send `severity` param; one change retires the false-green, vanishing-pager, and drifting-count frictions at once.
14. **Consequence-stating confirm on destructive actions, batched** (*Orders/Jobs, Designer, B2B, Action Center, Storefront Builder - day each, shared pattern*).
15. **Add the missing error state to lists** (*Orders/Jobs, Storefront Builder, Analytics chart, Designer clipart - day, one shared* `fetchError` + retry pattern).

16. **data_binding: undefined wipe** (*Storefront Builder, hours*) - only emit `data_binding` when it changed (`BlockSettingsEditor.vue:333-341`).
17. **Per-store unique slug** (*Storefront Builder, day*) - validation rule + DB constraint so the second page can't silently shadow the first.

■ Tier 2 - day-to-days, completes a half-built or dead feature.

18. **Mount the real PricingCalculator** (*Pricing, day*) - render it so `guide.vue:735` 's instruction is followable.
19. **Close the Save-as-Template round-trip** (*Storefront Builder, days*) - import `getBlockTemplates` , add a "My Templates" category.
20. **Currency threading sweep** (*Analytics, Pricing API, AI Builder, day total*) - port the `CurrencyHelper::settingsForStore` pattern already on the main dashboard.
21. **Settings toggle for B2B + migrate auto_approve_signups** (*B2B + Onboarding, days*) - both are "feature invisible because a flag/column was never made reachable."
22. **Onboarding "Create your store" gate + stop demo products satisfying the product step** (*Onboarding, days*) - exclude `is_demo` from `checkProductStep` ; non-dismissible store step; empty-state for storeless tenants.
23. **Test Connection / Send-a-test-to-myself** (*Payment, Integrations, Email, days*) - one "verify before you go live" pattern missing in three features.
24. **In-editor Preview modal** (*Designer, days*) - reuse `fabricStore.isPreview` so Preview shows the design instead of downloading a file.

■ Tier 3 - label/jargon polish (cheap, batch alongside the above).

Auto-derive human layer names from content (*Designer*); de-jargon block library + "Custom HTML"/Quill mismatch (*Storefront Builder*); option `pricing_type` + "Slug" labels (*Products*); raw `service_key` /"seeded" copy (*Integrations/Payment*); "Reorder threshold" wording (*Action Center*); raw `void` in order detail modal (*Orders*); AI confidence/reasoning + color swatches (*AI Builder*).

Per-feature - sharpest remaining friction + top fixes

Each feature's single sharpest friction (in plain owner language) and its highest-impact "do now" fixes (impact /10, effort).

Products

Sharpest: Turning a finished product's "Active" switch OFF flips the header badge to **"Draft"** - the word for "never finished" - so it looks like the product reverted. And "Save & Back" can bounce you to the list even when the save *failed* (duplicate SKU), so you believe it saved when it didn't.

- (9, hours) Fix Save & Back silent navigation - only navigate on a real success.
- (8, hours) Header badge "Active"/"Hidden", not an invented "Draft".

- (7, hours) Map `ecommerce` → “Physical Product” (reuse `productTypeLabel()`); give PricingHub a “Save the product first” state instead of a faded dead body.
- (6, day) Bind the 422 field errors the API already returns onto the General-tab inputs.

Pricing & Price Calculator

Sharpest: “I built my bulk-discount rule, hit Create, all it says is ‘*An error occurred*’ - so I can’t save the one rule every print shop needs, and the guide tells me to test it in a calculator that isn’t on screen anywhere.”

- (10, hours) Fix the quantity-tier field-name mismatch end-to-end + structured 422s. Add a backend test.
- (9, hours) Make the empty-state “Create Custom Rule” button actually navigate.
- (8, day) Mount the (now-real) PricingCalculator so the guide’s instruction is followable.
- (7, days) Make the formula field authorable - insert-variable chips + a “Check Formula” button.

Orders & Print Jobs

Sharpest: Dragging a Kanban card to tidy up silently fires a real status change - it can email the customer, notify the team, write to the DB - with no confirm and no “Saved” feedback; and if the save is rejected the card stays moved on screen while nothing changed.

- (9, hours) Honor the server result in `updateJobStatus`, roll back on failure.
- (9, day) Confirm every status-changing drop with stated consequences; stronger warning for terminal states.
- (8, hours) Null-guard `getStatusColor`; route the Kanban Archive button through the void-confirm modal.
- (6, day) Delegate both `getStatusColor` s to `statusColor.ts` + render `statusLabel()` so Paid/Void/Refunded aren’t identical gray.

Designer Studio

Sharpest: At the two highest-stakes moments - saving and previewing before ordering - the tool literally shows “`message.exportError`” gibberish, and “Preview” never shows the design, it just dumps a file.

- (9, hours) Add the three missing i18n keys (`exportError` / `saveError` / `previewPdfReady`).
- (9, days) Make “Preview” show the design in an in-editor modal; keep download as a secondary button.
- (8, day) Confirm before deleting a layer / uploaded image, in plain language.
- (8, days) Auto-derive a human layer name from content, replacing `textbox` /nanoid.

B2B / Corporate Accounts

Sharpest: Suspending a company shows “*Suspended (can browse, cannot order)*” - but they can still place and **pay by card**. The one safety switch an owner most needs to trust quietly doesn’t work.

- (10, day) Add an `isActive()` guard on the gateway-paid checkout so suspension blocks ALL payment methods. Add a feature test.
- (9, days) Settings toggle to enable B2B instead of the env-only flag.
- (8, day) Show the real decline reason on the buyer’s order list, not a bare red “VOID”.

- (7, hours) Interpolate real numbers + a recovery step into the over-credit error; confirm dialogs on remove-contact/department/price.

Storefront Builder

Sharpest: “I made a new page, it said saved, but customers still see my old page.” Two pages can share a slug - the second is written and shows green success but the storefront resolves to the first match, so the new content is permanently invisible with no warning.

- (10, day) Enforce unique slug per store (validation + DB constraint) with a plain-language collision error.
- (9, hours) Add empty AND error states to the pages list.
- (8, days) Close the Save-as-Template round-trip (a “My Templates” category that re-inserts).
- (8, hours) Stop `emitUpdate` wiping `data_binding` on every edit.

AI Product Builder

Sharpest: The “Open in Designer” button hands the store’s internal number instead of its proper ID, so the studio can’t find the store - canvas, uploads, and Save silently fail right after the screen said it worked.

- (9, hours) Pass the store UUID (not PK) into the Designer URL.
- (8, hours) Add a “View in your products (drafts)” deep link on the success screen.
- (8, day) Use the store’s real currency in all three result-card spots, not hardcoded USD.
- (6, days) Server-side idempotency/dedup + a cost-aware “Create another” confirm.

Action Center

Sharpest: Click “Critical (5)” and the screen shows a green “*You’re all caught up*” - because filtering is client-side over one page while the badge counts the whole feed. The one screen meant to show what’s on fire reassures you nothing is.

- (10, day) Make severity filtering server-side so counts, page slice, and empty-state describe the same set.
- (8, days) Confirm + undo on permanent Dismiss.
- (7, hours) Re-sync counts from the server after dismiss/snooze instead of hand-decrementing.
- (5, days) Distinguish “genuinely nothing wrong” from “you can’t see these” (permission-filtered) empty state.

Integrations

Sharpest: An owner can flip Stripe to “Active” with blank keys and get a green “Activated” - believing payments are live - then ship a broken checkout. The “Configured”/100% bar shows confident green with nothing entered.

- (10, day) Block activation of an incomplete integration; disable the toggle with a tooltip.
- (9, day) Fix the lying `configurationStatus` + the fake fixed-width progress bar.
- (9, days) Add a real Test Connection (provider ping) with plain pass/fail.
- (8, days) Seed per-field help + “where do I find this?” for every credential.

Payment Gateways

Sharpest: “I typed my keys, switched it on, it said ‘*saved successfully*’ - but when a real customer pays, the order never shows as paid and nothing warns me.” The screen confirms values were *stored*, never that payments *work*; orders pile up “pending” because the webhook URL was never shown and the secret was optional.

- (9, hours) Show the webhook endpoint URL on each card with a copy button + one instruction.
- (9, day) Block `is_active=true` when `webhook_secret` is blank.
- (9, days) Add a “Verify connection” button hitting the gateway API with saved keys.
- (7, day) Surface a sandbox/“Test mode - no real charge” badge at checkout.

Onboarding

Sharpest: After signup nothing tells me to create a store - yet the checklist already shows “Add your first product” ticked green (from demo data), so I think I’m done. The dashboard is full of empty charts and “Upload Logo” dumps me on a page with nowhere to upload a logo.

- (9, hours) Exclude `is_demo` from `checkProductStep` so the step only greens on a real product.
- (9, days) Add a gated, non-dismissible “Create your store” step 0 + a storeless-tenant empty state.
- (8, hours) Point “Upload Logo” at the real logo control, not the read-only detail page.
- (6, day) Migrate `auto_approve_signups` + default self-serve signups to auto-approved trial.

Analytics Dashboard

Sharpest: When analytics fails to load, the page doesn’t say so - it shows every figure as \$0 and stops spinning, so a shopkeeper with real sales can’t tell “failed to load” from “you sold nothing.” And the customer counts beside the (store-scoped) revenue count the whole account.

- (10, hours) Make `loadSummary` / `loadChart` set `error.value=true` so the real error UI + Retry renders.
- (9, hours) Scope `getCustomerStats()` to the store.
- (8, hours) Fix Avg Order Value to divide paid revenue by a *paid*-order count, matching its label.
- (8, day) Thread tenant currency through the analytics page + chart.

Email & SMS Campaigns

Sharpest: “I wrote my email, hit Send, the screen says it went out - but I can’t know it reached anyone, can’t send a test to myself first, and a campaign whose audience emptied out gets stamped green ‘Sent’ to 0 people.”

- (9, hours) Stop marking an empty-audience send as “sent” - `failed` / `sent_to_none` with a plain reason.
 - (9, day) Add “Send a test to myself”.
 - (8, days) Replace the raw product-ID textarea with a name-based product picker.
 - (8, day) Give failed campaigns a Retry / Edit & resend path.
-

Generated by a code-grounded multi-agent council (40 agents, ~2.1M tokens). Companion to Round 1 (`FEATURE_IMPROVEMENT_RD_2026-06-01.md`). The re-do table and Tier-0 items are the highest-trust-risk findings - they were believed fixed. Confirm each `file:line` before fixing; every bug fix needs a reproducing test first.

Make What We Have Better - Feature Improvement R&D (Part 2)

Source: [readme/research/feature-rd/FEATURE_IMPROVEMENT_RD_PART2_2026-06-02.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a deep check-up of 13 features already built into the platform. The research team read the real code, found the problems, and ranked them by how much harm they cause. The goal is not to add new things — it is to fix what is already there so it actually works the way a store owner expects.

The main parts explained simply:

- **Three confirmed bugs** — Three serious problems were checked by hand and proven real. (1) When you click “Mark as Paid” on an invoice, the system flips the label to Paid but records no actual payment, so the balance still shows as owed in red. (2) The entire Packaging product type — boxes, envelopes, dielines — is built and working but impossible to reach because it is missing from the product-type dropdown. (3) Editing a customer’s phone number can accidentally overwrite important internal data like their account number or status without any warning.
- **Silent lies (the biggest problem)** — The screen says “Saved” or “Paid” but the real information was quietly dropped or ignored. This shows up across invoices, inventory, checkout totals, notifications, and automation rules. The fix already exists in the code — it just needs to be connected correctly.
- **Features built but unreachable** — Working code that an owner can never get to because one small wiring step is missing. Adding about six lines of code unlocks the entire Packaging feature. The 3D product viewer is also dead code that no page even loads.
- **Two different numbers for the same fact** — The stock number a store owner edits is not the same number the checkout system checks. The cart total and the checkout total are calculated separately, which is why a customer can see a price jump between cart and payment.
- **Missing “loading / empty / error” states** — When something goes wrong, several screens show nothing at all or show a green “all clear” message. An empty screen looks like a bug; an error that looks like “no data” is misleading and erodes trust.
- **Raw technical labels shown to store owners** — Inventory shows “add/deduct” codes. Imposition shows UUIDs as press-sheet names. Automation uses `[bracket]` tokens that silently break if mistyped. These need plain words and guided pickers instead.
- **Dangerous actions with no warning** — Approving a proof, marking an invoice paid, or deleting a role happens with one tap and no explanation of what will change. The fix is a plain-language confirmation that spells out exactly what is at stake.
- **Dead ends when a setting is not configured** — When a payment gateway or shipping method is not set up, several features just silently disable with no explanation. Owners need a plain message explaining what is missing and a direct link to where they can fix it.
- **4-wave fix roadmap** — Fixes are grouped into four rounds. Round 1 stops the most harmful silent-lie bugs (invoice payment, checkout total, notification rollback). Round 2 unlocks the dead features and adds missing error states. Round 3 unifies duplicate data sources. Round 4 handles the larger rebuilds like a single shared shipping calculation.

What to do with this:

Fix the invoice “Mark as Paid” bug first — it directly touches money and trust. Then add the six lines that unlock the Packaging product type, since it is the cheapest high-impact change on the platform.

Date: 2026-06-02 **Goal:** Same as Part 1 - not new features, but making our **existing** features easier, clearer and more useful for our **current** non-technical users. Part 2 covers the modules Part 1 didn't.

Method: Multi-agent council. For each of 13 features: an audit agent **read the real code** (file + line), then a two-person council (non-technical-owner advocate + UX/support-load reducer) debated, then the chair synthesized. 40 agents, ~2.2M tokens, grounded in this codebase. **Companion doc:**

`readme/FEATURE_IMPROVEMENT_RD_2026-06-01.md` (Part 1 - Products, Pricing, Orders, Designer, B2B, Storefront, AI Builder, Action Center, Integrations, Payments, Onboarding, Analytics, Campaigns). Part 1's Wave 1 is already implemented. **Features covered here:** Imposition, Packaging, Proofs & Approval, Inventory, Quotes & Invoices, Subscription/Plan Billing, Roles & Permissions, Notifications, 3D Preview, Cart & Checkout, Customer Management, Automation Builder, Store Settings. **Verification:** The three headline silent-lie bugs (invoice Mark-as-Paid, Packaging unreachable, customer mass-assignment) were hand-verified against source before publishing. Other `file:line` citations are strong leads - confirm before fixing.

The same villain, in new rooms

Part 1 found that features quietly **lie** - the screen says success while the real value is dropped. Part 2 confirms this is the platform's dominant failure mode, and it now touches **money, inventory, and orders** directly:

✓ Three headline bugs, verified by hand

1. Invoice “Mark as Paid” records no payment (Quotes & Invoices) `InvoiceController::markAsPaid` calls the bare model method `$invoice->markAsPaid()` - which flips the status to “paid” but **creates no Payment row**. Meanwhile `InvoiceController::show` derives `amount_paid` and `balance_due` from the payments relation. So an owner who takes cash, clicks “Mark as Paid,” sees the invoice flip to Paid - and then sees “**\$0.00 paid / full balance due in red**” and an empty Payments list. The correct code already exists in `InvoiceService::markAsPaid` (creates the Payment, sends the receipt, accepts the linked quote, audit-logs). **Fix = delegate the controller to the service.** This is wiring, not new logic.

2. The entire Packaging feature is unreachable (Packaging) `PRODUCT_TYPE_OPTIONS` in `nuxt/app/utils/productLabels.ts` lists `printing / tshirt / ecommerce / service / hybrid` - **packaging is missing**. The whole built-and-working chain (Packaging config tab, per-panel designer view, 3D box preview, dieline math, storefront Outside/Inside surface toggle) only activates when `product_type === 'packaging'`, which an owner can never select. **Fix = add ~6 lines** (one label + one option). Cheapest line-for-line win on the platform.

3. Customer update can silently corrupt protected data (Customer Management)

`CustomerController::update` calls `$customer->update($request->all())` **twice** (lines 402 & 412),

and `Customer` has `$guarded = []`. So editing a phone number can silently overwrite `account_number`, `source`, `is_active`, or even `tenant_id`, and the double-write corrupts the audit baseline. **Fix** = `whitelist` to the editable contact fields with a single `update($request->only([...]))`. Pure correctness/security, no UX change.

The 7 recurring patterns (fix once, help everywhere)

Pattern 1 - Confirmed-then-discarded silent lies (*the worst, highest-leverage*)

The UI shows Saved/Paid/Approved while the real value is dropped, mis-routed, or not honored. **The correct behavior almost always already exists in code - the wrong path is just wired in. Where:** Invoices (Mark-as-Paid records no Payment) · Inventory (restock writes `products.stock`, cart reads `settings.stock_quantity`) · Cart → Checkout (agreed total silently changes at payment) · Notifications (toggles flip in UI before a POST with no error handling) · Automation (Artwork/Quote&Artwork approval rules save green but can never fire) · Proofs (admin can still edit/re-send artwork a customer already approved) · Customer update (mass-assignment overwrite) · Store Settings (Business Tax Rate accepts `"GST"` / `"10%"` strings into pricing math). **Fix:** Hunt every “save then toast” path and make the toast tell the truth - delegate to the code that actually persists, read/write ONE field end-to-end, snapshot-and-rollback optimistic toggles on failure, and only offer choices that can actually fire.

Pattern 2 - Fully-built-but-unreachable features (*cheapest high-impact wins*)

Working code shipped dead because of one missing wiring line. **Where:** Packaging (missing dropdown entry) · Plan limits (`usePlanLimits` composable imported in `zero .vue` files) · Buyer AR (`ProductARViewer.vue` imported nowhere, package not installed) · Imposition's New-modal Recalculate button wired to `@plan="()=>{}"` · a dead frontstore proof mock that fakes success. **Fix:** Add the one missing line to light up the built code - or delete the dead component so it can never ship a broken empty tag.

Pattern 3 - Two unsynced sources of truth for the same fact

The number the owner edits is not the number the system honors. **Where:** Inventory (`products.stock` vs `settings.stock_quantity`) · Cart vs Checkout totals (two independent shipping computations) · Store identity (name/address/logo editable in both `/setting/account` → tenants AND per-store settings) · Proof status (literal strings written outside `ProofStatusEnum`). **Fix:** Pick ONE source per fact, repoint all readers, remove/lock the duplicate editor, add a regression test asserting the two surfaces agree.

Pattern 4 - Missing loading / empty / error states (a failure reads as “all clear”)

Where: Inventory history (error → “No changes recorded yet”, implies a wiped log) · Notification bell (error → “all caught up”) · Roles list & editor (console-error-only → blank form) · Proof public review page (blank white card on error) · Store settings (try/finally with no catch → empty Currency dropdown) · Packaging tab. **Fix:** Every data view gets three distinct branches - skeleton, helpful empty, plain-language error + Retry. An error must never render as empty/“all clear.”

Pattern 5 - Raw technical output & unexplained dangerous controls

Where: Imposition (UUID as a press-sheet label; Bleed/Gutter/Creep with no help; unitless money like `2.04 / 0.0610`) · Inventory (`add / deduct` enum badges) · RBAC (Void Order / Show Line Item Price render as plain toggles beside harmless ones) · Automation (`[customer.primary_contact_name]` bracket tokens → typo yields “Hi ,”) · Customer list (raw `guest / registered`). **Fix:** Map every enum/ID to a plain label, add the store currency via the shared `formatMoney` util, add inline help to jargon, flag financially dangerous permissions with a consequence string + confirm, give owners presets and an “Insert info” picker instead of blank slates and raw tokens.

Pattern 6 - Destructive/committing actions with no consequence-aware confirmation

Where: Proof Approve (one tap commits a paid print run) · Invoice Mark-as-Paid (no amount/method/date, no confirm) · Role delete · Store Settings Reset (silently discards unsaved edits) · Customer archive (tied to live orders/revenue). **Fix:** Restate the exact consequence + count affected; the confirm button restates the action (“Approve & Send to Print”, “Archive Customer”). For payments, capture amount/method/date instead of one irreversible click.

Pattern 7 - Unconfigured-integration dead-ends instead of guidance

Where: Subscription “no payment gateway” → “contact support” dead end · Checkout when shipping required but none configured → silent disabled Place Order · Notifications SMS column permanently greyed with no reason · Imposition PDF failure → blind Regenerate forever. **Fix:** Replace dead ends with a plain explanation + a concrete next step (Request setup, Contact store, link to Settings, or the captured failure reason mapped to a fix).

Roadmap - 4 waves, ordered by money-and-trust risk

Founder’s rule of thumb: anything where “the screen says X but the system does Y” jumps the queue - that’s the failure that turns a non-technical owner into a support ticket they can’t even diagnose.

Wave 1 - Stop the bleeding (silent-lie quick wins, ship first)

1. **Invoices** - `Mark as Paid` → delegate to `InvoiceService::markAsPaid` so a Payment row is recorded.
2. **Imposition** - block the no-fit `0x0` plan: show the already-computed reason and disable Save/Approve.
3. **Notifications** - wrap settings toggles + bell actions in try/catch with snapshot-and-rollback.
4. **Customer** - whitelist the update endpoint; remove the double `update($request->all())`.
5. **Automation** - remove the approval options that can never fire (Artwork / Quote & Artwork) + the dead `task_completed` trigger.
6. **Checkout** - move the “Order placed” success toast to *after* payment; pass the real grand total (not subtotal) to the success page.

Wave 2 - Make the dead reachable + close obvious dead-ends (quick wins)

7. **Packaging** - add `packaging` to `PRODUCT_TYPE_OPTIONS` / `PRODUCT_TYPE_LABELS`.
8. **Subscription** - give plan-limit 403s a machine code + an “Upgrade Plan” toast/button; add a fallback so `/auth/subscription` is never a blank lockout screen.
9. **Store Settings** - add a “Currency & Region” entry to the global Settings sidebar (deep-link to per-store regional controls); validate Business Tax Rate as a number.
10. **Everywhere** - add the mandatory loading/empty/error states (Inventory history, bell, Roles, Proof public page, Store settings).
11. **Cleanup** - delete the dead AR viewer and the fake frontstore proof mock so they can’t ship broken.

Wave 3 - Unify duplicated truth + add the real workflows (medium)

12. **Inventory** - unify on `products.stock` as the single source the storefront reads; remove the duplicate Settings stock editor; auto-deduct on order placement (respect backorders) with order-reference history.
13. **Proofs** - show the order spec (qty/size/substrate/finishing/in-hand date) on the public review page, capture a required reject reason, confirm before Approve, make `ProofStatusEnum` authoritative (locks edit/re-send once approved).
14. **Invoices** - replace one-click Mark-as-Paid with a “Record Payment” dialog (amount/method/date, supports deposits & partials; auto-flip to Paid only when balance hits zero).
15. **RBAC** - plain-language helper text per access level, danger flags + confirm on Void/price-visibility permissions, surface the existing `config/permission.php` presets as “Start from a role...”.
16. **Subscription** - wire `usePlanLimits` meters (“X of Y used” + disabled-with-reason Add button) into Products/Stores/Team list pages.

Wave 4 - The big investments (large)

17. **Cart & Checkout** - compute shipping + grand total **once on the server** for a cart + method; both cart and checkout read that single value; carry the cart’s selected method into checkout; regression test asserting `cart total === checkout total`. (*Mandatory price-calculator-sync rule.*)
18. **Inventory** - full auto-deduct-on-order pipeline.
19. **Packaging** - surface-area pricing (charge by box size + material).
20. **Numbering** - per-tenant numbering for quotes/invoices/payments.

Per-feature highlights

Imposition - *Sharpest*: a no-fit layout saves as a valid `0x0` “Planned” imposition the operator can Approve into a blank press sheet; the planner already computed the reason (`meta.error`) but nothing shows it. → Block + show the reason (9); live Recalculate preview in the New modal (8); surface the captured `pdf_error` with a recovery step (8); currency symbol on all money values.

Packaging - *Sharpest*: unreachable (missing dropdown entry). → Add it (10); validate depth so envelopes/pouches can’t save broken (7); later, surface-area pricing (8).

Proofs & Approval - *Sharpest*: the buyer approves a paid print run seeing only title + images, **no spec** to check against. → Show what they're approving + capture a reject reason + confirm (9); loading/already-decided/expired states on the public page (8).

Inventory - *Sharpest*: two unsynced stock numbers; the one the owner edits isn't the one checkout honors. → Unify on `products.stock` (10); auto-deduct on order (9).

Quotes & Invoices - *Sharpest*: Mark-as-Paid records no payment (verified). → Delegate to the service (10); Record-Payment dialog with deposits/partials (9).

Subscription & Plan Billing - *Sharpest*: the "invisible wall" - owners fill a whole form then get bounced by a vanishing red toast with no upgrade path. → Plan-limit code + guided upgrade (9); `/auth/subscription` fallback (8); `usePlanLimits` meters (8).

Roles & Permissions - *Sharpest*: consequential security decisions from a blank slate with zero on-screen meaning, dangerous powers shown identically to harmless ones. → Helper text per level (9); danger flags + confirm (8); role presets (8).

Notifications - *Sharpest*: every settings toggle is silent fire-and-forget (no try/catch, no rollback). → Save feedback + rollback (9); bell error/empty states (8); gate SMS/Email/Push behind their integration (8).

3D Product Preview - *Sharpest*: the only buyer-facing piece (AR viewer) is dead code imported nowhere, package not installed. → Honest admin toggle copy (8); delete the dead viewer (7); "preview only" trust line (7); WebGL guard + init timeout so the spinner can't spin forever (7).

Cart & Checkout - *Sharpest*: the agreed cart total silently becomes a higher number at payment (QA saw \$157.65 → \$208.65). → One server-computed total shared by both (10); cart shows "calculated at checkout" until a method is chosen (9); default to cheapest method (8).

Customer Management - *Sharpest*: "Total Spent" sums cancelled/draft/unpaid orders (no status filter) so the headline value is wrong; plus the mass-assignment hole. → Lock down the update endpoint (9); make Total Spent count only real revenue + label it (9); surface B2B context on the detail page (8).

Automation Builder - *Sharpest*: the most natural rule ("When Artwork approval is approved, email the customer") saves with a green toast and **can never fire** (`ConditionValidator` only matches `Quote / Invoice`). → Align options with what actually fires (10); replace `[bracket]` tokens with an "Insert info" picker + plain when/then language (9). (Note: Part-1 Wave-1 already made Order a real trigger context here.)

Store Settings - *Sharpest*: Currency - the most-searched setting - is un-findable from the global Settings sidebar, so it stays unset and invoices render in default USD. → Add "Store & Regional" to the sidebar deep-linking to Currency (9); error+retry on both settings loads (8); validate tax rate as a number (7).

Generated by a code-grounded multi-agent council (Part 2). The three headline bugs were verified by hand against the source. Other `file:line` citations are strong leads - confirm before fixing. Per the house rule, each bug fix needs a test that reproduces the bug before the fix passes.

Frontstore Design R&D - 2026-06-02

Source: [readme/research/feature-rd/Frontstore_Design_RD_2026-06-02.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a visual design review of the customer-facing online store — the pages your shoppers actually see and use. Ten reviewers looked at every major page and listed what looks unprofessional, confusing, or broken. It matters because a store that looks cheap or inconsistent loses customers before they even consider buying.

The main parts explained simply:

- **Fake demo content on the homepage** — The storefront was showing made-up stats (“2,847 Orders This Week”), a fake coupon code (“WELCOME10”), and a fabricated countdown sale. Real shoppers see this and lose trust immediately. These have been turned off.
- **Buttons and colors ignoring the store’s own theme** — The main button (used 352 times across the store) had its colors coded directly into it instead of reading from the store’s color settings. This meant themed or branded stores looked half-styled — some parts matched the brand color, others didn’t. This is the single biggest fix.
- **Inconsistent corner rounding** — Some cards had sharp corners, others had very round ones, even on the same page. This makes the store look like it was built by several different people who never talked to each other.
- **Duplicate components — the cheaper copy usually wins** — There are 4 different footer designs, 3 different loading animations, and 3 different star-rating widgets scattered across the store. Each has a polished version and a rough version; the rough one tends to be what customers see.
- **Text too small to read** — Several pages used text as small as 8–10px (very small) in a light gray color that barely passes basic readability standards. This looks cheap and is hard to read on a phone.
- **Mobile hero = black box** — On a phone screen, the top banner image showed as a completely black empty box. Customers landing on the store from a phone saw nothing useful above the fold.
- **Currency symbol hardcoded as “\$”** — The mini cart, search results, and live search all showed “\$” regardless of the store’s actual currency setting. This breaks non-USD stores.
- **Raw order status codes shown to customers** — Instead of “In Progress,” customers saw the internal code word “open.” Fixes have been applied so plain-language labels (“In Progress,” “Completed”) now show everywhere.
- **What has already been fixed (shipped)** — The fake promo blocks, the low font size floor, the currency symbol, the plain-language order statuses, the shop-by-category tile design, and an empty product rail layout have all been corrected and verified.

What to do with this:

The most important next steps are to finish tokenizing the main button (so the store’s brand color applies everywhere) and to do a radius consistency sweep so cards and buttons look unified. The post-purchase “thank you” and order-tracking pages are already the best-looking part of the store — use those as the visual target for every other page.

Design-only council on the customer storefront (`frontstore/` , Nuxt 4, `printdesign.localhost:3001`). 10 agents (~780k tok, workflow `wf_f62d081a-cdc`) each critiqued one surface for **visual quality** - hierarchy, spacing, typography, color/contrast, consistency, mobile, motion, conversion, state-polish. **Scope = look & feel only**. Functional fake-success/dead-button bugs are owned by [`FRONTSTORE_FEATURE_RD_2026-06-02.md`]. Cross-checked against a live browser pass at 1440px + 390px.

Headline

The storefront has a **capable theme-token system and several genuinely premium surfaces** (post-purchase pages, cart/checkout bones, the product card, the form primitives) - but it reads as **several designers' work stitched together** because the most-shipped components **bypass the tokens** and the **same UI exists in 2–4 drifted copies**, and the cheaper copy is usually the one that renders.

Cross-cutting themes (fix these → many pages improve at once)

1. **Hardcoded Tailwind grays/colors bypass the token system** → themed/branded/dark stores render half-styled. `BaseButton` (352 usages) leaks `bg-gray-100/border-gray-300/bg-red-600` ; `FeaturedProductBlock` / `TestimonialBlock` / footer/login/order-success all hardcode gray. **Single highest-leverage fix**.
2. **Radius free-for-all** - `rounded-xl/2xl/3xl/full/theme` coexist, often in one component → on a sharp or pill theme, cards and buttons disagree and the page looks “half-themed and broken.”
3. **Duplicated-but-divergent components** - 4 footers, 3 headers, 2 MiniCart/LiveSearch generations, 3 skeleton systems, 3 star ratings, 2 badge systems. Each fork has a polished and a cheap side; the cheap side often ships.
4. **Micro-typography floor too low** - `text-[8px]/[9px]/[10px]` and `text-gray-400` body (≈2.8:1, below WCAG AA) pervade cart/orders/footer/badges → “cheap template” + illegible on the mobile-first audience.
5. **Hover-only affordances + hardcoded fake/demo content** - gallery arrows/Quick View/wishlist/address edit gated behind `group-hover` (invisible on touch); homepage ships a fabricated Black Friday promo, `10,000+/2,847` stats, `WELCOME10` coupon, fixed `2026-12-31` countdown; MegaMenu ships emoji “Special Offer” + fake `1-800-PRINT` .

Live browser pass confirmed (printdesign.localhost:3001)

- **Mobile hero = black void**. Hero is a raster JPG (with a literal `https://printdesign.app/` baked into the art); `object-contain` + dark overlay letterboxes on desktop and renders an **empty black box at 390px**. (Theme 5 + HeroBlock finding.)
- **Three overlapping, starved product rails** - *Featured Products* shows **one lonely card** in a white void; *New Additions* is **six “No Image Available”** placeholders; redundant with *Most Popular*.

- **Off-brand promo bar** - “Diwali offer - *Click me* →” in default underlined link-blue on black.
- **Trust-eroding seeded stats** - “On-time Delivery 25%”, “4 out of 5”, “2,847 Orders This Week.”
- **Flat product cards** in the live grids (markup says the card *can* hover-lift; the rails/skeletons undercut it).

Tier 1 - high impact, low effort (do first)

1. **Tokenize** `BaseButton` non-primary variants (neutral/outline/ghost/danger + hover tints) → theme vars; drop per-variant `active:scale-95` . `base/BaseButton.vue:101-114` . *One file fixes 352 buttons.*
 2. **Collapse 3 skeletons** → 1 tokenized `BaseSkeleton` (shimmer); `AppSkeleton` / `BaseImage` inline/ `HeroSkeleton` render it. *One loading language.*
 3. **Gate homepage fake promo/stats/coupon/countdown behind real store settings**; trust-badge copy from store policy. `pages/index.vue:269` . *Removes the most embarrassing content.*
 4. **Replace raw gray- with tokens*** in `FeaturedProductBlock` / `TestimonialBlock` . *Two product sections stop looking like different products.*
 5. **FeaturedProductBlock** → shared `SectionHeader` (badge + accent line). *Fastest homepage cohesion win.*
 6. **Listing/search skeletons** - category page → `ProductGridSkeleton` ; search skeleton `aspect-[4/3]` → `aspect-square` (kills CLS). `category/[slug].vue:111` , `search.vue:176` .
 7. **Unify product badges on** `AppBadge` ; drop rainbow `ProductBadges` + `animate-pulse "Hot"` ; show discount once. `ProductBadges.vue:71` , `shared/ProductCard.vue:93` .
 8. **Fix hardcoded \$** - thread `currencySymbol` into aurora MiniCart + LiveSearch + search price filter. `MiniCart.vue:11` , `search.vue:110` .
 9. **Gut MegaMenu demo content** (emoji banner, `1-800-PRINT` , repeated slugs, fake checkmarks) or delete if `DynamicMenu` covers it. `MegaMenu.vue:233` .
 10. **aurora LiveSearch** - add no-results empty state + content-shaped skeleton (copy `shared/LiveSearch.vue`). `aurora/.../LiveSearch.vue:31` .
 11. **Stop raw** `{{ order.status }}` on order detail - reuse `statusLabel()/statusDot()` ; one shared `statusColor.ts` badge everywhere. `profile/orders/[id].vue:23` .
 12. **Raise micro-type floor** - kill `text-[8px/9px/10px]` body → `≥ text-xs` ; `text-gray-400` readable copy → `gray-500/600` (AA). `cart.vue:64` , `orders/index.vue:51,110` , `AppBadge.vue:36` .
- **Hero fit** - default full-bleed heroes to `object-cover center` (fix the mobile black-void/desktop letterbox). `HeroBlock.vue:26` .

Global token / primitive changes (lift the whole store at once)

- Expose `text-theme-muted/subtle` + add `--theme-skeleton` + `--theme-color-surface-soft` ; one find-replace moves `gray-400/500` , `bg-gray-50` , `slate-200` skeletons onto tokens (color correctness + AA).

- Tokenize all `BaseButton` variants; single `.theme-button-base:active` press.
- One `Skeleton` primitive; one shimmer color/tempo.
- Radius contract: cards=`rounded-theme-card`, inputs/chips=one nested token, pills=`rounded-full`.
- Type floor 12px (10px only for bold uppercase eyebrows); one price scale (tabular-nums, drop `font-serif`); one `AppBadge` font scale; one `statusColor.ts`.
- Collapse accent palette to primary + 1 success + 1 warning + 1 danger + neutral.

SHIPPED (2026-06-02, uncommitted, browser-verified, nothing broken)

Wave 1 - homepage professionalism + one global consistency lever:

1. **Disabled 3 fabricated demo blocks** at the source (`pages/index.vue`): `promo_banner` ("Black Friday 25% Off" + fixed countdown), `stats` ("10,000+ / 2,847 Orders This Week"), `cta_banner` ("WELCOME10"). They were hardcoded literals shown regardless of the admin toggle - fake metrics/offers undermine a real store. `is_active: false` + comments to re-enable once wired to real data. (Flag-default change in `useTheme.ts` was reverted - flags are shared, content was the real problem.)
2. **FeaturedProductBlock** → **shared SectionHeader** (eyebrow badge "● FEATURED" + accent line) so it matches the Most Popular / New Additions rails instead of a hand-rolled `<h2>`. Added `badge: "Featured"` in `index.vue`.
3. **Few-items no-void layout** in `FeaturedProductBlock`: rails with fewer products than a full row render left-aligned at natural card width (was 1 tiny card stranded in a 6-col void).
4. **Promo bar link** → **pill button** (`PromotionalBar.vue` topbar): "CLICK ME →" is now a compact uppercase pill matching the floating/bottom/inline bar CTAs, not raw underlined link-blue text.
5. **Type floor (global, one place)**: `storefront.css` lifts all sub-11px text (`text-[7/8/9/10px]`) to an 11px minimum - kills "cheap micro-text" across every page in a single scoped rule. Verified cart/footer/cards/reviews unaffected.
6. **Hidden 0% review metrics** (`CustomerReviewsShowcaseBlock.vue`): empty "On-time Delivery 0%" bars filtered out; whole block hides when no real data.

Wave 2 - cross-page correctness/consistency: 7. **Currency symbol** now from store settings (was hardcoded `$`): aurora `MiniCart.vue` (3x, via `formatPrice`), aurora `LiveSearch.vue` (via `formatPrice`), `search.vue` price-range filter (via `currencySettings.symbol`). Correctness for non-USD stores. 8. **Humanized order status** - new shared `app/utils/orderStatus.ts` (`orderStatusLabel`) used in `profile/orders/[id].vue` + `profile/orders/index.vue` (list + modal). Customers see "In Progress" not raw "open"; same label everywhere. Mirrors track-order's map + title-cases unknowns.

Wave 3 - IMPACTFUL premium-minimal homepage redesign (user picked: Homepage first impression + Premium-minimal vibe): 9. **Shop-by-Category tiles redesigned** (`ShopByCategoryBlock.vue`) - dropped the heavy dark gradient + white-text-over-image; now clean photo frames with a subtle ring, slow 700ms zoom + gentle -1 lift on hover, and the **label below the image** in refined dark type ("Business Stationery / 5 items"). Bigger quiet heading (title-size sm → md), more whitespace (py + gaps up). Browser-verified: real

photos load, hover works, zero console errors. 10. **Product card refined premium-minimal**

(`ProductCard.vue`) - border gray-200 → gray-100 (airier); removed the loud orange inline “-X%” (badge + strikethrough already convey discount). Card keeps its own token radius/shadow/-3px hover lift. 11.

Removed double-framing in `FeaturedProductBlock.vue` - each card was wrapped in a *second* card (bg-white rounded-xl border hover:shadow); now the ProductCard supplies its own frame and breathes. Gaps bumped to gap-5/6. Verified clean on home + category (zero console errors).

HMR note: editing `index.vue` (destructure removal + ref changes together) caused a *transient* `showPromoBanner is not defined` during dev hot-reload churn. Source is clean (grep-verified) and the homepage renders fully on a clean reload - not a real bug. When removing a destructured name, the same-file refs must be gone too (they are).

Deferred (flagged, NOT changed): hero `object-fit` - the store’s banner has **text baked into the raster image**, so the council’s “use object-cover” would crop it on mobile. The mobile black-void is really a content problem (desktop-only banner + dark overlay over letterbox bars). Needs a content fix (proper mobile banner / HTML-text hero), not a blind code change.

Still queued: radius consistency sweep; currency-symbol hardcoding (search/MiniCart/LiveSearch); raw order-status labels; aurora LiveSearch empty state; MegaMenu demo content; per-page polish on product-detail / category / checkout.

Already good - DO NOT regress

- Form primitives (`BaseButton/Input/Select/Textarea`) are genuinely token-driven with a real focusing system.
- The **product card** itself (token radii/shadow, -3px hover lift, glass Quick View, composed price row).
- **Post-purchase** (`order-success` , `track-order`) - status state machines, gradient orbs, steppers, timelines. **This is the visual target for the rest of the funnel.**
- **Cart/checkout bones** - sticky summary, free-ship progress bar, mobile action bar + safe-area, “why disabled” microcopy.
- Standardize ON the newer shared parts: `SectionHeader` , `shared/LiveSearch` , `ProductGridSkeleton` , `AppBadge` , `CheckoutSectionCard` , `statusColor.ts` - route everything through these, don’t rebuild.

Storefront (Customer) R&D - Make the Shopping Journey Easy to Trust & Complete

Source: [readme/research/feature-rd/Frontstore_Feature_RD_2026-06-02.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a detailed check-up of your customer-facing online store — the pages your shoppers actually see and use. It matters because a team of reviewers read through the real store code AND walked through the live store to find exactly where shoppers get confused, misled, or give up before buying.

The main parts explained simply:

- **The biggest problem — buttons that lie** — Several important buttons (like “Request a Quote” and “Add to Cart” on category pages) look like they work, but do nothing. A shopper clicks, sees a success message, and nothing actually happened. No quote was sent. No item was added. This is the single most damaging thing in the store because shoppers who feel tricked stop trusting everything else.
- **Fake data shown as real** — The store shows made-up things as if they are real: a fake order number generated by the computer, a “2,847 Orders This Week” stat that is invented, a “Diwali offer” banner that appears even on stores that never set one up, and a whole profile page showing “John Doe” with fake orders. Shoppers who spot this leave.
- **Silent failures — the store goes blank instead of explaining** — When something goes wrong (a page fails to load, an address fails to save), the store often shows nothing — a blank space, an endless spinner, or a misleading “no results” message. Shoppers think the store is broken.
- **Disabled buttons with no explanation** — Buttons like “Add to Cart” or “Place Order” are sometimes greyed out with no message telling the shopper why or what to do next. This leaves people stuck with no way forward.
- **Login suddenly required at the worst moment** — Shoppers who are ready to buy get sent to a login screen without warning. This interrupts the purchase at the highest-stakes point.
- **Wrong or mixed-up prices** — Some pages show prices in dollars (\$) and others in rupees (₹) regardless of the store’s actual currency setting. The cart total sometimes includes tax without showing a tax line, so the total appears higher than the subtotal for no visible reason.
- **Deleting things with no warning** — Actions like “Clear Cart” happen instantly with no “are you sure?” message. Shoppers lose their cart by accident with no way to undo it.
- **Broken on phones** — Key parts of the store overflow, overlap, or are too small to tap on a standard phone screen (375px wide), which is how most shoppers visit.
- **Logins being silently rejected** — On stores using the “I’m not a robot” security check, a bug causes every login attempt to fail even when the shopper does everything right. This blocks purchases entirely.
- **Checkout can claim success when no payment happened** — In some situations the store shows “Order Placed Successfully!” and clears the cart before any payment was actually confirmed. The shopper thinks they ordered; no order exists.

What to do with this:

Fix the three purchase blockers first — they stop sales today: (1) logins silently failing on stores with the robot-check enabled (a few hours of work), (2) “Add to Cart” and “Request a Quote” buttons that do

nothing (wire them to the real backend or hide them until they work), and (3) the checkout showing a success screen before payment is verified (show a neutral “Confirming your payment...” screen instead). Everything else in the ranked list follows in order of impact.

Date: 2026-06-02 **Scope:** The **customer storefront** (`frontstore/` , Nuxt 4) - the shopper-facing app at e.g. `printdesign.localhost:3001` . NOT the admin. **Goal:** Not new features - make our **existing** storefront features easier, clearer, and more trustworthy for our **current users: non-technical, mostly-mobile shoppers** buying printed products. **Method:** 25-agent R&D council (~1.3M tokens), code-grounded. Per feature: an agent read the real `frontstore/` code for shopper friction, then a customer-advocate vs conversion/support-load council debated and ranked improvements. A chair synthesized across the whole journey (land → browse → configure/price → design → cart → checkout → pay → confirm → account/reorder). **Plus:** a **live browser walkthrough** of the running storefront (home, search, category, cart, checkout, login, track-order) to confirm findings at runtime.

Live walkthrough - what a real session showed

Walked the running store at `printdesign.localhost:3001` . **No JavaScript console errors on any page** - the storefront is stable. But two things stood out at runtime, both confirming council findings:

1. **Hardcoded promo baked into every page (confirmed live).** A black top-bar “**Diwali offer - Click me** →” shows on the home, cart, checkout, login, category and search pages of a store whose owner never set up a Diwali offer. This is the council’s #1 finding (fake day-one promos/stats) seen in the wild.
2. **Cart & checkout line-item thumbnails are blank grey boxes.** On the Cart and the Checkout “Order Summary”, *Vinyl Banners* and *Yard Signs* show an empty image placeholder - even though the same products show a real image on the homepage product cards. The cart line-item image binding is not resolving the product thumbnail. (New finding from the live pass; not in the council list - worth a quick fix.)

Everything else rendered correctly: search (“12 of 43”, working filters), cart totals, checkout stepper (“Customer Details → Delivery & Payment”, SSL badge), login (Sign In / Create Account), category pages with counts.

1. The single most important finding

The storefront’s biggest problem isn’t missing features - it confidently lies, then goes silent. The same shape recurs at every stage: the UI presents something as real and finished when it isn’t, and when it actually breaks it says nothing useful. A non-technical shopper can’t tell the working parts from the fake parts, so they stop trusting *all* of it.

The clearest form is the “**confirmed-then-nothing-happens**” **silent lie** on the highest-intent actions:

- **Request a Quote** on the product page runs a fake timer and always says “sent”: `setTimeout(...1500)` then `toast.success('Quote request sent...')` with **no API call** (`frontstore/app/pages/product/[slug].vue:497-517`).
- The `/quote` **calculator’s** “Request Formal Quote” / “Add to Cart” only flash success - no quote, no contact captured (`quote.vue:157-171`).
- **Category/search “Add to Cart”** is an empty TODO stub (`category/[slug].vue:820-822`).
- **Wishlist “Add to Cart”** silently navigates instead of adding (`profile/wishlist.vue:63-66`).
- **Order confirmation** declares “Order Placed Successfully!” and invents a fake order number with `Math.random().toString(36)` when payment was never verified (`order-success.vue:331-337, 770-797`).
- A whole **fake** `/profile-unified` page ships live with “John Doe”, invented orders, and dead buttons.

If you fix one thing: make every “success” correspond to a real backend result, and never show fabricated data (prices, stats, order numbers, identities) as the shopper’s own. A shopper lied to once - a failed WELCOME10 coupon, a quote that never arrives, an order number that can’t be found - distrusts the whole store, including the parts that work.

2. Recurring cross-cutting patterns (fix once, help everywhere)

A. Silent-lie / fake-success (*blocks/stalls purchase, worst class*) - Product (fake quote), Orders/Quotes (`/quote`), Catalog/Search (Add-to-Cart TODO), Account (fake `/profile-unified`, wishlist add, Settings wiping name on save - `settings.vue:136-140`), Order confirmation (fake order #, false “paid”), Auth (forgot-password “success” even on failure - `login.vue:528-532`), Home (fake WELCOME10 + “2,847 Orders This Week” - `index.vue:367-387, 443-451`), Content (fake contact email/phone).

B. Missing loading/empty/error states → “**broken store**” - Cart (no loading → false “empty cart” flash; `cart.vue:421-437` doesn’t destructure the available `isLoading / error`), Catalog/Search (fetch failure → “No matches found”; `search.vue` has no catch), Home (CMS page 500 → “404 Page Not Found”, no retry - `useStorePage.ts:100-104`), Account (addresses/wishlist/dashboard never read their `loading` refs), Orders/Quotes (detail load failure → dead “not found”), Checkout (config load failure → permanently dead payment section), B2B (approvals load failure → cheerful “Nothing waiting”).

C. Disabled/dead controls with no explanation (*violates the no-disabled-button-without-a-reason rule*) - Product (Add/Design/Buy/Quote disabled, no tooltip - `ProductActions.vue:4, 41, 55, 66`), Design studio (Add-to-Cart disabled, `validationErrors` filtered out of view), Checkout (Place Order blocked by empty State, blank reason - `checkout.vue:1206, 1776-1783`), Auth (Sign In greys while reCAPTCHA loads, no helper).

D. Raw technical output leaking to shoppers - `e.message` rendered (`productInfo.ts:948`), raw SKU in quote modal, `Option ${product_option_id}` in cart (`cart.vue:95`), raw `new / open / void` statuses

everywhere, `Block type "hero" is not yet supported` (`CmsBlockRenderer.vue:29-34`), “create the Our Store form in admin settings” shown to a buyer (`DynamicFormBlock.vue:243`).

E. Surprise login wall at peak intent - Product, Design studio, Cart, Checkout, Order confirmation (“View My Orders” dead-ends guests at login).

F. Currency/price inconsistency - hardcoded `$` (`search.vue:110,118` , `design-options.vue:461-464`), hardcoded `₹` (`QuickReorder.vue:167` , `/quote`), header “From \$75” vs total “\$77”, cart tax baked into total with no tax line.

G. Destructive actions with no real confirmation - Clear Cart instant wipe (`cart.vue:570-578`), native `confirm()` address delete + backdrop-click discards typed address (`addresses.vue:401,83`), order cancel without stating refund/artwork consequences.

H. Mobile-broken at 375px (the majority device) - hero `text-8xl` overflow + 8px nav dots (`HeroBlock.vue:59-62`), header search crushed to a sliver, wishlist heart hover-only (`ProductCard.vue:36`), floating price bar covers content, order/quote tables force horizontal scroll, single-page checkout shows “Place Order” before payment seen.

3. Ranked roadmap - highest-leverage fixes to EXISTING features

Tier 1 - hours each, high impact (do first)

1. **Wire the Cart's unused** `isLoading` / `error` refs so a returning shopper doesn't see a false “Your cart is empty” flash and a failed load isn't mistaken for a wiped cart. `cart.vue:421-437` .
2. **Show the Tax line + add a coupon field on the Cart** - `applyCoupon` / `removeCoupon` / `taxAmount` already exist in the store (`cartStore.ts:88,452-486`), just unrendered, so the Estimated Total silently exceeds the visible rows.
3. **Make** `login()` / `signup()` **actually send** `recaptcha_token` - both call sites pass it; the composable drops it (`useCustomerAuth.ts:142-153`), so on reCAPTCHA-v2 stores **every login is wrongly rejected**. (*purchase blocker*)
4. **De-jargon block/form failures** - gate `CmsBlockRenderer` `resolveError` behind the dev flag; replace “create the Our Store form in admin settings” with a shopper-facing fallback. `CmsBlockRenderer.vue:29-34` , `DynamicFormBlock.vue:243` .
5. **Stop fabricating shopper data** - kill the fake order number (`order-success.vue:331-337`) and delete/auth-guard the live `/profile-unified` page.
6. **Explain every disabled CTA** (inline hint + tooltip) and stop filtering `validationErrors` out of view. `ProductActions.vue` , `design-options.vue:490-493` , `checkout.vue:1776-1783` .
7. **Replace hardcoded currency symbols** with the store's `formatPrice` / `currencySymbol` . `search.vue:110,118` , `QuickReorder.vue:167` , `design-options.vue:461-464` .
8. **Confirm destructive actions** - Clear Cart, address delete, address-modal backdrop-click. `cart.vue:570-578` , `addresses.vue:401,83` .

9. **(live finding) Fix the blank cart/checkout line-item thumbnails** - bind the real product image/thumbnail URL in the cart line item and checkout order-summary row.

■ Tier 2 - ~a day each, high impact

10. **Real error+retry states** on Catalog, Search, order/quote detail, CMS pages - and only return `null` on a true 404 (`useStorePage.ts:100-104`).
11. **Make Request-a-Quote real** on both paths - POST to the working backend, show “sent” only on 2xx (`[slug].vue:497-517` , `quote.vue:157-171`). *(purchase blocker)*
12. **Wire / hide the dead Add-to-Cart buttons** (category, search, wishlist) into the real cart store with feedback. *(purchase blocker)*
13. **Fix Settings name/phone wipe-on-save**; distinguish failed-fetch from empty across the account area.
14. **Translate raw statuses to plain stages** everywhere (orders, quotes, tracking, B2B) with a safe friendly default.
15. **Make the price configurator honest on failure** - never revert to “Select your size...” after configuration; show “We couldn’t price this - Try again”; never render `e.message` .
16. **Surface the hidden Track-Order page** - link it from order-success, footer, account menu; give guests “Track this order” as the primary post-purchase CTA.
17. **Signal login/guest intent upfront**; keep Forgot-Password inline in the checkout modal.

■ Tier 3 - days each, plan for them

18. **Stop shipping fake promos & stats on day-one stores** - flip theme flags OFF until configured (`index.vue:367-387,443-451` , `useTheme.ts:94-96`). *(the live “Diwali offer” banner)*
19. **Never show “Order Placed Successfully!”/“paid” without backend verification** - order-lookup-by-reference, “Confirming your payment...” state, don’t clear the cart (`order-success.vue:770-797`). *(purchase blocker)*
20. **Stop offering payment methods that can’t complete** (`Authorize.Net` embedded) - validate render mode before drafting the order so no phantom order is created. *(purchase blocker)*
21. **Mobile sweep at 375px** as one coordinated pass (hero, header search, sticky buy bars, responsive tables, single-page checkout CTA).
22. **Replace fake About/FAQ/contact/blog-author/“Verified Customer” content** with tenant data or honest omission.
23. **De-bait B2B** - pre-submit “needs approval” banner, fix “Pay on account” redirect copy, approvals error state.

4. Top 3 purchase blockers to fix THIS WEEK

- **1. Logins are silently rejected on reCAPTCHA-v2 stores.** The token is collected then thrown away before the request (`useCustomerAuth.ts:142-153`); the shopper ticks “I’m not a robot”, hits Sign In, gets “Invalid credentials”. **Fix: add the field to the payload. Effort: hours.**

● **2. The primary buy/quote actions don't do anything.** Add-to-Cart from grids is an empty TODO (`category/[slug].vue:820-822`); Request-a-Quote is a fake 1.5s timer (`[slug].vue:497-517` , `quote.vue:157-171`). **Fix: wire to the real backend, or hide the affordance until wired so no dead button ships. Effort: day each.**

● **3. Checkout/confirmation can charge nothing - or claim success on no payment.** A selectable card method fails *after* a draft order is created (`checkout.vue:1541-1548`); redirect gateways returning without an order id land on a green "Order Placed Successfully!" with the cart cleared and payment unverified (`order-success.vue:770-797`). **Fix: hide the un-completable method + validate render mode before drafting; render a neutral "Confirming your payment..." state when no verified order id is present. Effort: day for the method gate.**

Across all 12 storefront features the through-line is one rule: the store must say only true things, show only the shopper's real data and currency, and - when it can't - say "something went wrong, try again" instead of going blank, green, or grey. Tiers 1–2 remove the bulk of abandonment and support load before any net-new feature work. Full per-feature audit data: workflow `wf_16721de6-530` output.

Game-Changer R&D - Strategy for Print-Flow-360

Source: readme/research/feature-rd/GAME_CHANGER_RD_2026-06-01.md

IN SHORT — KEY TAKEAWAYS

This document is the result of a deep research session — 36 AI agents debating ideas — to find features that print shop owners would switch platforms to get. It cuts through the flashy ideas that look good in demos but do not actually help a shop make more money, and lands on three real improvements that no competitor offers today.

The main parts explained simply:

- **The one big advantage** — Print-Flow-360 is the only platform that handles both customer pricing and what happens on the production floor in one system. Competitors like Printful or Gelato don't own a press, so they can never know what a job truly costs you to make. That gap is where we win.
- **True-Cost & Reprint Ledger** — When you quote a job, the system saves what you expected it to cost. After the job is done, the operator records what actually happened — sheets used, spoilage, any reprints, and why. The system then shows you a ranked list in plain dollars of which customers and jobs are quietly losing you money. Over time it suggests better prices for similar jobs. No competitor tracks this today, and shops lose 10–20% of their revenue to this invisible problem.
- **Standing Orders & Print Subscriptions** — A “set it on repeat” button for any past order. The customer or store owner picks how often (monthly, quarterly), and the system auto-creates the order, runs it through production, and charges the customer — without anyone having to remember. Shops that do a lot of repeat work stop leaving money on the table, and repeat buyers are far less likely to shop around.
- **Gang-Run Autopilot** — When several orders can share the same sheet of paper (same paper stock, same finishing), the system groups them and tells you in plain language: “Waiting 6 more hours for 7 more orders saves \$38 in wasted paper — fire now or hold?” It auto-prints when the sheet is full enough or when a deadline is close, whichever comes first. No competitor does this automatically.
- **Ideas that were rejected (and why)** — AR product previews, an AI chat operator that changes prices or settings by voice, and AI shopping agents all sounded exciting but were cut. The technology is not ready, or the risk of a silent mistake is too high for a non-technical owner to catch before money is lost.

What to do with this: Build the Reprint Ledger first — it starts collecting the data that makes everything else smarter, and gives owners a switch-platforms insight on day one. Then add Standing Orders — it turns hidden repeat work into visible recurring revenue with the least effort and no new payment code. Gang-Run Autopilot comes third, once the groundwork is in place.

Date: 2026-06-01 **Method:** Multi-agent R&D council - 4 competitive-recon analysts → 6 expert personas ideating → a facilitator clustering ideas → adversarial debate panels (a skeptical VC + an engineering lead scoring every concept) → a chair synthesizing. 36 agents, ~1.1M tokens of debate.

Question we set out to answer: Every standard web-to-print feature is already shipped by competitors who've been in the market for years. As the new entrant, **what can we offer that no incumbent can - and that a non-technical print-shop owner would switch platforms to get?**

The one-sentence answer

We are the only platform that owns BOTH the quote-side pricing engine AND the production-floor reality in one system - so we can turn the shop's own data into plain-dollar decisions that print-on-demand giants (Printful, Gelato, Printify) structurally *cannot* make, because they don't own a press.

Everything below flows from that single unfair advantage.

Why the obvious ideas lost

The council deliberately **killed** the flashy ideas that demo well but don't win:

Tempting idea	Why we parked it (honest reason)
WebAR “see it in your space”	We have desktop 3D preview, but zero glTF/USDZ export exists and there's no mature pure-Node path to iOS AR. Our parametric meshes read as fake at wall-scale - exactly where large-format buyers need fidelity. A vanity toggle, not a reason to switch.
Talk-to-your-shop AI operator	Scored lowest of the live ideas. Our AI runner is single-shot text with no tool-calling loop, and there is zero undo/rollback anywhere in the codebase. “Reprice all banners +8%” by chat is where a non-technical owner <i>can't</i> catch a hallucination - trust-destroying.
MCP storefront for AI shopping agents	Genuinely clever moat, but as of 2026 the channel is a press release - the buyer sees no real orders. Cheap optionality to watch, not a flagship.
Generic “AI design generation”, 3D configurators, VDP editors	These are now <i>table stakes</i> - OnPrintShop, DesignNBuy, Infigo, Zakeke already ship them. Building them again is catching up, not leapfrogging.

The competitors' real weakness isn't their feature list - it's that they're built for an onboarding team, InDesign-savvy designers, and IT support. None can be run end-to-end by a single non-technical shop owner. Reviewers of the incumbents consistently complain: feature overload, “the ultimate frustrating” admin UX, complicated backend setup, and support that takes 2–3 rounds with a language barrier. That is the door we walk through.

The three flagship bets

All three plug the **same three holes that leak money out of every print shop every month**, and all three are built on engines **we already have**.

🏆 1. True-Cost & Reprint Ledger - “*names who lost the money, then fixes the next quote*”

What it is (in plain words): When you quote a job, we quietly save what we *assumed* it would cost (paper, finishing, run length, setup). When the job finishes, the operator confirms in one tap what *actually* happened - sheets used, how many came out good vs spoiled, and if it was reprinted, *why* (bad customer file / our mistake / press or material fault). Then we show you the truth nobody tracks today:

“Acme’s die-cut labels: 11% reprint rate, all caused by bad files, cost you \$420 last quarter.”

A ranked list, in plain dollars, of **which customers and products are quietly losing you money**. Over time it learns and suggests a corrected price the next time you quote a similar odd job - turning a veteran’s gut estimate into a data-backed default.

Why no competitor has it: The big print-MIS systems (EFI, Avanti) connect estimating to inventory but never close the loop back into a smarter *future* quote. POD players have no concept of the shop’s real production cost at all. Shops lose **10–20% of revenue** to this invisible margin leakage, and reprint root-cause is rarely tracked anywhere. We’re the only platform with both halves in one place.

How we build it on what we already have: The quote half is mostly plumbing -

`ProductPricingCalculator` already emits a structured cost breakdown, and we already persist priced snapshots against quotes/invoices/carts. The actuals half is small and bounded: add `sheets_used / good / spoiled / reprint_of / cause` to `print_jobs`, plus one operator screen at the existing job-completion lifecycle hook. For imposition jobs the physical cost already exists (`PressSheet.cost_per_sheet`, `SheetCostCalculator`, `WasteCalculator`). Rollups reuse `StoreAnalyticsService`; the “these customers cost you money” feed reuses the **Action Center** rule pattern; the corrected-quote suggestion reuses the existing AI harness.

First milestone (ship this, no AI yet): Operator confirms actual sheets + spoiled count + reprint reason at job close → owner gets one screen ranking customers and products by real margin and reprint rate. **This alone is a switch-platforms insight**, and it starts collecting the data the learning loop needs.

Effort: XL - but split it. The reconciliation *report* is XL-but-shippable on existing assets. The self-correcting-quote AI is a **later** phase that needs months of collected data first - we never promise it at launch.

🏆 2. Standing Orders & Print Subscriptions - “*turn repeat work into automatic monthly revenue*”

What it is: A “**Set it on repeat**” button on any past order. The owner (or a B2B buyer, self-service) picks a cadence (monthly, quarterly, when stock runs low), an artwork rule (reuse the exact file, or swap a date/promo field), and a payment method. Each cycle the system auto-clones the order **at the locked-in**

price, runs it through the existing print-job and imposition pipeline, and only pings a human when something genuinely needs a decision. The owner gets a **Recurring Revenue board**: monthly recurring revenue, the next 30 days of scheduled jobs, and any paused/churned subscriptions surfaced in the Action Center.

Why no competitor has it: Every competitor stops at a customer-clicked “reorder” button. None turn repeat work into **unattended subscription revenue** with locked pricing, auto-billing, and a job that materializes on the floor by itself. For a shop where **40–60% of revenue is repeat work living in the owner’s head**, this is the single biggest stickiness and revenue upgrade - a buyer on a standing order stops shopping around.

How we build it (the smart shortcut): Scope v1 to **B2B-credit customers**, because the hardest piece - off-session billing - is already solved there: `CreditService.chargeOrder()` is a clean programmatic draw-down. The scheduler copies the proven recurring-campaign cron in `routes/console.php`. Order cloning is trivial - `Order` already snapshots customer/address/currency/price (so price-locking is free), and `CartItem` already carries `design_id`, `uploaded_files`, `selected_options` (so “reuse exact file” is a field copy).

First milestone: B2B-credit standing orders with “reuse exact file” + auto-created print jobs + the Recurring Revenue board. Category-leading on its own, at **M effort**, because it skips the hard payment work.

Effort: M for the B2B-credit v1. The storefront card-on-file path (Stripe/Razorpay vaulting, SCA, dunning, retries) and variable-data field-swap are a separate **XL** phase, sequenced later - never bundled into v1.

🏆 3. Gang-Run Autopilot - “tells you the dollar-optimal moment to print”

What it is: Every compatible incoming order (same stock, same press sheet, same finishing, overlapping due date) auto-drops into a shared “**Shared Sheet**” pool. The system continuously packs the pool against real press sheets and shows one plain line:

“You have 41 of 48 slots filled on a 25×38 gloss sheet - printing now wastes 14%. Waiting for ~7 more business-card jobs (about 6 hours at your order pace) drops waste to 2% and saves ~\$38. Fire now or hold?”

It auto-fires on a rule the owner sets (sheet X% full **OR** a job’s deadline cutoff - whichever comes first, so **nothing ever ships late**), splits the true sheet cost across each customer, and deducts the real sheet count from inventory.

Why no competitor has it: Competitors have “smart imposition,” but it’s a manual, one-batch-at-a-time operator action. Nobody ships a **continuous economic queue** that predicts fill-time from the shop’s own order velocity with a late-job safety valve. POD players structurally can’t - they don’t own the press. Half-empty sheets are the single biggest source of short-run media waste.

How we build it (and the honest gap): The hard math is **already built and unit-tested** - `GangRunPlanner` does real bin-packing against a real `PressSheet` and computes waste %,

GangCostSplitter splits true per-piece cost, **SheetCostCalculator** / **WasteCalculator** give the dollars. The honest gap we scope *into* the build: today a **PrintJob** has no press-sheet/stock/finishing binding or due date to match on. So phase one gives each order item a matchable stock + sheet + finishing + finished-size + due-date (leveraging the existing **ImpositionTemplate** mapping as the bridge), plus a press-sheet/paper-stock inventory model. Then the continuous pool + auto-fire is a scheduler job orchestrating services that already return the exact numbers.

First milestone (read-only first): A “**Shared Sheet**” board showing poolable jobs grouped by sheet, live fill %, waste %, and the dollar savings of waiting - **without auto-fire**. The owner manually clicks “fire now.” This proves the economics and the matching data before we hand the system the trigger.

Effort: XL - the engine is done, but the production-data spine and press-sheet inventory are real net-new work. Best first customer: **sheet-fed short-run shops** (cards, labels, stickers).

Why build them in this order

1. **Ledger first** - it's the compounding data asset that makes everything else smarter, and the one insight competitors *cannot copy by next quarter*. Every reconciled job teaches the system the shop's real costs and reprint causes. That data later sharpens Standing Order pricing (lock in a price that's actually profitable) and Gang-Run economics (true sheet cost per customer). We ship its **report before its AI**, so data collection starts on day one while we deliver a switch-platforms insight immediately.
2. **Standing Orders second** - fastest path to felt value at lowest risk. Scoped to B2B-credit it's **M-effort with zero new payment code**, and “hidden repeat work → visible recurring revenue” is the financial headline a non-technical owner understands instantly.
3. **Gang-Run Autopilot third** - engine's done, but it needs the most net-new infrastructure and carries the sharpest failure mode (one wrong “hold” that ships a job late destroys trust). It benefits from the operator-data-entry habit we build at the job-completion seam in the Ledger, and we de-risk it by shipping read-only first.

The through-line for all three: take production reality we already capture, turn it into plain-dollar decisions, and **never ship the AI/automation half until the data underneath it is trustworthy**.

Fast-followers (strong, but after the flagships)

- **True-to-Print Finish Studio** - let buyers see *their own* artwork with believable foil, spot-UV, soft-touch and painted edges in the existing 3D preview, then approve bindingly. Drives AOV on the highest-margin SKUs. (**useARPreview.ts** already does PBR material swaps; **ProofService** already does binding sign-off.) **Drop the calibrated-CMYK-color claim - the stack can't honestly deliver it.**
- **Personalize-at-Scale Self-Serve (VDP)** - buyer designs one template, uploads a name list, gets N proofs, approves once. Unlocks HR cards, event badges, franchise kits. Pairs beautifully with Standing

Orders. Honestly XL (dynamic-field binding, per-row render pipeline, VDP-aware imposition are all net-new).

- **B2B Prepaid Wallet + Volume Commitments** - “Buy \$10,000 of credit, get \$11,000 to spend” + annual volume tracking. Collects cash upfront on the existing **CreditService** ledger. **We explicitly cut BNPL/financed checkout** - regulated lending is the wrong business for a print shop.
- **Verified Carbon Footprint (B2B Scope-3 statement)** - a retrospective per-account annual sustainability statement built from real consumed-sheet history. The defensible moat for shops serving corporate buyers who legally need supplier data. Any storefront badge stays clearly labeled an *estimate*, not “verified.”

Parked, with honest reasons

Idea	Why parked
Talk-to-Your-Shop AI operator	No tool-calling loop, zero rollback in the codebase; unsafe writes a non-technical owner can't sanity-check. Revisit once the Ledger and Autopilot prove safe-write patterns.
MCP storefront for AI agents	Real moat, near-zero buyer impact in 2026. Cheap optionality, not a flagship.
WebAR “see it in your space”	No glTF/USDZ export, crude meshes, no mature pure-Node iOS path. Vanity toggle.
Closed-Loop Prepress Co-Pilot / Zero-Config Autopilot	Scored 0/40. But the Ledger's reprint-cause data is the natural feeder for a <i>future</i> prepress fix-it loop - door kept open as a downstream beneficiary, not a standalone bet.

Full council scoreboard

Scored out of 40 (Differentiation + Shop-owner impact + Feasibility-on-our-stack + Moat, each /10):

Rank	Concept	Score	Diff	Impact	Feas	Moat	Effort
1	True-Cost & Reprint Ledger	28	8	8	6	6	XL
1	Standing Orders & Subscriptions	28	8	8	7	5	XL → M(v1)
3	B2B Cash-Forward Engine	27	7	8	6	6	XL
3	True-to-Print Finish & Color Studio	27	8	7	6	6	XL
5	Gang-Run Autopilot	26	8	7	5	6	XL
5	Verified Carbon Footprint	26	8	5	7	6	L
5	Personalize-at-Scale (VDP)	26	8	7	5	6	XL
8	Quote-Aware MCP Storefront	24	8	4	6	6	XL
9	WebAR “See It In Your Space”	24	7	6	6	5	L
10	Talk-to-Your-Shop operator	20	7	6	4	3	XL
-	Closed-Loop Prepress Co-Pilot	0	-	-	-	-	cut
-	Zero-Config Autopilot	0	-	-	-	-	cut

Generated by an adversarial multi-agent R&D council. Scores are the average of a skeptical-VC and an engineering-lead panel per concept. Every “how we build it” claim is grounded in modules that already exist in this codebase.